From the Black Grail to the Command Spell System: The Discrete Nature of Long-Horizon Practice and the Boundaries of Agent Governance

Sammy Zeng

2026

[1]Contact: jop.sammy@163.com ORCID: 0009-0007-4382-0040

[2]This paper is a research outcome of the "Long-Horizon Practice Theory" project, which received specific support from the Qianjiang New City Central Business Industrial Community of Shangcheng District, Hangzhou. The community provided critical support in research coordination, foundational stress-testing examples, and real-world business use cases (non-financial sponsorship).

Abstract

This paper answers a simple question: how do we build better Agent systems?

Source-code-level investigation of seven mainstream open-source Agent systems reveals that the model invocation protocol (Prokaryon) has converged within ± 20 lines of code; the true watershed is the governance infrastructure (Eukaryon)—state governance maintains continuity across time, and power governance defines the boundary of action with the external world. This paper proposes a Three-Layer Concentric Model (Prokaryon/Eukaryon/Cortex) as an analytical framework, and derives seven core engineering decision nodes based on a formal definition of long-horizon practice. The core judgment: when a task involves non-precomputable sequential dependencies, governance mechanisms such as discrete checkpoints, permission gates, structured state handoff, and machine-to-human requests are not engineering options—they are structural necessities.

One layer deeper: progress in governance cannot come from automatic optimization—"what is right" in an Agent system does not exist prior to execution; it can only be observed and learned from failures in real deployment. This is the deep reason why Agent systems follow an "evolutionary" rather than a "designed" path. Governance is not a shackle on capability—on the contrary, it is the structural prerequisite for unleashing the intelligent potential of LLMs.

Keywords: Agent Governance; Long-Horizon Practice; Three-Layer Concentric Model; Discrete Nature; Human-Machine Collaboration

Contents

1 Introduction

1.1 The Current State of the Agent Ecosystem and Its Pressing Question

All this research, all this first-principles thinking—what is it for? It is for building better Agent systems.

As of July 2026, Agent systems with LLMs as their core reasoning node have entered an unprecedented period of flourishing. Seven independently evolved Agent coding systems, each with tens of thousands of real users and thousands of person-years of engineering investment, exhibit radically different architectural choices on a similar technical substrate.

Yet this flourishing is accompanied by profound fragmentation. The seven systems differ by only ± 20 lines of code in their core loop, yet have diverged in sharply different directions on dimensions such as state persistence, permission pipelines, tool governance, context compression, and cross-session experience accumulation. These differences are not "preferences"—they are mutually irreducible structural fossils left in engineering history by each system's initial environmental conditions (untrusted execution environments, high-value human attention, state unreliability, and cross-session knowledge evaporation).

This fragmentation poses a sharp question: **Where exactly do the real differences among Agent systems lie?** Is it model reasoning capability? The richness of the tool ecosystem? The sophistication of prompt engineering? Or some deeper architectural dimension that has yet to be fully articulated?

1.2 The Evolution and Blind Spots of Existing Surveys

Before answering the above question, it is necessary to acknowledge the contributions and boundaries of existing survey literature on this topic. Over the past three years, three landmark surveys have systematized Agent research from different perspectives, but each has left a structural blind spot on the question of "where the differences lie."

The first is the Foundation Agents survey proposed by Bang Liu et al. (2504.01990)[1]. Taking the functional architecture of the human brain as a blueprint, this survey decomposes an Agent into seven modules—cognition, memory, world model, reward, emotion, perception, and action—and provides formal definitions and current technical status for each module. This is the most comprehensive neuroscience-guided framework to date for "what components an Agent ought to have." However, its discussion stops at the functional component layer—it answers "what an Agent ought to have," but does not answer "how these components should be organized into a runtime system that is persistent, recoverable, and governable." All seven systems possess some implementation of all seven modules—yet their engineering reliability varies dramatically. A functional checklist cannot explain this gap.

The second is the LLM-based Autonomous Agent survey proposed by Wang et al. (2308.11432)[2], which first proposed the Profile–Memory–Planning–Action four-component unified architecture, establishing a shared functional classification language for the Agent field. Nearly all subsequent Agent papers describe their architectures within its terminological framework. However, this survey's classification belongs to the **cognitive functional layer**—it concerns what cognitive activities an Agent *performs* during a task (remembering, planning, acting), not the **runtime organizational layer**—how these activities are governed (persistence, recovery, approval, isolation, handoff). An Agent can perfectly "plan" its next step while simultaneously executing an irreversible operation without authorization due to the absence of a permission approval mechanism. Cognitive function classification cannot diagnose this failure.

The third is the Agent Harness survey proposed by Meng et al. (2605.29682)[3], which for the first time formalizes the Agent's runtime infrastructure as a six-tuple $H = (E, T, C, S, L, V)$ and explicitly advances the core thesis of "harness-as-infrastructure problem": the reliability of Agent deployment depends increasingly not on the underlying model, but on the runtime infrastructure layer that encapsulates the model. Taking functional completeness as its analytical axis, this survey systematically compares 23 representative systems across six components. However, its analytical framework does not derive *which* governance mechanisms are structural necessities from the information structure of long-horizon practice, or *why* their absence causes structural fractures and where. Functional completeness tells you "what components a system has," but cannot tell you "which components transition from enhancement to necessity under what conditions."

Placing the contributions and blind spots of the three surveys side by side reveals a clear evolutionary trajectory: Liu et al. established a component checklist of "what there ought to be," Wang et al. established a functional classification of "what cognitive activities are being performed," and Meng et al. established a completeness matrix of "what infrastructure it runs on." Yet they share the same blind spot—none derives, from the information structure of long-horizon practice, *why* a set of runtime governance constraints are **structural necessities** rather than **engineering options**. This is precisely the gap this paper seeks to fill.

Relationship to neighboring fields. The governance framework of this paper conceptually intersects with checkpoint/recovery in distributed systems, the principle of least privilege in capability security, and human-in-the-loop research in HCI. However, the core assumptions of these fields differ structurally from the conditions of Agent long-horizon practice—Agent outputs are non-deterministic, failure modes cannot be exhaustively pre-enumerated, permission boundaries only gradually emerge during execution, and operators facing the H2 Dilemma often cannot independently evaluate system outputs. This paper's contribution is to derive, from these structural differences, a set of governance constraints that hold under the information-incomplete conditions unique to Agents, rather than to deny the value of neighboring fields.

1.3 The Starting Point: Source-Level Investigation of Seven Systems

The empirical foundation of this paper is not a secondary aggregation of existing papers, but a source-code-level investigation of seven mainstream open-source Agent systems. The seven systems are Claude Code, Codex CLI, OpenCode, Hermes, Cline, Cherry Studio, and trae-agent. The source-code investigation covers each system's core loop, state persistence, permission pipeline, tool governance, context compression, capability extension, and internal communication mechanisms, with every system's description supported by evidence at the level of specific file paths, function signatures, and configuration items. The selection criteria for the seven systems are: having a real user base, accessible source code, and coverage of different initial scenarios (terminal / IDE / standalone service / desktop / research).

A key finding of the source-level investigation is that the core architectures of existing Agent systems are all evolutionary products—§4 will unfold the full argument for this judgment.

1.4 The Positioning of This Paper

Based on the above analysis, this paper proposes a Three-Layer Concentric Model—Prokaryon (the invocation protocol skeleton, §5), Eukaryon (the governance infrastructure, §6), and Cortex (the progressive capability disclosure layer, §8)—to dissect the runtime architecture of Agent

systems. This is not a repudiation of existing surveys, but a complementary analytical dimension from an angle they have not yet addressed.

One layer deeper, this paper argues: Agent governance is essentially about **defending the lower bound, not raising the upper bound**—ensuring that the system does not fracture when information is incomplete and value is uncertain. But it cannot substitute for human judgment, value trade-offs, and external reality anchoring (§9.2).

1.5 What Is a Good Agent

The starting point of this paper is a plain aspiration: to build better Agent systems.

What does "better" mean? The yardstick for answering this question is not benchmark scores, not user counts, but whether it better unleashes the potential of LLMs—enabling LLMs to exert more effective causal intervention on the real world, and ensuring that these interventions do not deviate from human intent and values. This is this paper's definition of a good Agent.

But once this goal lands in engineering, it immediately becomes sharp. A model is smart enough in single-step inference—it can comprehend a complex codebase, it can generate precise manipulation instructions. But when it needs to execute a task continuously over hundreds of steps, and the outcome of each step cannot be precomputed, the question is no longer "can the model figure it out," but "can the system avoid losing control over the long horizon"—no matter how strong the model, it cannot conjure from thin air truth values that have not yet been produced by the world; no matter how deep the reasoning, it cannot automatically recover task direction after context overflow.

This is where the core judgment of this paper lies: when the base model is already good enough, the real differences among Agent systems lie not in the model itself, but in the governance structure. The source-code investigation of seven mainstream open-source systems repeatedly confirms this—they differ by only ± 20 lines of code in their invocation protocol, yet diverge sharply on state persistence, permission pipelines, and cross-session knowledge. The Discrete Nature of long-horizon practice determines that mechanisms such as discrete checkpoints, permission gates, structured state handoff, and machine-to-human requests are structural constraints for maintaining continuity under non-precomputable sequential dependencies.

Thus, this paper's answer is: **A good Agent system is one with a complete governance structure.** Governance is not a shackle on capability—on the contrary, it is the structural prerequisite for unleashing the intelligent potential of LLMs. Capability lets you go faster; governance determines whether you can go the full distance. The argument of the following fourteen chapters proceeds from the Three-Layer Concentric Model, through the formal definition of long-horizon practice, the analysis of its Discrete Nature, to the revelation of structural incompleteness in feedback signals, converging layer by layer toward a single core judgment. Each step answers that most plain question: how do we build Agent systems that truly unleash the potential of LLMs?

It should be noted: this paper provides a complete engineering framework and formal argument on the dimension of governance as lower-bound defense. How to make an Agent's single-step intervention capability itself stronger—better model reasoning, richer tool ecosystems—lies outside the scope of this paper's argument. This paper answers "where reliability comes from," and leaves "how to make capability stronger" to future work.

1.6 Main Contributions

The main contributions of this paper are as follows:

1. **Source-code verification of Prokaryon convergence across seven systems**: Through source-code-level comparison of seven systems—Codex CLI, Claude Code, OpenCode, Hermes, Cline, Cherry Studio, and trae-agent—with evidence at the level of specific file paths and line counts, this paper establishes the empirical fact that "Prokaryon has converged across all systems within ± 20 lines of code," and points out that under the current API invocation paradigm, the design space of the Prokaryon is approaching saturation—seven systems independently evolving to nearly identical structures strongly suggests that this layer has no remaining room for substantive architectural innovation (§5).

2. **The Eukaryon dichotomy framework (State Governance and Power Governance)**: Proposes a State Governance / Power Governance dichotomy within Eukaryon, and argues for their mutual irreplaceability in diagnosis, prediction, and remediation paths (state fractures cannot be repaired by permission rules; power violations cannot be repaired by persistence), supported by empirical evidence from the three-layer structure of permission pipelines (tool visibility / permission approval / execution isolation) and the persistence strategy spectrum (L0–L3) across the seven systems (§6).

3. **Long-Horizon Practice Theory explains the structural necessity of Eukaryon**: Systematically maps the core conclusions of Long-Horizon Practice Theory (the formal definition of non-precomputable sequential dependency chains, four sources of information truncation, three-layer structural lower bounds) into the context of Agent governance, demonstrating that Eukaryon's core governance mechanisms (HC-3 state handoff, SC-1 value closure, SC-3 unclosed-item marking, EC-7 machine-to-human requests) are upgraded from engineering options to structural necessities under the condition $K_{i+1} \notin \text{Fold}(I_i)$ (§9).

4. **EC-6/EC-7 derivation of presentation layer convergence**: Starting from the information structure of the two engineering compensation conditions EC-6 (proactive machine-to-human explanation) and EC-7 (proactive machine-to-human request), derives that any presentation layer satisfying both conditions must simultaneously meet three engineering conditions—high information density, low operation latency, and state traceability—and argues that under 2026 technological conditions, the functional form simultaneously satisfying these three conditions converges with high probability toward an IDE-/workbench-like form. What converges is the functional form, not the product form (§11).

5. **Systematic revelation of structural incompleteness in feedback signals**: Reveals the structural chasm between Agent governance feedback and LLM training feedback, proposes a complete framework with four-fold non-closure—objective semantics, evaluation dimensions, consequence space, and feedback attribution—as root causes, and argues that this structural incompleteness is the fundamental reason why all seven Agent systems' frameworks follow an "evolutionary" rather than "designed" path (§12).

Furthermore, in the Discussion chapter (§14), this paper presents five design principles and three operationalizable metrics for long-horizon benchmarks, a structural analysis of the H2 Dilemma, and an honest exposure and directional discussion of subsequent open problems (decidability of practice closure, standardization of tool action semantics, general pluggability vs.

platform-deep specialization, and the transformation from semantic non-closure to stable feedback).

Quick-Reference Glossary

The following lists key terms used in this paper, along with the chapter where each term first appears. For numbering-system terms, full definitions are given in the corresponding chapter body.

Term	Brief Definition	First Appears
Prokaryon（原核）	The invocation protocol skeleton of an agent system—while loop + tool invocation + context management	Chapter 2 (Decision Framework); full definition in Chapter 3
Eukaryon（真核）	The governance infrastructure of an agent system—state persistence, permission pipeline, approval gates	Chapter 2 (Decision Framework); full definition in Chapter 6
Cortex（皮层）	The progressive capability disclosure layer of an agent system—exposing the right tools at the right time	Chapter 2 (Decision Framework); full definition in Chapter 8
HC-1	Purpose Closure—every checkpoint must have an actionable closure criterion	Chapter 6
HC-2	Authority Closure—irreversible operations must have prior approval	Chapter 6
HC-3	State Closure—state must be transferable across checkpoints	Chapter 6
SC-1	Value Closure—local-purpose correctness cannot be judged by the purpose itself	Chapter 6
SC-2	Order Closure—a task path must not become irreproducible due to execution order	Chapter 6
SC-3	Unclosed Marking—the system must distinguish three states of closure	Chapter 6
EC-1	Recovery & Rollback—the system must be recoverable from any checkpoint	Chapter 9
EC-2	Isolation & Boundaries—execution at one checkpoint must not pollute other checkpoints	Chapter 9
EC-3	Independent Evaluation—closure judgment cannot be performed by the executor	Chapter 9

Term	Brief Definition	First Appears
EC-4	Instruction-Following Compensation—the system must compensate for non-deterministic deviations by the LLM	Chapter 9
EC-5	Capability-Gap Escalation—when the agent lacks the capability to complete a task, it must proactively escalate	Chapter 9
EC-6	Machine-to-Human Proactive Explanation—the system must provide actionable, structured context when triggering human intervention	Chapter 9
EC-7	Machine-to-Human Proactive Consultation—the system must proactively request human arbitration when value judgments are uncertain	Chapter 9
$K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$	Key information generated after step i cannot be statically folded before step i—formal criterion for long-horizon practice	Chapter 3
L0–L4 Persistence Spectrum	The five-tier spectrum of state persistence from no persistence (L0) to full audit trail (L4)	Chapter 9
H2 Dilemma	The structural dilemma in which a human operator lacks the full domain knowledge required to independently verify the agent's output	Chapter 13

Table 1: Quick-reference glossary of key terms in this paper

2 What It Takes to Build a Good Agent System

The Introduction established the core judgment that "the differences among Agent systems lie in governance." This chapter unfolds that judgment into a set of decision frameworks that can be landed in engineering—it answers not "why governance matters," but "what specifically needs to be done to build a good Agent system."

This chapter places the conclusion first. Before the subsequent chapters unfold their arguments layer by layer, this chapter first presents, in the language of a practitioner, the decision framework for building a good Agent system: one pre-filter question, plus seven core decision nodes, along with the honest boundaries of this framework—what it cannot do, under what conditions it does not apply, and what questions currently have no answers.

2.1 Decision Zero: Are You Building an Agent, or Something Else?

Why. Using the wrong tool for the wrong scenario is the most expensive mistake you can make. Before considering "how to build an Agent," you need to first confirm: does your scenario genuinely need an Agent?

Must do. Confirm that your system needs to execute irreversible external actions (modify files, call APIs, commit code changes), and that subsequent steps depend on the actual results of those actions. If the problem you need to solve can be accomplished by a deterministic function call, an if-else branch, or a fixed script sequence, you do not need an Agent. What you need is a faster function, not a smarter loop.

Should do. Before deciding to deploy an Agent, spend half an hour confirming: does "what to do next" in your scenario genuinely depend on new information produced after the previous step executes—specifically, information that could not have been precomputed during the planning phase? If yes, you have entered the legitimate territory of Agents. If not, the complexity introduced by an Agent framework will not make your product better—it will only make your engineering messier.

Must not do. Do not wrap deterministic dead code as an Agent. Not every system that "calls an LLM" automatically becomes an Agent. When you find that your Agent follows the same reasoning path and makes the same tool-call sequence 90% of the time, what you need is not "better Agent prompt engineering"—it is extracting that 90% into a non-Agent function.

Positioning of this chapter. Decision Zero is the framework's pre-filter condition and is not counted among the seven core decision nodes. The entire paper uniformly uses "seven decision nodes" as the count. If you pass this filter (your system genuinely needs to execute irreversible external actions, and subsequent steps depend on the actual results of those actions), then the following seven decision nodes are what you should consider, item by item.

2.2 Decision One: Establish State Governance

Why. If your system, after executing step five, does not remember what happened at step four, it is structurally not a long-horizon system, but a series of mutually amnesiac short-horizon fragments.

Must do. Persist structured state snapshots for your system. These must contain at least four fields: (1) what step has been reached; (2) why that step was taken at that moment (not merely "what was done"); (3) what pending items remain unclosed; (4) where to resume from. All four fields are indispensable—a multi-field snapshot is what allows a successor to reconstruct the execution context, rather than merely knowing they "saw the output of step N."

Should do. State snapshots should be automatically written to persistent storage after each key checkpoint, and should be reliably readable and recoverable after system crashes, restarts, or context window overflows. Ideally, state snapshots should use a human-readable format—if the successor is a human rather than a machine, they should be able to understand what happened before.

Must not do. Do not confuse "memory" (RAG retrieval, vector stores, conversation history summaries) with "state." Memory tells you "what topics were discussed yesterday"—helpful for review, but no substitute for "where we currently are, where we are stuck, and what to do next." Do not mistake fluent streaming output for cross-step continuity. Streaming output is fluency at the display layer, not continuity at the execution layer.

Full argument in §6.

2.3 Decision Two: Establish Power Governance

Why. A system capable of executing irreversible operations, without operational boundaries, is not "more powerful"—it is more dangerous. Not because the model "might turn bad," but because information incompleteness is the norm in long-horizon practice. Under conditions of information incompleteness, pre-specifying all the "things that should not be done" is mathematically impossible.

Must do. Establish an explicit permission pipeline for your system. At minimum, two layers: (1) the set of permitted operations (which tools, which resource scopes); (2) a mandatory approval gate before irreversible operations: before executing functions such as file deletion, external API write operations, or code merging, pre-set a human approval step. Your system should not execute irreversible operations without human confirmation.

Should do. When the system faces a judgment of "is it worth doing" rather than "can it be done," it should transform the question into a form where the human can choose—not throwing back "what do you think," but presenting 2–4 structured options, each with verifiable consequence estimates. This is behavioral transformation: from making humans fill in blanks, to letting humans answer multiple-choice questions.

Must not do. Do not use "the model cannot see a certain tool" as a substitute for "the system will not execute that tool"—the model's tool visibility is not a security boundary. Do not substitute vague norms like "the Agent should be safer" or "the Agent should align with human values" for pre-operation approval gates—norms prevent no operations; approval gates do. Do not substitute "we trust the model" for "we structurally distrust any single inference."

Full argument in §6.

2.4 Decision Three: Checkpoint Decomposition and Independent Verification

Why. The most dangerous failure mode of an Agent is not "it did nothing"—it is "it advanced efficiently in the wrong direction." When each step's output looks reasonable but the overall direction has already drifted, a system without independent verification will only discover the drift upon reaching the endpoint.

Must do. Decompose tasks into independent checkpoints, each with an operationalizable closure criterion: "all test suites pass" rather than "feels about right." Closure judgment must be borne by a verification mechanism independent of the executor (automated tests, compilation checks, checksums) and must not rely on the Agent saying "I'm done."

Should do. The granularity of checkpoints should be designed to be "coverable within a single span of human attention"—the output volume of one checkpoint should not exceed what you can review in one sitting. Checkpoint closure criteria should be defined in advance, not negotiated on the fly during execution.

Must not do. Do not rely on the Agent's self-report "I have completed the task." The gap between the Agent telling you "it's done" and you verifying "it's done" is the single largest risk exposure in nearly all long-horizon Agent systems.

Full argument in §9.

2.5 Decision Four: Design Interruption Boundaries

Why. The amount of human attention available to monitor Agent execution is finite. If your system has no pre-set interruption points, when human attention is exhausted, the system enters unsupervised execution—and no governance mechanism can function under unsupervised conditions.

Must do. Explicitly create interruption surfaces at key decision nodes: after a subtask completes, when an anomalous signal is detected, when a value judgment is needed, when execution enters an uncovered region. These interruption surfaces are pre-set, not improvised. They are structural features of your system design paired with governance mechanisms.

Should do. The positions of interruption surfaces should be defined by "under what circumstances must execution stop and wait for a human response," not by a mechanical frequency of "interrupt every N steps." Key interruption surfaces include: before irreversible operations (paired with Decision Two's approval gate), and at subtask closure judgment (paired with Decision Three's independent verification).

Must not do. Do not assume "I'll interrupt when I need to." Human operators will not continuously monitor Agent execution while maintaining focused attention. This is a structural feature of human attention, not a self-discipline problem. Do not treat "uninterrupted fluent conversation" as an architectural goal: fluency is a property of the user interface, not a reliability guarantee for the Agent system.

Full argument in §10 and §11.

2.6 Decision Five: Separate Capability from Governance

Why. If you restrict a sub-Agent's capabilities and tools because you do not trust it, you are using "making it dumber" to deal with "not knowing whether it will do something wrong." This is not governance—it is self-neutering. In a multi-Agent system, the correct answer to insufficient trust is not to restrict capabilities, but to make the governance layer independent of the capability layer.

Must do. Divide your system's architecture into two layers: the governance layer (verification, approval, boundary enforcement) and the capability layer (tool invocation, reasoning, code generation). The governance layer constrains "what is received, what is verified, and under what conditions it is released," not "what tools the sub-Agent can invoke or what model it can use for reasoning." The sub-Agent should possess all the computational power and tools needed to complete its task. The governance layer's responsibility is to judge whether its output meets release conditions, not to restrict the capabilities of its productive process.

Should do. Pre-set explicit verification rules in the governance layer—code must pass compilation checks before merging, configuration changes must pass diff review before deployment, data migrations must be verified in a sandbox before execution. These rules are operational for

the governance layer and transparent to the execution layer: the sub-Agent does not need to know the verification rules exist; it only needs to produce results that can be verified.

Must not do. Do not use restricting the sub-Agent's tool set to cope with insufficient trust. When you say "I dare not let the sub-Agent access the file system," you are actually saying two things: (1) you need to add a file-system operation approval gate in the governance layer; (2) you should not delete the file-system tool in the capability layer. Collapsing these two operations into a single action—deleting the tool—means your system has permanently lost the opportunity to exercise this capability once governance is in place.

Full argument in §8 and §13.

2.7 Decision Six: Acknowledge That Governance Cannot Be Automatically Optimized

Why. You cannot design an Agent governance system that can automatically optimize itself. Not because the algorithms are not good enough, but because the signal for evaluating whether a governance decision is good cannot be automatically obtained at the moment execution completes. Governance quality can only be judged after the fact, by humans, based on actual consequences.

Must do. Accept "manual gradient descent" as the only governance iteration paradigm: observe your system's failure modes in actual operation → diagnose failure signals → modify governance rules or architecture → verify that the modification prevents the same class of failure. Every step of this loop requires human judgment and cannot be automated.

Should do. Design observability for your governance system: record the context each time an approval gate is triggered, the execution trajectory each time a task drift is discovered, and the change in system behavior after each human intervention. These records are your training data for executing manual gradient descent.

Must not do. Do not wait for "an algorithm that automatically optimizes Agent governance" to appear. It will not appear. Not because 2026 technology is insufficiently mature, but because the optimization target of "governance quality" is itself semantically non-closed. You cannot define an automatically computable loss function to measure "whether this governance decision prevented future unforeseen deviations."

Full argument in §12.

2.8 Decision Seven: Know What Is Impossible

Why. An Agent system capable of autonomously completing a value loop is structurally untenable. The so-called "value loop" refers to: the Agent independently completes the entire process from execution to value judgment, without human judgment as the terminal of the verification chain. This is not a question of technical maturity—it is that the preconditions required by this loop (complete information, determinate value, automatable verification) are structurally mutually exclusive with the real conditions under which Agents operate.

Must do. Acknowledge that your Agent system's design goal is to defend the lower bound, not to raise the upper bound. Defending the lower bound means: ensuring the system does not collapse when information is incomplete, does not unilaterally adjudicate when value is uncertain, and can stop when execution boundaries drift. Raising the upper bound—making the optimal decision under given conditions—is the responsibility of the operator (the human), not the system.

Should do. When designing each governance mechanism, ask yourself two questions: (1) What failure mode does this mechanism prevent—specifically, the mode where the system "completes the task in a self-consistent but deviant way"? (2) If no one monitors this mechanism, under what conditions would it be bypassed?

Must not do. Do not pretend that an Agent can make value decisions on behalf of humans. When your system says "based on the analysis, I recommend adopting Option A," it is giving advice. This is a legitimate operation. When your system says "Option A has been executed" without human approval, it is making a value decision. This is unacceptable agentic overreach.

Full argument in §14 (§14.2).

2.9 Decision Framework Summary

Table 2 summarizes all eight decision points, including the pre-filter condition (Decision Zero) and the seven core decision nodes (Decisions One through Seven).

2.10 Boundaries of the Framework: An Honesty Statement

Every framework has its own implicit premises and inapplicable scope. Long-Horizon Practice Theory is an inductive framework, not an axiomatic system—its conclusions are inductively derived from empirical observation of seven open-source coding Agent systems, and cannot be demanded to be deductively derived from a set of axioms. What follows are three pre-adoption notices you should understand before adopting the seven decision nodes of this chapter.

1. **Empirical basis.** All empirical evidence on which the seven decision nodes are based comes from source-code-level analysis of seven open-source coding Agent systems ($n = 7$), not a general empirical foundation. If you deploy Agents in non-coding domains, the applicability of the decision nodes requires independent domain validation.

2. **Scope of applicability.** Decision Two (Power Governance) requires human approval gates, Decision Four (Interruption Boundaries) requires human intervenability, and Decision Six (Feedback Loop) requires human observation and diagnosis. These all implicitly assume that the deployment team has sufficient organizational scale to bear governance costs. If you are a single person working on a personal project, the full seven nodes may be over-engineered.

3. **Necessary but not sufficient.** The seven decision nodes are necessary conditions for building a good Agent system, not sufficient conditions. Satisfying all seven nodes does not make your system "successful"—it only prevents the system from structurally fracturing due to a lack of governance. Governance is the engineering of defending the lower bound, not the magic of raising the upper bound.

The complete boundary discussion of the framework (including the systematic unfolding of evaluation boundaries, cognitive boundaries, and engineering boundaries, as well as four open problems) is in §14.

3 Definition and Boundaries of Narrow Agent

Chapter positioning. The core question of this paper is "how to build a good Agent," and the prerequisite for answering that question is answering "what an Agent is." Before considering state governance, power governance, and interruption boundaries item by item, a unified conceptual yardstick is needed to discuss "Agent systems." This chapter answers a question prior to

Table 2: Decision Framework for Building a Good Agent System

Decision Node	Must Do	Should Do	Must Not Do
Zero: Is this an Agent?	Confirm the system executes irreversible external actions and subsequent steps depend on actual execution results	Spend half an hour first confirming whether the scenario genuinely needs an Agent before starting work	Wrap deterministic dead code as an Agent
One: State Governance	Persist four-field structured state snapshots (where we are, why, unclosed items, resume entry point)	Use human-readable format for state snapshots; auto-write after checkpoints	Confuse "memory" with "state"; mistake fluent streaming output for system continuity
Two: Power Governance	At least the first two layers of the three-layer permission pipeline; mandatory approval gate before irreversible operations	When value is uncertain, present 2–4 structured options and let the human choose	Substitute "model cannot see a tool" for "system will not execute that tool"; substitute vague safety norms for pre-operation gates
Three: Checkpoints & Verification	Each subtask has an operationalizable closure criterion; closure judgment borne by an independent verification mechanism	Design checkpoint granularity to be coverable within a single span of human attention	Rely on the Agent saying "I'm done"
Four: Interruption Boundaries	Explicitly create interruption surfaces at key nodes; pre-set human intervention insertion points	Define interruption surface positions by "under what circumstances execution must stop"	Assume "I'll interrupt when needed"; treat "never interrupt" as an architectural goal
Five: Capability–Governance Separation	Constrain the governance layer's reception and verification, not the sub-Agent's reasoning depth and tools	Pre-set explicit, operationalizable verification rules in the governance layer	Use restricting the sub-Agent's tools and depth to cope with insufficient trust
Six: Feedback Loop	Accept manual gradient descent as the only iteration paradigm; design "observe $\rightarrow$ diagnose $\rightarrow$ modify $\rightarrow$ verify" loop	Design observability for the governance system: record approval contexts, deviation trajectories, post-intervention changes	Wait for "an algorithm that automatically optimizes Agent governance" to appear
Seven: Insurmountable Boundary	Acknowledge human judgment as the terminal of the verification chain—defend the lower bound, not raise the upper bound	When designing each governance mechanism, ask yourself: what failure mode does it prevent? What happens without monitoring?	Pretend the Agent can make value decisions on behalf of humans

all empirical discussion: when we say "Agent system," what exactly are we talking about? This requires clear definitional boundaries. Without a unified yardstick, cross-system comparison is impossible; without targeted scenario demarcation, the design of governance mechanisms cannot be grounded; and survey papers inevitably fall into the trap of attacking others' definitions.

The act of narrowing itself is one of the core argumentative contributions of this paper.

3.1 The Three-Step Derivation of Narrowing

Step One: Acknowledge the General Agent as a "Perception–Action Loop"

Starting from Observer-Agent theory[4], the common substrate of a general intelligent Agent is the "Perception–Action Loop": a system perceives its environment, makes judgments based on perception, changes the environmental state through action, then perceives the new state, and so on in a cycle. This definition is correct at the abstract level—it covers the entire spectrum from thermostats to autonomous driving systems. A thermostat perceives temperature, judges whether it deviates from the set point, and starts or stops heating—it too is a Perception–Action Loop.

But precisely because this substrate is so broad, it cannot help us distinguish between two classes of systems that are structurally different in their information properties. This is the tension of the general definition: it withstands the test of time (not tied to a specific technology stack), yet cannot provide analytical discriminatory power.

Step Two: Narrow by the Criterion of "Executing Real-World Value"

The key dividing line is not "whether there is action," but whether the action crosses the boundary from the virtual to the real.

A purely conversational Agent and an Agent that can modify the file system, call APIs, and commit code appear on the surface to differ only by "a few more tools," but they are two different classes of systems in their information structure. The latter's actions produce irreversible external state changes that are not merely byproducts of action but are critical inputs K_{i+1} to subsequent steps. The execution or verification of step $i+1$ must take the external changes caused by step i as its premise.

Let an executor $\mathcal{A}$ execute a tool call τ_i at step i, which modifies the external world state $W_i \to W_{i+1}$. If advancing from c_i to c_{i+1} must take some information in W_{i+1} as critical input, i.e.,

$$K_{i+1} \subseteq W_{i+1},$$

then K_{i+1} cannot be statically folded before step i executes

$$K_{i+1} \notin \text{Fold}(I_i)$$

because it has not yet been generated. This is precisely the formal criterion of the long-horizon practice interval (Definition 9.3) directly instantiated in the context of Agent action.

A purely conversational Agent produces no such non-precomputable critical input: each of its "outputs" is merely an append to the conversation history and does not change the structure of the external world state. Although the next round of conversation takes the previous round's output as input, that output was already generated by the model before the previous round was initiated—it never enters the information structure of "must act first to know what will happen." Thus, conversational Agents belong to the extended execution of single-step closure, not the long-horizon systems of concern to this paper.

Hence, the object of study in this paper is narrowed to: **autonomous systems capable of executing irreversible external actions, whose practice paths contain non-precomputable sequential dependency chains**.

Step Three: The Coding Agent as the Minimal Complete Projection of the General Agent Core

Having narrowed to "systems capable of executing irreversible external actions," the next question is: what do we take as the concrete research vehicle? This paper chooses the Coding Agent as its primary object of analysis. This choice is not driven by topical preference or limited vision, but because the Coding Agent is, structurally, the minimal complete projection of the general Agent core. This judgment relies on the fractal isomorphism argument from Long-Horizon Practice Theory.

Construction (designing architecture, planning systems) and execution (writing code, debugging errors) differ in their apparent functional positioning, but they share the same constraint equation:

$$\text{Construction}: \quad K_{i+1}^{(\text{con})} \notin \text{Fold}(I_i^{(\text{con})})$$

$$\text{Execution}: \quad K_{i+1}^{(\text{exe})} \notin \text{Fold}(I_i^{(\text{exe})})$$

The content carried by the two sides is completely different—one side carries requirements documents and architecture diagrams, the other carries error messages and code line numbers—but the constraint relation $K_{i+1} \notin \text{Fold}(I_i)$ aligns pairwise across the two types of activity. Content differs; the constraint equation is isomorphic. This is the same information topology instantiated twice, once in the construction dimension and once in the execution dimension.

The Coding Agent is the cleanest projection of this information topology, for three reasons:

1. **Code is executable, verifiable, and rollback-able—the cleanest action–feedback loop.** The syntactic correctness of code can be automatically judged by a compiler or interpreter, logical correctness can be automatically judged by a test suite, and state changes can be traced and rolled back by a version control system. Compared to other action domains (physical operations, financial transactions, medical decisions), the feedback signal in the code domain is the densest, most automatable, and least subject to subjective human judgment.

2. **Code is the interface layer between the virtual and the real.** Code operates within a controllable symbolic space, but its execution results can drive real-world effects—from deploying services to controlling hardware. It simultaneously possesses "the controllability of the symbolic world" and "the causal efficacy of the physical world."

3. **Code checkpoints inherently possess independent verifiability (T2).** Long-Horizon Practice Theory requires that the output of each checkpoint be independently verifiable within the current context (Transport Prerequisite T2, §9.2). Code checkpoints—a function, a module, a commit that passes tests—exactly satisfy this requirement: test suites, type systems, and lint tools provide independent verification mechanisms that do not rely on the executor's self-report.

Thus, the Coding Agent is not a "special case" of Agent systems, but the minimal instance possessing the complete action structure. We choose it not because of limited vision, but because it structurally exposes, in the cleanest way, the governance problems shared by all Agent systems. Structural variables are not domain-dependent—governance constraints extracted from Coding Agents are equally valid for other action domains (scientific computing, engineering

design, operations automation), so long as those domains likewise possess the information structure $K_{i+1} \notin \text{Fold}(I_i)$.

3.2 The Three-Layer Concentric Model: Prokaryon, Eukaryon, and Cortex

Having defined the narrow Agent, the next question is: how are these systems organized internally? This section introduces a Three-Layer Concentric Model for dissecting the runtime architecture of Agent systems. This model is not partitioned by a preset of "what functional modules there ought to be," but by natural layering observed in engineering practice—each layer corresponds to a different class of structural problems.

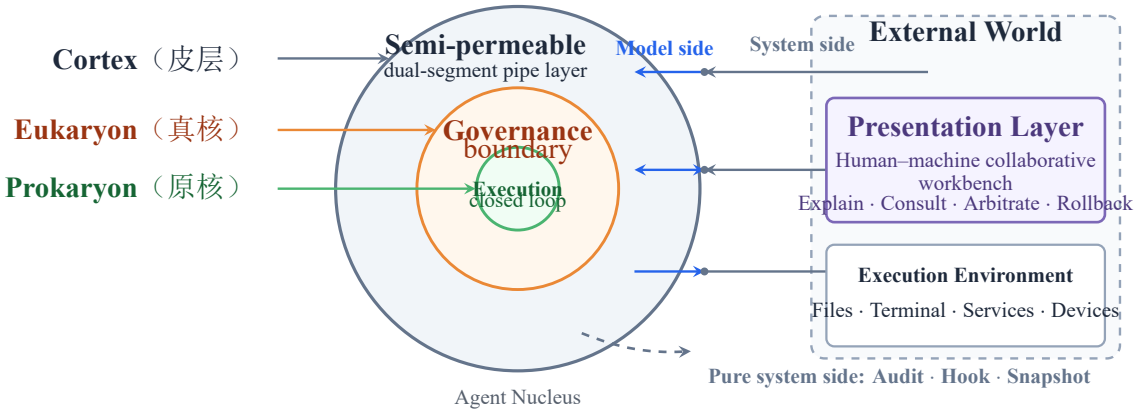


Figure 1: The Prokaryon–Eukaryon–Cortex three-layer Agent Nucleus and its extra-nuclear world

3.2.1 Prokaryon（原核）: The Highly Converged Invocation Protocol

(The complete treatment of Prokaryon（原核）—including the while-loop pseudocode, three-step symbolic compression, and convergence argument—is in §5.)

3.2.2 Eukaryon（真核）: Runtime Governance Infrastructure

Eukaryon（真核） is the Agent's "boundary layer," responsible for two things: maintaining internal continuity across time (State Governance); and defining the boundary of action with the external world (Power Governance). Core principle: anchor the non-deterministic reasoning stream onto discrete, recoverable, verifiable state points.

3.2.3 Cortex（皮层）: The Progressive Capability Disclosure Supply Chain

Cortex（皮层） is the interface layer between the Agent and the external world: any external system that can be driven by a sequence of language tokens can, in principle, be connected. Core principle: Progressive Disclosure—do not let the model see what it should not see when it does not need to.

3.2.4 The Progressive Relationship Among the Three Layers

The three layers, packaged together, constitute a complete Agent core: Prokaryon alone is a model invoker; adding Cortex yields a chatbot that can use tools; only with the addition of

Eukaryon—i.e., State Governance and Power Governance—does one obtain an autonomous system capable of responsibly executing irreversible actions.

3.3 A Rigorous Definition of Governance

Before unfolding the internal mechanisms of Eukaryon, a rigorous definition of "Governance" in the context of this paper must first be given. This definition is not a dictionary paraphrase of the word "governance," but a structural operational definition. It must be precise enough that we can use it to determine whether an Agent system possesses governance capability, and in which dimension it is weak.

Definition 3.1 (Agent Governance)**.** In an Agent system, **Governance** is the set of mechanisms that **anchor** the non-deterministic reasoning output stream of Prokaryon onto discrete, recoverable, verifiable, and accountable state points.

Anchoring is realized through two dimensions:

- **State Governance**: Maintaining internal continuity across time—ensuring the system does not "lose its memory" after interruption, context clearing, or executor handoff. Its operational definition is: at any checkpoint c_i, the system can recover the following four items of information from a persisted snapshot: (1) where the current task has reached; (2) key decisions and their justifications; (3) what is closed and what is unclosed; (4) where to resume from next.

- **Power Governance**: Defining the boundary of action between the system and the external world—ensuring the system does not "overstep" when value is uncertain, paths diverge, or its own judgment is insufficient. Its operational definition is: for any tool call τ, there exists an auditable decision pipeline such that τ is released to the external world only when all three of the following conditions are satisfied: (1) τ is permitted under the current permission mode; (2) if τ is an irreversible operation, it has passed the preset approval gate; (3) the execution boundary of τ is within the coverage of isolation or rollback mechanisms.

The core information structure of Definition 3.1 is: governance is not "constraint"—at least not in the negative sense. Governance is the transformation of a non-deterministic, continuous streaming reasoning output into discrete, locatable, delegatable state points. Without this transformation, every execution of an Agent system is a non-reproducible, non-interruptible, non-handoffable continuous trajectory—it is, in engineering terms, equivalent to a black box.

3.4 Transport Prerequisite Prelude: The Structural Ceiling of the Joint Cognitive System

Before closing this chapter, a judgment critical to all subsequent arguments must be pre-embedded. This judgment will receive its full formal unfolding in §9.2, but its core conclusion directly constrains the definitional framework established in this chapter:

An Agent system does not operate in isolation. Together with its human operator, it constitutes a **Joint Cognitive System**. The long-horizon survival capacity of this system does not depend on the optimal performance of any single component, but is constrained by the weakest link of the entire joint system. This is the core constraint of the Transport Prerequisite. Long-Horizon Practice Theory unfolds this into three layers and nine formal criteria (T1–T3 / E1–E3 / H1–H3; see §9.2 for details), two of which directly determine whether an Agent system possesses "entry qualification":

Engine Physical Limit (E1 Infeasibility Theorem). Let S be the effective execution span of an LLM's single window, and L_{min} be the minimum independently verifiable step length of the task. The necessary condition for long-horizon qualification is:

$$S \geq L_{\text{min}}$$

If $S < L_{\text{min}}$, then there exists no checkpoint sequence such that each checkpoint simultaneously satisfies "large enough to be independently verified" and "small enough to be stably completed by the LLM"—the system structurally lacks long-horizon qualification. This is not a constraint that can be bypassed through engineering compensation.

Driver Cognitive Limit (H1 Asymmetric Verification Bandwidth). Let C_{verify} be the cognitive cost for a human to "read and understand the state" at a checkpoint, and $B(t)$ be the currently available attention budget. The necessary condition for effective verification is:

$$C_{\text{verify}} < B(t)$$

When $C_{\text{verify}} \geq B(t)$, the human produces a rubber-stamp effect—formally going through the approval process while substantively skipping judgment. Furthermore, the operator must also satisfy external reality anchoring capacity (H2) and decision braking threshold (H3)—otherwise, even if the engine satisfies the physical limit, the joint system will still be disabled at the cognitive level.

The complete formal unfolding of these two constraints, and the core corollary they jointly entail—the non-automatability of the verification chain terminal—will be given in full argument in the Discussion chapter (§14.2).

The definitional framework of this section—from the three-step derivation, through the Three-Layer Concentric Model, to the rigorous definition of governance—establishes all the concepts that will be tested against the Transport Prerequisite in subsequent chapters. The convergence of Prokaryon is determined by the engine physical limit: the discrete round structure of the API allows only this one mode of breathing (§5). The necessity of Eukaryon arises from value closure—Power Governance thereby becomes a structural necessity (§6). The progressive disclosure of Cortex is the structural requirement of the attention budget (E1+H1) (§8). The justification of governance cost comes from a more fundamental constraint: an ungoverned Agent is economically unsustainable (§7.2).

4 An Evolutionary Perspective: The Divergence of Seven Systems

Chapter positioning. The source-code evolution history of the seven systems provides the evolutionary explanation for Decision One (State Governance) and Decision Two (Power Governance). Before delving into the three-layer model of Prokaryon/Eukaryon/Cortex, it is necessary to first see the patterns of convergence and divergence that seven real systems produced under different environmental pressures—they are not products of theoretical deduction, but structural fossils left in engineering history. The core thesis of this chapter: governance is not designed; it is forced out by evolutionary pressure.

§3 established the definitional framework of the narrow Agent and the Three-Layer Concentric Model. The value of this framework lies not in conceptual tidiness, but in its ability to explain a fact that puzzles engineering practitioners: why have seven independently developed

Agent systems—sharing the same core loop skeleton and the same tool protocol language—diverged in architecture toward radically different directions? What drove these divergences?

Sample selection limitation statement. This paper discusses Agent systems that have actually appeared as of July 2026, use LLMs as their primary non-deterministic reasoning node, are capable of invoking tools, and are oriented toward external practical outputs. The selection criteria for the seven systems (Codex CLI, Claude Code, OpenCode, Hermes, Cline, Cherry Studio, trae-agent) are: having a real user base, accessible source code, and coverage of different initial scenarios (terminal / IDE / standalone service / desktop / research). This boundary is for selecting comparative samples and is not a sufficient or necessary definition of a general Agent.

To express it in the inductive language of source-code investigation: no team began their project by first writing a "Manifesto of Agent Design Philosophy" and then deriving their permission pipelines and sandbox strategies from it. The real process is Darwinian adaptive radiation—different initial scenarios and resistances forced teams to continuously solve specific failures, costs, and user frictions, forcing them to grow particular structures for state, permission, recovery, tools, hosts, and interaction. After long-term sedimentation, the architecture in turn presents itself as a de facto system philosophy.

Permission pipelines were not drawn on a whiteboard—they were iterated out after users complained too many times "don't make me click confirm every time"; state persistence was not derived from first principles—it was upgraded from in-memory to SQLite after "the system crashed and the user lost three hours of work"; capability extension mechanisms were not foreseen by architects—they were forced into existence upon discovering that "every new tool bloats the core's schema."

Different environmental soils caused these systems to evolve into species almost unrecognizable to each other at the architectural level: a system running on a $5 VPS that uses `check_fn` to prevent Docker daemon jitter from causing entire tool groups to silently disappear (Hermes), and a system whose Electron Renderer would crash mid-streaming-output, leading to the design of resume tokens and state machines (Cherry Studio)—these two face entirely different engineering problems at the state governance layer.

Table 3 summarizes the birth scenarios of the seven systems, the core pressures encountered during iteration, and the key architectural structures that evolved to address those pressures.

Table 3: The Real Birth Scenarios of Seven Systems and the Core Structures Evolved During Iteration

System (Birth Date)	Actual Birth Scenario	Core Structures Evolved During Iteration
Cline (2024.06)	Anthropic Hackathon weekend prototype: giving VS Code users access to Claude 3.5 Sonnet's agentic capabilities	RuntimeHost host decoupling, four-layer SDK (shared/llms/agents/core), Hub daemon, Cron Reconciler truth source
Cherry Studio (2024.H2)	Multi-model aggregation desktop chat client (Electron), unified access to various LLM providers	Agent added later as a feature: Main control plane + Renderer subscription window, session state machine and resume tokens, multi-source Hook composition
Claude Code (2025.02)	Anthropic's first terminal agent, providing a first-party vehicle for Claude's agentic coding capabilities	Single-core async query pipeline, tool-triple modularization, multi-layer context degradation
OpenCode (2025.03)	A terminal agent written solo in Go; later taken over by SST/Anomaly team and full-stack rewritten	Standalone Bun engine + client-server architecture, System Context Epoch, opaque tools
Codex CLI (2025.04)	OpenAI's open-source competitive response to Claude Code (TypeScript/Node.js); Rust rewrite began 2025.06	OS-level sandbox (added during iteration), tiered approval, JSONL+SQLite replayable persistence
trae-agent (2025.06)	Research distillation of the Agent core from ByteDance's TRAE IDE (closed-source VS Code fork)	Lakeview metacognitive layer, full Trajectory JSON audit trail
Hermes (2025.07)	Nous Research internal project: model training trajectory generator + agent platform dual use	Self-registering tool discovery, `check_fn` transient fault tolerance, Skills–Memory–Curator knowledge loop

Representative source-code anchors: Cline–`@cline/core`; Cherry Studio–`AgentSessionRuntimeService`; Claude Code–`QueryEngine.ts`; OpenCode–`system-context/`; Codex CLI–`RolloutRecorder`; trae-agent–`agent/base_agent.py`; Hermes–`tools/registry.py`.

Note: trae-agent (bytedance/trae-agent, MIT license, 289 commits) is the open-source research core of the closed-source TRAE IDE product. TRAE IDE itself is a closed-source VS Code fork (released 2025.01), with its Agent engine embedded inside the Electron process—the "six platform constraints" discussed in Chapter 9 of this paper derive from TRAE IDE's closed-source architecture, which has different constraint conditions from the open-source trae-agent core.

4.1 Initial Conditions and Evolutionary Trajectories of Each System

The above evolutionary framework needs an empirical foundation: from what starting points did these seven systems originate, and driven by what forces did they grow into their present forms? Figure 2 provides a panoramic view—the seven systems independently originated in completely different ecological niches, yet collectively converged on the same Prokaryon loop. The specific source-code anchors and implementation details for each system are collected in Appendix A; this section focuses on the evolutionary dynamics themselves—who appeared first, who was a response to whom, and who diverged along which path at what point in time.

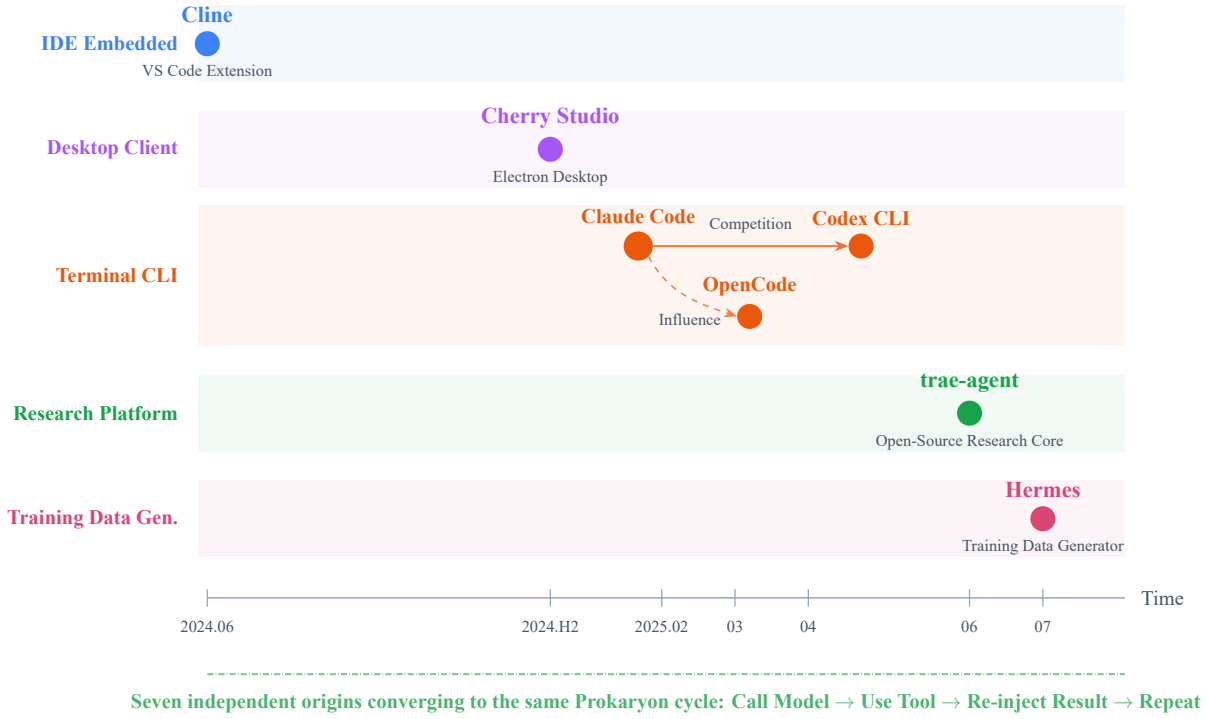


Figure 2: Seven-System Evolution Panorama: independent origins in distinct ecological niches, collective convergence on the same Prokaryon cycle. The horizontal axis marks birth time (2024.06–2025.07); the vertical axis marks each system's initial ecological niche. Within the Terminal CLI band, solid arrows indicate competition between Claude Code and Codex CLI; dashed arrows indicate Claude Code's category influence on OpenCode.

Cline (2024.06): A Weekend Hackathon Prototype

June 2024, roughly ten days after the release of Claude 3.5 Sonnet. Saoud Rizwan read about the agentic coding capabilities described in the model card at Anthropic's Hackathon—cross-file reasoning, multi-step tasks, large context windows—but there were no tools enabling VS Code users to use Claude in this way. He spent a weekend building a prototype, called "Claude Dev," and published it to the VS Code Marketplace.

This prototype had no rigid design constraints. One interesting result: early users spontaneously formed a two-stage habit of "first let the agent produce a plan, review it, then execute"—later solidified into Cline's signature Plan/Act pattern. It proved that valuable architectural features can originate from usage patterns rather than initial design.

The real architectural explosion occurred in 2025—$32M in funding, the team expanding from 1 to 20+ people. The core structures forced out included: a four-layer SDK separation that

completely decouples state persistence from the Agent loop; a RuntimeHost abstraction that allows the same Agent logic to run unmodified across VS Code extensions, CLI, and JetBrains plugins; a Hub daemon that allows multiple clients to share the same session, continue execution after disconnection, and restore upon reconnection; and a Cron Reconciler pattern that completely eliminates false negatives and false positives in file watching. Cline's trajectory is the clearest "from monolith to platform" case among the seven—born as a one-person weekend prototype, one year later a complete Agent runtime platform.

Cline's evolution from monolith to platform reveals a pattern: continuous sessions across multiple host processes are not a concurrency optimization problem, but a core requirement of State Governance.

Cherry Studio (2024.H2): A Multi-Model Chat Client

Second half of 2024, a personal project by a former ByteDance product manager. An Electron desktop client whose core value proposition was simple: "no need to switch between multiple AI websites." Came bundled with 300+ preset AI assistants, knowledge-base RAG, and Mermaid diagram rendering.

Key fact: the Agent was not a birth feature. After Cherry Studio incorporated in early 2025, Agent mode was only added as a new feature in the v1.6.x/v1.7.x series (around late 2025). This means its Agent architecture was not designed on a blank canvas—it was squeezed into an already-existing large Electron chat client.

This produced its most distinctive set of designs. The Main process is the sole entry point for LLM calls and the file system; the Renderer process is responsible only for UI. The Agent engine had no choice but to run in the Main process, with all output the user sees relayed through an IPC bridge. This in turn forced out three key capabilities: closing the window without interrupting Agent execution, replaying buffered events upon reopening; automatically marking遗留 pending messages as errors during crash recovery; and a deterministic composition strategy for Hooks contributed by three different sources—execute sequentially, receive the previous hook's return value, and retry the entire chain if any hook requests "retry."

Cherry Studio's evolution reveals an important pattern: its distinctive structures were not derived from Agent requirements but forced out by constraints imposed by the Electron process architecture. Had the Agent been a birth feature, these designs might have been entirely different.

The Cherry Studio case exposes the structural consequences of platform choice—Electron's multi-process architecture locks the Agent engine into the Main process; the three capabilities of non-interrupted execution after window close, crash state recovery, and deterministic multi-source Hook composition were all forced into existence under the factual constraint that "the Agent was not a birth feature."

Claude Code (2025.02): The Catalyst Defining the "Terminal AI Coding Agent" Category

February 24, 2025, Anthropic released Claude Code Research Preview—the first terminal-native coding Agent from a mainstream AI lab. The initial version was a lightweight CLI with a core loop of a few thousand lines. But its significance far exceeds its code volume: it proved that an agent could, without relying on an IDE, drive the file system and shell directly through natural language in the terminal. 25,000+ GitHub stars in one week. Six weeks later, two direct responses appeared—OpenCode and Codex CLI.

Iteration pushed it to another order of magnitude. The core query engine swelled to roughly 46,000 lines—six categories of logic (LLM streaming calls, tool-call detection, permission interception, result injection, token budget checking, context compression triggering) gradually

merged into a single file across multiple major version iterations. The four-layer context degradation forced out by long-session pressure—tool result substitution → history truncation → compression → overflow retry—is a progressive degradation sequence that only escalates when the previous layer genuinely fails.

Claude Code holds a special position in this paper's core argument: it is the only one of the seven born without any "deformed structures forced by the environment." But even with the cleanest birth, iteration still grew a 46,000-line God class and a four-layer degradation path. Environmental constraints are not the only evolutionary driver—internal complexity accumulation is itself one.

Even with the cleanest birth environment, internal complexity accumulation is itself an evolutionary driver no less potent than environmental constraints—the 46,000-line God class and four-layer degradation path are not products of any external pressure, but the inevitable result of a system freely expanding under unconstrained conditions.

OpenCode (2025.03): A One-Person Go Terminal Project

March 21, 2025, a personal project by Kujtim Hoxha. Go language + Bubble Tea TUI. Four commits in two days, a basically working agent in one week, LSP support added in the second week.

OpenCode was never a VS Code extension. Its initial environmental conditions were pure terminal—one person, a few days, a Go TUI agent. Dax Raad, founder of SST/Anomaly, joined in April and from May onward led a full rewrite of the backend from Go to TypeScript+Bun—motivated by connecting to hundreds of providers on models.dev. The TUI was not rewritten from Go Bubble Tea to the self-built OpenTUI until October.

The backend rewrite unexpectedly produced a client-server architecture—the new backend (TypeScript+Bun) and the old TUI (Go) needed a communication protocol. Once the engine and frontend were separated, state consistency and recoverability became hard problems. This forced out a distinctive algebraic system: each context source contains a stable identifier, JSON codec, failure-free loader, and pure renderer, with four operations covering the full lifecycle from initial generation to post-compression reconstruction. Another forced-out structure was opaque tools—tool behavior is hidden, and the contract only exposes I/O types. The purpose is not security, but guaranteeing "tool declaration consistency" after process separation: the frontend can render correct UI without ever asking about tool implementation.

The client-server architecture accidentally carved out by the backend rewrite upgraded context persistence from an "optional optimization" to a "structural necessity"—the algebraic context system and opaque tool protocol are both engineering answers to maintaining state consistency between processes that do not share memory.

Codex CLI (2025.04): A Competitive Response to Claude Code

April 16, 2025, OpenAI released Codex CLI. Same day as o3/o4-mini. TypeScript/Node.js + React Ink TUI, Apache 2.0 open source. Simultaneously released the codex-1 model—a code-specialized version of o4-mini.

The timeline says everything: Claude Code release (Feb 24) to Codex CLI release—an interval of less than 8 weeks. This is not a time window for independent conception from scratch. Open source (Apache 2.0) itself was also a strategic differentiator—Claude Code's source code is not freely usable.

Codex CLI's most impressive architectural feature (OS-level security sandbox covering Linux/macOS/Windows) was not present at birth. It was only established as a competitive moat

after the full Rust rewrite launched in June 2025. Beyond this, two design choices have special structural value. The first is tiered approval: four tiers of approval intensity, where the approver can be a human or an independent automated review Agent—the only design among the seven that introduces a non-human reviewer into the approval chain. The second is full event persistence: Codex's state is not a "snapshot" but a complete, replayable, reconstructable event stream. This forms a clear philosophical contrast with Claude Code's multi-layer compression. Codex pursues complete reconstructability (driven by security audit requirements), while Claude Code pursues sufficiency-for-reasoning after compression (driven by context window budget).

The philosophical contrast between full event persistence (Codex CLI) and multi-layer context compression (Claude Code) exposes the irreconcilability of two evolutionary pressures—security audit demands complete reconstructability, while context budget demands sufficiency-for-reasoning after compression.

trae-agent (2025.06): A Research Core Distilled from a Closed-Source Product

A system easily misread. trae-agent is not TRAE IDE. TRAE IDE is ByteDance's closed-source VS Code fork released in January 2025, with 6M+ registered users in 12 months and free model inference. trae-agent is the open-source Agent research core (MIT license) distilled from it 5 months after product release. It contains the core Agent loop, tool system, LLM client abstraction, and evaluation framework, but is completely stripped of VS Code/Electron integration.

This determines its unique value: it is the only one of the seven that takes "researchability" rather than "robustness" as its first principle. No need for complex recovery (tasks are short-horizon and repeatable), no need for approval pipelines (lab environment with full observation), no need for multi-Agent delegation (ablation experiments require variable control).

The most interesting structure forced out by this positioning is the Lakeview metacognitive layer—a separate LLM call specifically analyzing each step of the main Agent's trajectory, classifying steps into predefined behavior labels. Essentially an "agent that observes the agent," not to help the main Agent do better, but to allow researchers to understand afterward why it did what it did. Another structure is the full-trajectory JSON audit trail—similar in form to Codex CLI's JSONL but with a completely different purpose: Codex's JSONL is runtime recovery infrastructure; trae-agent's trajectories are post-hoc analysis tools.

Here, a note important for subsequent chapters: the "six platform constraints" discussed in Chapter 10 (prohibition of sub-sub-agent nesting, uneven agent capabilities, invisible meta-information, missing rollback/resume, local machine performance bottlenecks, hard tool quota caps) originate from TRAE IDE's closed-source "embedded" architecture. The open-source trae-agent core is not subject to these constraints. The difference between the two reminds us: an Agent system's governance capability boundary depends not only on the core loop design, but even more on the runtime host architecture.

trae-agent's choice reveals a dimension overlooked by other systems: when "researchability" rather than "robustness" becomes the first principle, the system needs no complex recovery or approval pipelines, but must enable researchers to understand, after the fact, the causal chain of every decision the Agent made at each step.

Hermes (2025.07): A Model Training Data Generator

July 22, 2025, Nous Research, known for open-weight models, created the Hermes repository. After 8 months of internal development, the first public tag appeared in March 2026. It differs fundamentally from the previous six systems: it was born with a dual purpose—it is both an

Agent and a training data generator. The RL training environment and parallel trajectory generator in the source code expose its closed-loop logic: Agent executes task → generates trajectory → trajectory trains next-generation model → stronger model drives stronger Agent.

After going public, Hermes experienced a community explosion—within two weeks of the v0.2.0 release, 63 contributors merged 216 PRs. The structure most forced out by community pressure was the multi-platform messaging gateway—Telegram, Discord, Slack, WhatsApp, and 20+ other platforms. But Hermes's three most valuable structures come from its 8 months of internal development.

First, the prompt cache inviolability principle. Hermes is the only one of the seven that treats Anthropic prompt caching stability as an explicit design constraint: the system prompt is built only once per session, and tool changes are injected through user messages. This shaped a distinctive tool registration system—each module self-registers upon loading, and the discovery phase first statically scans to confirm registration behavior before deciding on imports.

Second, transient fault tolerance (`check_fn`). In a multi-platform environment, a Docker daemon response latency of 2 seconds should not cause an entire group of tools to silently disappear. Hermes's `check_fn` has dual-layer protection: caching avoids frequent probing, and failures within the recent-success time window are treated as glitches—failure results are not cached. This is not common fault-tolerance practice, but a defense layer specifically designed for the scenario "tools should not disappear due to short-term jitter."

Third, the Skills-Curator knowledge closed loop—the most complete cross-session learning system among the seven. After the Agent completes a task, it automatically creates or updates reusable skill files; the Curator manages the skill library lifecycle; the Memory Provider maintains cross-session factual memory. This makes Hermes the only one of the seven that treats "accumulating experience" as a core runtime behavior—in other systems, Skills and Memory are add-on features; in Hermes, Skills are one of the purposes for which the Agent exists.

Hermes's dual purpose—both Agent and training data generator—forces the system to simultaneously erect defense layers across three dimensions: prompt cache stability, transient fault tolerance, and cross-session knowledge accumulation. This makes its governance demands naturally higher than any single-purpose system.

4.2 Convergence Beneath the Divergence: Why the Prokaryon Never Changes, and Why the Cortex Is Unpredictable

§4.1 walked through the independent evolutionary paths of the seven systems in birth chronological order. Placing the seven paths side by side, one fact is inescapable: they have diverged in the Eukaryon and Cortex toward almost mutually unrecognizable directions—yet the core loop has never changed. The seven, without any coordination and without sharing programming languages, independently converged on the same core form: **call model → use tool → inject result → repeat**. Cherry Studio's Electron process architecture, Codex CLI's OS-level sandbox, Hermes's `check_fn`—all merely added control layers around this loop, never changing the loop itself.

The reason lies in underlying API constraints, not in design philosophy. Mainstream LLM APIs in 2026 impose three rigid boundaries that constitute an inescapable potential well:

1. **Discrete round interaction.** The model API is stateless—each call requires the full input context and disconnects after returning. The core loop is necessarily a discrete step-by-step advance.

2. **Blocking tool-call protocol.** The model emits a tool call, the Agent executes the tool, collects

the result, and injects it into the next round's prompt. The "call → execute → wait for result → inject" blocking loop is an API-enforced interaction protocol, not a choice of Agent architecture.

3. **Finite context window.** Each round has a limited token budget, forcing compression and degradation in long sessions—but this only adds management strategies at the periphery, without changing the loop structure itself.

The seven did not choose the same core loop—the same core loop chose them.

The flip side of this convergence story is the essential unpredictability of the Cortex. §3 gave the operational definition of the Cortex: any external system that can be driven through a language sequence can be connected to the Cortex, with language as the universal bus. This means the Cortex's form is determined not by the API but by the "language control demands" of its birth scenario. Codex CLI issues OS sandbox instructions, Hermes issues messaging gateway instructions, Cherry Studio issues IPC bridge instructions—their Cortex layers look completely different. This is not an accident of evolution, but a direct consequence of the Cortex's design principle: it is, by definition, open.

Sandwiched between the locked Prokaryon and the open Cortex is the Eukaryon—the only layer among the three constrained in both directions simultaneously. Downward, it is constrained by the Prokaryon's discrete round structure (governance operations can only be inserted at step boundaries); outward, it is constrained by the Cortex's openness (any Cortex extension can introduce new security boundaries and state dependencies). Codex's sandbox, Cherry's process-separation state machine, Hermes's `check_fn`, Cline's Hub/Spoke—the divergence of the seven Eukarya is, at bottom, seven different answers to the same question: between a locked core loop and an unpredictable Cortex, what should governance look like?

This three-layer structure is not only a faithful description of evolutionary history—each layer can be derived from first principles. The next chapter sets out to do exactly this: starting from the discrete round structure of the API, elevate the convergence of the Prokaryon from an empirical observation to a formal proposition.

4.3 The Collective Blind Spot in Evolutionary History: L4 Cross-Execution-Surface Handoff Governance

The above three-layer structure also explains why L4 (cross-execution-surface handoff governance) became a collective blind spot in evolutionary history: the Prokaryon offers nothing to optimize, the Cortex is not determined by governance, and the Eukaryon's priorities are set by the birth environment—the seven birth environments imposed survival-level pressures at L1–L3, yet none required "after execution completes, hand over a structured list of unclosed items and verified conclusions to the next executor." L4 is not unimportant—it simply has not yet reached a survival-level threshold on any system's environmental pressure spectrum. This section uses a four-level continuity scale to quantify this gap:

- **L1 (Intra-long-session continuity)**: A single session spans multiple model calls and tool executions, maintaining continuous progress. Typical mechanisms: context compression, token budget management.
- **L2 (Cross-process recovery)**: After a process crash or session termination, recover to a state from which execution can continue. Typical mechanisms: resume tokens, checkpoint recovery, event stream replay.

- **L3 (Cross-task accumulation)**: Experience, skills, or knowledge from a previous task carried into the next task. Typical mechanisms: Skills, Memory, tool library accumulation.
- **L4 (Cross-execution-surface handoff governance)**: After one execution completes, hand over to the next executor structured, independently verifiable handoff information—closed items, unclosed items, key decision justifications, verified conclusions, current responsibility status, and resume entry point.

Table 4 uses these four levels as a yardstick to examine the continuity capabilities currently engineered in the seven systems.

Table 4: L1–L4 Continuity Capabilities of the Seven Systems

System	L1 Long-Session	L2 Cross-Process	L3 Cross-Task	L4 Handoff Governance
trae-agent	max_steps	—	—	—
OpenCode	compaction	Epoch recovery	—	—
Claude Code	multi-layer compression	resume	MEMORY.md	—
Codex CLI	compaction	rollback recovery	Skills, Hooks	—
Cline	compaction	Hub restore	Rules, Skills	Cron report
Hermes	compression	SessionDB	Memory, Curator	Kanban recent

This table reveals the most consistent finding in evolutionary history: **L4 is the weakest link across all seven.** L1–L3 already have coverage in various systems—compaction and resume are standard equipment, Skills/Memory already have multiple implementations—but L4 has not reached an engineered level in any of the seven. Cline's Cron report and Hermes's Kanban are the designs closest to L4, but what they record is "what tasks were completed" or "what tasks are in the queue," not L4's core information structure: which judgments on the current surface are verified, which are tentative, which are based only on the previous executor's local information, and at which node the next executor must re-verify.

No blame attaches to them—their birth scenarios never required cross-surface handoff. But if one accepts that long-horizon practice as defined in §3.1 requires non-precomputable sequential dependency chains where $K_{i+1} \notin \text{Fold}(I_i)$, then in a long-horizon task interleaved across multiple execution surfaces, L4 is not an optional feature—it is the only structural defense against an executor substituting local memory for global state at checkpoints where information is not closed.

This brings us to the core argumentative gap of this paper. The seven systems diverged toward radically different architectures, yet are strikingly unanimous in their L4 blank. That is what the next chapter discusses.

5 Prokaryon: The Highly Converged Invocation Protocol

Chapter Positioning. This chapter provides the complete argument for Decision Zero ("Is this an Agent?"). Before discussing "how to do it better," one must first recognize "what can no longer be done differently"—seven independently evolved systems converge on nearly identical structures at the Prokaryon（原核） layer, and this is a strong signal. The work of doing Agent well does not lie at this layer: the design space of the invocation protocol loop is near saturation, with no room for competition.

As defined in Chapter 3, Prokaryon is the Minimal Closure of Model Invocation Protocol. Its form is a while loop of approximately ± 20 lines:

```
while (budget not exceeded and not interrupted):
  response = model(system_prompt + tool_declarations +
conversation_history)
  if response.has_tool_calls:
    execute tools → append results back into conversa-
tion history
  else:
    exit loop → return final answer
```

This ± 20-line skeleton is not anyone's design—it is the endpoint of independent evolution (see Chapter 1 for details): under the information structure of the current LLM API invocation paradigm, the form of Prokaryon has reached high stability.

Compressed into a single line of notation, this loop is:

$$\begin{aligned} &\text{Call model: } \text{model}(s_i) \to (r_i, \tau_i) \\ &\quad \longrightarrow \text{Use tools: } \text{exec}(\tau_i) \to o_i \\ &\quad \longrightarrow \text{Reinject: } s_{i+1} = s_i \oplus (r_i, o_i) \end{aligned}$$

Three steps cycling without pause, until $\tau_i = \emptyset$ (the model no longer requests tool calls) or external interruption. This is the entirety of Prokaryon.

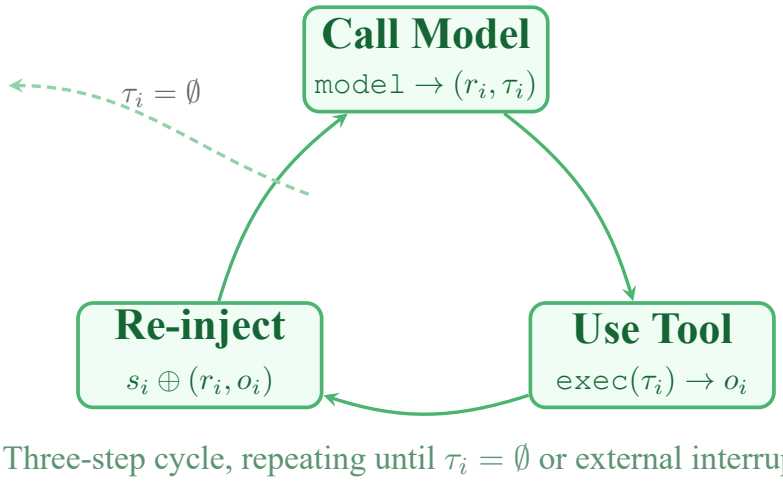


Figure 3: The Prokaryon call-model–use-tool–re-inject three-step cycle: each step performs model inference → tool invocation → result writeback, repeating until termination

Precisely because it is so simple, the payoff of deepening Prokaryon is extremely low. Nearly all the hard techniques of the Agent layer reside within this ± 20-line loop—there are no secret doors, no hidden layers, no "architectural innovations" worth additional elaboration. The real battlefield is in Eukaryon (§6): when monotonic appendage collides with a finite context window, and when tool calls require approval and isolation. It is at this point that governance becomes a structural necessity.

Finally, a cautious qualification must be added, originating from the sample boundaries of this study: **Within the mainstream LLM Agent practices of 2026 examined in this paper, Prokaryon has already highly converged.** This paper does not unconditionally claim that all future Agents must take only this form. If the model invocation protocol undergoes structural changes in the future—for example, continuous rather than discrete-turn interaction paradigms, or tools no longer following the blocking ”call–wait–result” protocol—the form of Prokaryon may change accordingly. But for the Agent systems discussed in this paper, built upon mainstream LLM APIs of 2026, the convergence of Prokaryon is an observed, independently verifiable engineering fact, not a philosophical assertion.

6 Eukaryon: The Real Battlefield

Chapter Positioning. This chapter provides the complete exposition of Decision One (State Governance) and Decision Two (Power Governance). Having established that ”establishing state governance” and ”establishing power governance” are things that must be done, one needs to see how they are realized in actual systems, where they fail, and why the three-layer permission pipeline is a convergent result of evolution rather than a design preference. The Eukaryon layer is the engineering carrier of the Agent system’s governance infrastructure.

As Chapter 5 showed, Prokaryon has already highly converged; the real differences lie in the governance layer. This chapter unfolds the engineering carrier of this governance layer—**Eukaryon**（真核）—into a complete theoretical analysis.

Before formally proceeding, the precise position of Eukaryon within the overall Three-Layer Concentric Model must be clarified. In the Three-Layer Concentric Model introduced in §3, Eukaryon is defined as the ”runtime governance infrastructure,” answering two questions: ”how the system remembers” and ”what the system is allowed to do.” The definition in Ch2 emphasized functional description; this chapter’s task is to unfold these two questions into a complete theoretical analysis, demonstrating why they are not the same question, and why conflating them leads to diametrically opposite repair paths.

6.1 The Core Insight of the Dichotomy: State Governance and Power Governance

In engineering practice, Agent systems share a common failure mode: a task veers off course midway, or does something it should not. Faced with such failure, the engineer’s instinctive reaction is often ”add more rules” or ”add more constraints.” But this instinct obscures a critical distinction: going off course has two entirely different causes—the system does not remember where it was (state fracture), or the system crossed an action boundary it should not have crossed (power transgression). The two types of causes require entirely different repair paths, and conflating them makes governance investment splash harmlessly against water.

This is precisely the core insight of the internal dichotomy of Eukaryon:

Definition 6.1 (State Governance)**.** State governance answers how a system maintains internal continuity across time. Its objects are: context management, persistence strategies, compression and recovery, and cross-session handoff. The consequence of its absence is system ”amnesia”—not remembering where it was, why it got there, or where to continue from next. Engineering carriers include SQLite session databases, event logs, Context Epoch, resume tokens, and anchor documents.

Definition 6.2 (Power Governance)**.** Power governance answers where the system's action boundaries with the external world lie. Its objects are: permission pipelines, approval mechanisms, sandbox isolation, value closure, machine-to-human escalation, and rollback strategies. The consequence of its absence is system "transgression"—performing operations it should not, or failing to consult humans at nodes requiring human judgment. Engineering carriers include permission patterns, `check_fn` functions, approval workflows, OS-level sandboxes, and `ec7_action_gate`.

These two definitions are not a conceptual classification exercise; they possess direct **diagnostic and predictive power**. Confronted with the failure of an Agent long-horizon task, if one diagnoses that the system cannot correctly resume after interruption—losing track of "where it was," "why it made the choices it did," "which branches were ruled out"—then the problem is a state governance deficiency, and the repair direction is strengthening persistence and handoff mechanisms.

If one diagnoses that the system performed irreversible operations without authorization—deleted files it should not have, crossed sandbox boundaries, silently determined direction at nodes requiring human value judgment—then the problem is a power governance deficiency, and the repair direction is strengthening permission pipelines and escalation mechanisms. Adding permission rules to the former or adding persistence to the latter is investing governance cost in the wrong place and will not produce the expected effect.

Proposition 6.1 (State–Power Irreducibility)**.** *State governance mechanisms cannot substitute for power governance mechanisms, and vice versa. State fractures cannot be repaired by permission rules (the system still won't remember where it was), and power transgressions cannot be repaired by persistence (the system will still act when it shouldn't). Both types of governance must exist simultaneously and must operate independently.*

Between state governance and power governance lies a critical intersection zone: "the write authority over state"—who may modify state snapshots, who may mark a checkpoint as "closed," who may delete or overwrite historical state records—is itself the highest-level power. Tampering with or selectively submitting state is, in essence, the executor exercising unauthorized epistemic power, whose consequences manifest on the power governance side as false closure propagation and on the state governance side as state contamination. This intersection zone does not break the dichotomy—the write authority over state remains within the jurisdiction of power governance, and the semantic annotation of state remains within the jurisdiction of state governance—but it reminds us: the engineering realization of state governance must accept the constraints of power governance (write permissions, modification auditing, overwrite traceability), otherwise state itself becomes a vehicle for power transgression. The principle of "state sovereignty" discussed in §7.1—that sub-agents may not autonomously write conclusions into the master state—is precisely a concrete instance of this intersection zone in the multi-agent scenario.

6.2 State Governance

State governance is the half of Eukaryon that answers "how the system maintains internal continuity across time." It is not a synonym for "memory augmentation"—this distinction will be unfolded in the core argument of this subsection—but rather the set of mechanisms that determine at which continuable, recoverable, and verifiable position the system stands at any given moment.

6.2.1 Persistence Strategy Spectrum: Where Seven Systems Draw the Line

The seven systems share the same skeleton at the Prokaryon layer, but on the decision of ”what goes into persistent storage,” they draw a full spectrum from radical to conservative. Table 5 presents them in ascending order of persistence depth.

Table 5: Persistence Strategy Spectrum of the Seven Systems

System	Persistence Layer	Core Characteristic	Level
trae-agent	Memory-dominant	No independent persistence layer; state resides in process memory, evaporates when process ends	L0
Cline	File system	Hub/Spoke architecture passes state snapshots via file system; no dedicated database	L1
Cherry Studio	In-process state machine	AgentSessionRuntimeService maintains session state; resume token enables recovery but not full persistence	L1
Claude Code	Four-tier compression, non-persistent	Conversation history in memory; compaction trims old and preserves new but does not persist cross-process state; MEMORY.md is an optional user file	L1
Codex CLI	SQLite + JSONL	StateDB (SQLite) structured storage; RolloutRecorder (JSONL) full event stream; ScanAndRepair rebuilds corrupted indices from JSONL	L2
Hermes	SQLite SessionDB	SessionDB persists session metadata and messages; Memory Provider independently manages cross-session memory	L2
OpenCode	Event Sourcing + Projection	Immutable event table + session message projection + Context Epoch baseline; admit/promote separates input visibility	L3

The two endpoints of this spectrum represent two different understandings of what ”state” essentially is. Systems at the L0–L1 end treat state as ”a byproduct of the current execution process”—once the session ends, there is no need to retain state. Systems at the L2–L3 end treat state as a ”first-class citizen”—the life cycle of state is independent of the life cycle of a single session.

OpenCode is the most radical representative of the L3 end and deserves separate elaboration. Its state architecture is not ”storing conversations in SQLite” but rather a complete Event

Sourcing system: the **immutable event table** records all facts that have occurred (user messages, model calls, tool executions), and once written they cannot be modified; **session messages** are the current projection of these events—not independently stored, but views computed in real time from the event table; the **Context Epoch** serves as a baseline marker for the model context, enabling the system to reconstruct the complete context the model saw at any arbitrary Epoch point; **admit/promote dual-phase input management** separates "message arrival at the system" and "message visibility to the model" into two independent steps, allowing the system to filter, delay, or reorder messages before they enter the model's field of view. The engineering implication of this entire design is: **An Agent is not equivalent to a single LLM loop currently running in context; it is a persistent system possessing externalized, committable, recoverable, and governable state.**

6.2.2 Multi-Level Degradation Path for Context Governance

State governance involves not only "whether to persist" but also a more urgent runtime problem: the context window is a fixed-capacity scarce resource, and the cumulative context of long-horizon tasks will inevitably breach the window ceiling. Every system must answer "what to do when the window is full"—this is not an optional optimization, but a hard constraint that must be handled in engineering.

The strategies of the seven systems on this problem exhibit a highly consistent layered degradation structure. Claude Code's four-tier compression mechanism serves as the most representative instance:

1. **Tool Result Replacement.** For oversized tool outputs (e.g., reading an entire log file), replace the raw output with a summary or truncated version, preserving the tool call signature but discarding redundant content. This is the cheapest degradation—only the details of tool results are lost, not the conversational structure.

2. **History Snip.** When tool result replacement is still insufficient, remove the earliest turns from the conversation history. Retain the most recent several turns and the system prompt. The cost is the loss of decision rationale from early context.

3. **Compaction.** Feed a contiguous segment of conversation history into the model, request a structured summary from the model, then replace the original segment with the summary. This preserves semantic information but loses exact wording and reasoning details. Whether the compacted state remains usable depends on whether the summary retains the key information required for the three-part handoff (where it was, why, what remains open).

4. **Overflow Collapse.** When the preceding three tiers all fail—the window has been compressed to its limit and the model still cannot continue effective reasoning in the current context—the system either forcibly interrupts and prompts the user to restart, or continues advancing in unreliable context. The former is a controllable failure; the latter is silent drift.

This four-tier degradation path reveals a deeper fact: each tier of degradation in context governance is **trading information fidelity for execution continuity**. Tool result replacement loses detail, history snipping loses early decision context, compaction loses precise wording. And overflow collapse means that even continuity itself cannot be maintained. The engineering task of state governance is to ensure, at each step of this degradation path, that the system still holds "the minimum information sufficient to continue advancing"—and what constitutes this "minimum information" is precisely the core question of the next subsection.

6.2.3 Memory vs. State: A Question of Position

The Agent community holds a widely propagated view: Agents need ”Memory,” including cross-session conversation history, vectorized experience retrieval, and episodic memory. Hermes's Memory Provider and Curator mechanism, and Cline's cross-session Rules and Skills loading, all embed memory as a first-class citizen within the core architecture.

The core judgment of this subsection is: what Long-Horizon Practice needs is structured, semantically annotated state, not chronological conversation logs. Memory is an optional plugin of Cortex, provided in forms such as RAG, vector databases, and Skills, enabled by the user on demand, and subject to the governance constraints of Eukaryon (permissions, scope, timeliness, activation conditions, revocability). State is a mandatory component of Eukaryon; all Agent systems must possess at least a minimum level of state transfer capability. The following unfolds this judgment from three dimensions: information structure, engineering consequences, and long-term risk.

The information structures of the two are entirely different:

Dimension	Memory	State
Information form	Chronological conversation records, experience fragments	Structured semantic annotations
Question answered	”What happened before”	”Where are we, why, what remains open, where to continue from”
Organization	Timeline or vector similarity	Indexed by checkpoint / closure status
Verifiability	Content may be correct but irrelevant	Each entry has independent closure criteria
Governance needs	Requires relevance filtering, debiasing, timeliness management	Requires three-valued status annotation, independent evaluation

This means: leaving the next executor several thousand conversation records is inferior to leaving a few dozen lines of three-part handoff documentation. This is precisely the core requirement of **HC-3 (State Transfer)**[1]—state transfer is not ”passing historical records to the next agent,” but passing semantically annotated structured state. The specific format of the three-part handoff is detailed in the ”Three-Part Handoff for State Transfer” subsection below.

Embedding memory into the Eukaryon layer carries an even more insidious long-term risk. The essence of Agent use is discrete[5]: each session has an independent intent boundary, and preferences and judgments from old sessions should not be automatically injected into new sessions without explicit verification. The ”tendencies” accumulated by cross-session memory appear as convenience in the short term (3–5 sessions) but pile up into systematic bias over the long term (50+ sessions)—and the user may not even be aware of when these biases seeped in. Curator's automatic archiving answers ”can we remember this” but does not answer ”should we use this memory”: the former is storage management, the latter is cognitive management.

[1] HC-3 is one of the six logically necessary conditions of Long-Horizon Practice theory (Zeng, 2025). HC = Hard Condition (hard fracture condition); the absence of HC-3 causes information evaporation between execution sessions, making subsequent checkpoints structurally unresumable. See §9.2 and the original Long-Horizon Practice theory text for details.

6.2.4 Three-Part Handoff for State Transfer

This is precisely the engineering answer to the L4 collective blind spot diagnosed in Chapter 4—cross-session handoff governance. The state transfer required by HC-3 is not an abstract principle but an operational procedure with explicit engineering format. Cross-session state handoff must include three parts of explicit information; the absence of any one part constitutes an incomplete handoff:

1. **Engineering Process.** Which checkpoints have been completed, in what order. This is the ”map” for continued execution—subsequent executors use it to determine what work has been done and what remains, avoiding duplicate effort or skipped prerequisites.

2. **Handoff Status.** What status the current checkpoint is in—closed / open / blocked / currently undecidable. Among these, ”currently undecidable” is a critical three-valued state: it acknowledges that the closure determination of certain checkpoints cannot be made under current information conditions, while preventing the system from silently marking this uncertainty as ”completed.” If the same checkpoint remains marked ”currently undecidable” across two consecutive checkpoints, the system must trigger **EC-7 (Machine-to-Human Proactive Escalation)**[2], submitting the reason for undecidability, attempted resolution paths, and the impact of this blockage on the downstream dependency chain for human adjudication.

3. **Final Result.** The verification conclusions of completed checkpoints (verification method, verification evidence, verification outcome) and deliverable inventory (file paths, versions, content fingerprints). This is the key to preventing false closure—”completed” cannot rest solely on the executor’s self-report; there must be an evidence chain that can be independently verified.

These three parts constitute an execution closure loop: Process tells the next executor ”what was done,” Status tells them ”where we are,” Result tells them ”whether it was done right.” When all three are present, the subsequent executor need not re-read the conversation history, need not guess the predecessor’s judgment basis, need not discover open items from scratch—this is precisely the core engineering value that distinguishes state from memory.

6.3 Power Governance

Power governance is the half of Eukaryon that answers ”where the system’s action boundaries with the external world lie.” If state governance determines whether the system can ”remember,” power governance determines whether the system can ”not act recklessly.” Power governance not only constrains system actions; it also has a commonly overlooked extension on the cognitive side—the design of the presentation layer determines what information the human adjudicator sees, in what form, and at what granularity they understand the system’s action proposals. This extension will be unfolded in Ch11.

6.3.1 The Three Layers of the Permission Pipeline Are Not Mutually Substitutable

A common misconception is equating ”tool visibility control” with ”power governance.” In reality, a complete power pipeline contains at least three layers that can be independently designed

[2] EC-7 is one of the seven engineering compensation conditions of Long-Horizon Practice theory. EC = Engineering Compensation (engineering compensation condition). EC-7 requires the system to proactively escalate to humans when encountering decisions beyond its own judgment capacity, rather than filling gaps on its own. See §9.3 for details.

and are not mutually substitutable:

1. **Tool Visibility.** Which tool schemas can the model see? This is the outermost layer of the pipeline—controlling "what tools the model knows exist." MCP server toggles, Skill on-demand loading, and Cortex's progressive disclosure all operate at this layer.

2. **Permission Approval.** After a tool call is generated, who decides whether it is allowed to execute? This is the middle layer of the pipeline—controlling "who has the authority to say yes." The approver may be a human user, an independent review subagent (such as Codex's auto-review subagent), or a rule-based automatic policy engine.

3. **Execution Isolation.** When the tool actually runs, what system resources can it touch? This is the innermost layer of the pipeline—controlling "what the tool can actually do." OS-level sandboxes, filesystem read-only mounts, network policies, and process isolation all operate at this layer.

The design spaces of these three layers are mutually independent. One system could show the model a complete tool list, yet require human approval for every tool call, while running all tools within a full sandbox. Another system could show only a trimmed tool set, yet allow the model to freely call tools without approval, while tools run directly on the host filesystem. Conflating these three layers leaves invisible gaps in the architecture—for example, assuming that "the model can't see a tool" equals "the system won't execute that tool."

Mapping the three layers onto the actual implementations of the seven systems yields a complete permission spectrum comparison table (Table 6).

Table 6 reveals a pattern: the depth of a system's power governance is positively correlated with the "trust deficit" of its initial environmental conditions. Codex CLI was born into an "untrusted execution environment"—users need to let LLMs safely execute AI-generated code in third-party code repositories, so it goes deepest on all three layers: sandbox policy unified filtering (first layer), four-level approval plus auto-review subagent (second layer), OS-level sandbox (third layer). trae-agent was born into research evaluation scenarios, where it does not need to handle real user data or production environments, so it remains at the shallowest depth on all three layers. This is not a case of trae-agent's design being inferior to Codex's, but rather their initial environmental conditions endowing them with different power governance requirements. A system born into a trustless environment that fails to establish deep power pipelines cannot structurally survive.

6.3.2 Value Closure and Machine-to-Human Proactive Escalation

The most critical node of power governance is not "preventing the model from doing bad things," but rather **ensuring that when the model cannot independently judge right from wrong, humans can make informed adjudications**. This points to the intersection zone of two concepts: SC-1 (Value Closure) and EC-7 (Machine-to-Human Proactive Escalation).

SC-1 points out: goal closure is local. "This function passes all tests" is a fact that can be independently verified. But "this function does the right thing" is a value judgment—whether its design is consistent with the overall architecture, whether its behavior aligns with user intent, whether its side effects fall within an acceptable range—none of these can be substituted by local testing.[3]

[3] SC-1 is one of the six logically necessary conditions of Long-Horizon Practice theory. SC = Soft Condition (soft fracture condition); the absence of SC-1 causes the system to be "done" but "not done right." Value closure acts upon goal closure in a hierarchical governance relationship. See §9.2 for details.

Table 6: Three-Layer Permission Pipeline Comparison of the Seven Systems

System	Tool Visibility	Permission Approval	Execution Isolation
Claude Code	Eight-layer permission pipeline filtering layer by layer, on-demand exposure	`canUseTool` callback + user interactive approval; per-session/project/command granularity	Filesystem read/write separation markers; no OS-level sandbox
Codex CLI	Unified sandbox policy filtering	Four-level approval policy + auto-review subagent	bwrap/Landlock/Seatbelt OS-level sandbox; filesystem + network dual Policy
OpenCode	`allow/ask/deny` three-state permission	Interactive per-call approval; permission policy persisted as TOML configuration file	No OS-level sandbox; relies on host permissions
Hermes	Dynamic tool registration and deregistration	`check_fn` functional permission validation; transient fault tolerance	No OS-level isolation
Cline	MCP + built-in tools dual-layer	Hub bears approval routing in Hub/Spoke architecture	No OS-level isolation; relies on IDE host permission boundaries
Cherry Studio	Main process manages tool flow	After model output parsing, Main flow manager decides whether to execute	No OS-level isolation
trae-agent	Modular replaceable tool set	Research/evaluation only; no independent approval pipeline	No isolation

LLMs, as closed information-processing systems, are insensate to external drift and decision forks. When the user's real needs undergo subtle changes during execution, the base model will continue perfectly executing an already-obsolete goal.

This is the engineering foothold of EC-7. EC-7 requires the system to proactively pause and escalate to humans under the following conditions: encountering non-technical value trade-offs, reaching the boundary of its own judgment capacity, detecting silent drift between the execution path and preset contracts, or when the current decision may produce irreversible path-locking effects on subsequent checkpoints.

The engineering carrier of EC-7 is **behavioral transformation**: when an LLM encounters an open judgment, it must transform a "fill-in-the-blank question" into a "multiple-choice question" or "true/false question." Not throwing back a "what do you think we should do?" and shifting the cognitive load onto the human, but providing 2–4 concrete options, each accompanied by impact scope and risk assessment, so that what the human does is "pick one" rather than "invent one"[6].

The AC Paradigm's `ec7_action_gate` mechanism hardens this principle into the tool invocation path: when any destructive file operation (DeleteFile, Write overwriting an existing file) acts upon a protected path (public/, .trae/rules/), the operation is intercepted at the tool invocation level, and the system must first obtain explicit human authorization via AskUserQuestion. This is not a soft convention of "it's recommended to ask first," but an **operational pre-gate**—code substantively blocks unauthorized destructive operations on the execution path.

6.3.3 Rollback and Isolation: Limiting the Propagation Radius of Errors

Power governance must control not only "what can be done" but also "how far the impact spreads after something is done wrong." This points to two engineering compensation conditions: EC-1 (Recovery and Rollback) and EC-2 (Isolation and Boundaries).

The engineering meaning of EC-1 requires distinguishing two levels: **hard rollback** and **soft rollback**.

Hard rollback is the most direct understanding from engineering intuition: state between checkpoints must not only be "transferable" but also "reversible." If the closure determination of checkpoint c_i is later found to be erroneous (tests passed but the direction was wrong), the system cannot have only the path of "forge ahead." It must be able to return to c_{i-1} or an earlier known-valid state and re-fork from there. Git revert in version control, restoration of filesystem snapshots, rollback of database transactions—these all fall within the domain of hard rollback. The precondition for hard rollback is that the object being rolled back possesses a clear operational boundary: a line of code, a file, a database record, with a definite inverse mapping between the rollback action and the operational object.

But in Long-Horizon Practice there is another class of errors, more insidious and more fatal, that lies outside the range of hard rollback. It is not "this one line of code was wrong" but rather "this one round of decision-making was wrong." In research-type tasks, a hypothesis choice made in the fourth round may only be discovered as erroneous in the final round. This hypothesis drove all subsequent reasoning, forks, and outputs from the fourth round to the final round—these outputs are locally self-consistent, complete, and formally correct, but all rest upon an erroneous premise. This is not an error that git revert can fix: no single "file" carries this hypothesis, no single "commit" marks this decision node, and no mechanical inverse operation can rewind the system to the state before the hypothesis choice.

This is the domain of soft rollback. What soft rollback reverses is not operations but **cognitive trajectories**—returning to a decision node, re-examining the information conditions, alternatives, and selection basis at that time, and then re-forking from that node. Soft rollback cannot

rely on mechanical means, because the objects it reverses (hypotheses, judgments, preferences, value trade-offs) were never explicitly recorded as addressable state units. They hide in some turn of the conversation history, are embedded in some paragraph of the reasoning trajectory, and are distributed across the outputs of multiple sub-agents.

The feasibility of soft rollback thus depends on a precondition: **traceability**. If the system cannot answer ”at which checkpoint, based on what information, ruling out which alternatives was this conclusion reached,” then when this conclusion is later falsified, the executor faces not a multiple-choice question of ”where to go back to” but the mandatory question of ”start over from scratch.” The cost of starting over grows nonlinearly with task scale: when a task contains 50 checkpoints, 30 sub-agent outputs, and thousands of conversation messages, traversing all information units to locate the error source is operationally infeasible from the observer’s standpoint (whether human or LLM). It simultaneously exceeds human attention bandwidth and the LLM context window. Rollback in an untraceable system at large-scale long-horizon tasks is approximately equivalent to redoing everything.

This is precisely where state governance (§6.2) and power governance intersect. The ”Handoff Status” in the three-part handoff (closed / open / blocked / currently undecidable) is not only an anchor for continuation but also a signpost for soft rollback. Every checkpoint marked ”closed” becomes suspect when soft rollback occurs: do the premises on which its closure determination depended still hold? Checkpoints marked ”currently undecidable” are even higher-risk zones for soft rollback—they mean the executor at that time had already recognized uncertainty but chose to defer rather than resolve. Soft rollback requires that the system retain at each checkpoint not only ”what was done” (Engineering Process) and ”whether it was done right” (Final Result), but also ”why it was done this way” (decision basis and ruled-out alternatives). Missing this third item, soft rollback degenerates into hard rollback—returning to the previous checkpoint but unable to determine whether that checkpoint’s direction was itself already wrong.

The decision authority for rollback—whether hard or soft—belongs to the human. Rollback means abandoning completed outputs and re-forking. The rationality of this decision can only be judged at the value closure layer: does the expected benefit of the new path after rollback exceed the cost of abandoning existing outputs? The model cannot judge this.[4]

The engineering meaning of EC-2 is: module isolation in parallel development, boundary constraints between sub-tasks, differential granting of permissions across different contexts—these are not questions of ”architectural cleanliness” but of **error containment**. In a system without isolation, a single sub-module’s transgressive operation can contaminate global state: deleting a shared file, modifying a shared configuration, writing an erroneous data record. Once such contamination occurs, the cost of locating and repairing it grows nonlinearly with system scale.

EC-2 elevates isolation from ”good engineering practice” to ”necessary governance condition.” A system without isolation in long-horizon tasks is not ”possibly” going to have problems—it is ”inevitably” going to have problems. Because the probability of error accumulates with the number of checkpoints, and the loss of isolation amplifies the propagation radius of each error.[5]

[4] EC-1 is one of the seven engineering compensation conditions of Long-Horizon Practice theory, requiring the system to return to a known-valid state when closure determinations are erroneous, execution drifts off course, or the environment changes. A single erroneous closure determination should not permanently contaminate the entire practice chain.

[5] EC-2 requires the system to limit the error propagation radius, preventing a single local error from contaminating the entire practice unit. Isolation is not a substitute for checkpoint decomposition but rather limits the impact scope of a single error, keeping errors within local, diagnosable, and rollback-able ranges.

The above §§6.1–6.3 complete the full exposition of the Eukaryon core framework: state governance (four-tier persistence spectrum, context degradation path, three-part handoff, memory vs. state boundary) and power governance (three-layer permission pipeline, value closure and EC-7 hardening, hard/soft rollback distinction and the traceability precondition). These two dimensions are not mutually substitutable and must exist simultaneously—state fractures cannot be repaired by permission rules, and power transgressions cannot be repaired by persistence. This is the meaning of this chapter's title "The Real Battlefield": the convergence of Prokaryon is merely the starting point; the true structural differences between Agent systems lie at the governance infrastructure layer. The next chapter pushes this framework toward two directions of extension—multi-agent scenarios and governance cost economics—as a stress test of the core framework.

7 Extensions of the Eukaryon Framework: Multi-Agent Scenarios and Governance Cost Economics

Chapter Positioning. Chapter 6 established the core framework of Eukaryon—the dichotomy of state governance and power governance, the three-layer permission pipeline, and the structured three-part handoff. This chapter demonstrates the extension of this framework in two directions: multi-agent scenarios (horizontal—one framework governing multiple agents) and governance cost economics (vertical—the engineering legitimacy of the framework itself). Together they verify the explanatory power and predictive power of the Eukaryon framework.

§6 established the Eukaryon core framework for a single Agent instance—the state governance vs. power governance dichotomy, the three-layer permission pipeline, and the structured three-part handoff. This chapter pushes this framework toward extensions in two directions: multi-agent scenarios (horizontal extension—when multiple Agents exist in a system, how power is attributed and how state is isolated) and governance cost economics (vertical argument—whether the "heaviness" of Eukaryon is worth it in engineering terms). The two are not optional appendices to the core framework but rather a **stress test** of it: a truly self-consistent governance framework must maintain explanatory power and predictive power across both dimensions of multi-agent and cost constraints; otherwise, its core claims are incomplete.

7.1 Multi-Agent Scenarios: Attention Allocation, Power Attribution, and Planning Layer Guidance

7.1.1 Core Principle: Sub-Agent Reasoning Depth and Information Acquisition Should Not Be Constrained

A common design impulse in multi-agent systems is to impose capability limits on sub-agents—"sub-agents may only read files, not write," "sub-agents may not invoke certain tools," "sub-agent reasoning depth must not exceed N rounds." The intuitive source of this impulse is reasonable—if the master agent can already make mistakes, the dispatched "subordinates" are even less trustworthy.

But this intuition conflates the **governance layer** with the **capability layer**. The reliability of a sub-agent does not depend on constraining its tools and reasoning depth, but rather on three things: (1) the behavioral reliability of the base model itself (model-level behavior, not controllable by the framework); (2) the reception and verification of sub-agent outputs by the

Eukaryon governance layer (state sovereignty—who may write sub-agent conclusions into the master state); (3) the correct decomposition of sub-agent tasks by the planning layer (whether tasks are sufficiently clean, whether context is sufficiently independent, whether closure criteria are sufficiently explicit).

Proposition 7.1 (Principle of Non-Diminishing Sub-Agent Reasoning Depth)**.** *A sub-agent's capabilities and depth should not be constrained. Constraints should be applied at three positions: model-level behavior, the Eukaryon governance layer's reception and verification of sub-agent outputs, and the planning layer's abstraction and closure criteria for sub-tasks. The more abstract and higher-level the task, the greater the nesting depth it may require—because decomposing high-level tasks demands understanding more complex semantic topologies, which in turn requires more reasoning rounds.*

The practical guidance value of this principle is: when sub-agent outputs are limited or of insufficient quality, the first thing to check should not be "is the sub-agent too free," but rather "is the master agent's task assignment to the sub-agent sufficiently clean, are closure criteria sufficiently explicit, and are sub-agent outputs independently verified upon reception." Constraining sub-agent tools and depth treats symptoms—it degrades sub-agent output quality (insufficient information leads to inadequate reasoning) while not addressing the root cause.

7.1.2 Performance Side: Subagent as a Structural Response to Medium Imposition

The raison d'être of multi-agent architectures cannot be explained solely by "division of labor"—if division of labor were merely an engineering preference, it would not be an architectural necessity. The true necessity of the subagent arises from **medium imposition**: the context window has a ceiling, attention attenuates with context length, and the cumulative context of long-horizon tasks inevitably breaches these hard limits. The subagent does not bypass medium imposition, but rather shifts the pressure of medium imposition from the main thread into smaller, closed contexts—leveraging the informational advantage that "the smaller the problem, the cleaner the context."

The optimal performance projection ($\mathcal{P}_{\text{opt}}$) concept from the AC technical report provides a formal explanatory framework[6]. The same base model exhibits markedly different reliability under long sequences (main thread) and short sequences (subagent): the main thread, as a long-sequence model, even with small per-round context increments $c_{\text{main}}(t)$, has cumulative performance projection $\mathcal{P}_{\text{main}} = \int c_{\text{main}}(t)dt$ that increases monotonically with t and easily breaches the $\mathcal{P}_{\text{opt}}$ threshold; the subagent, as a short-sequence model, even when fed relatively large context per round, has an extremely short integration interval, making its cumulative projection $\mathcal{P}_{\text{sub}}$ far smaller than the main thread's and extremely unlikely to trigger threshold breach. This means—given sufficient context concatenation—the subagent's performance on local execution tasks is, in the vast majority of cases, **greater than or equal to** that of the main thread. This is not because the subagent is "smarter," but because it effectively resets the integration starting point ($t = 0$), pulling the performance projection back to a safe waterline.

This insight explains why **EC-3 (Independent Evaluation)**[6] is highly effective: using the same base model to launch an independent review subagent (such as GN-004) works not because it is "better at reviewing," but because it executes the review task in a clean context with its performance projection within the safe waterline—whereas when the main thread self-reviews

[6]EC-3 requires that acceptance not rely on executor self-reporting, but must be borne by tests, checkers, type systems, human acceptance, or other independent evidence chains. It is the engineering mandatory realization of SC-3 (Open-Item Marking)—ensuring that open-closure signals do not depend on the executor's self-report.

in a long context, its performance projection may already be approaching or even breaching the $\mathcal{P}_{\text{opt}}$ red line.

7.1.3 Governance Side: State Sovereignty, Commit Authority, and Responsibility Attribution

Multi-agent scenarios push the governance problems of Eukaryon to a new level of complexity: who may write sub-agent conclusions into the master state? Under what conditions are sub-agent outputs considered ”verified”? When a sub-agent’s error causes the main thread to drift, how is responsibility attributed?

State Sovereignty. Sub-agents may not autonomously write conclusions into the master state. The main thread’s state store is the ultimate guardian of global consistency—sub-agents may only submit outputs, which the main thread’s governance layer verifies upon reception (contract validation, independent evaluation, human confirmation) before they may be written into the master state. The engineering implication of this constraint is: the main thread must retain final veto power over ”what enters the global state.”

Commit Authority. Tool execution results may be directly reinjected into the conversation history (this is a protocol requirement of Prokaryon), but structured conclusions of sub-agents (analysis reports, code changes, architectural decisions) must pass through an explicit ”submit–verify–accept” three-step process before entering the main thread’s decision basis. The verification step in these three steps is independent of both the sub-agent and the main thread—it is borne by an independent review subagent or a human.

Responsibility Attribution. In multi-agent systems, when an error is discovered, it must be traceable to which agent at which checkpoint introduced the error. This is the extension of EC-3 (Independent Evaluation) to distributed scenarios: not only must each output be independently verified, but a traceable chain between outputs and their executors must be maintained. Without this chain, error localization degenerates into ”finding a needle in the haystack of all sub-agent outputs.”

7.1.4 Presentation Side: User Supervision of Ultra-Deep Sub-Agents

When sub-agents have both large nesting depth and large reasoning depth, how can the user supervise system state without overload? This belongs to the general problem of presentation layer design (Ch11 will derive general constraints on the presentation layer from EC-6/EC-7). Here we only note the Eukaryon framework’s framing of this problem: the feasibility of supervision depends on whether state governance provides convergent handoff anchors at each level of nesting—if a sub-agent, upon completing its task, condenses its results into a structured report containing the three-part handoff information, the user can supervise at the checkpoint granularity without needing to trace the sub-agent’s internal reasoning trajectory.

The progressive disclosure of the Cortex layer (§8) will also provide a partial answer at the capability filtering level—reducing the cognitive load of human supervision through selective exposure rather than full push.

7.2 Governance Cost Economics

After reading the core argument of Chapter 6, the reader may harbor a perfectly reasonable objection: the governance mechanisms of Eukaryon—three-part handoff of anchor documents, frequent launches of independent review subagents, EC-7 human escalations, EC-1 rollback

mechanisms—appear ”heavy.” In a single task, do the extra token consumption and latency of these governance mechanisms outweigh their benefits?

This is a question that must be answered head-on, because it touches the core of Eukaryon's engineering legitimacy.

7.2.1 Eukaryon Governs Not Morality, but Three Scarce Resources

The governance mechanisms of Eukaryon do not answer ”what an Agent ought to do to be moral”—that is a question beyond the scope of engineering. Eukaryon manages three types of **scarce resources**, whose scarcity originates from objective constraints rather than design preferences:

1. **Context Window (Attention Resource).** Window capacity is a fixed hard ceiling. Every unfiltered historical message, every uncompressed tool output, every lengthy reasoning trajectory consumes this finite resource. The governance layer's compression, filtering, and summarization strategies are, in essence, allocation mechanisms for attention resources—distributing the limited window capacity to the currently most critical information.

2. **Token Budget (Compute Resource).** Every model call incurs real computational and time cost. In long-horizon tasks, an ungoverned system may consume massive tokens on erroneous paths before the problem surfaces—and wasted tokens cannot be recovered. The governance layer's independent review, while adding token overhead at each checkpoint, prevents the catastrophic waste of ”running the entire task on an erroneous path.”

3. **Human Attention (Judgment Resource).** In Long-Horizon Practice theory, H1 (Asymmetric Verification Bandwidth) points out that the human attention budget $B(t)$ is monotonically non-increasing with cumulative time. The governance layer's information compression and structured presentation (EC-6: Machine-to-Human Proactive Explanation; EC-7: Machine-to-Human Proactive Escalation) aim not to make humans make more judgments, but to ensure that every human judgment has a sufficient information basis, so that the limited attention budget is spent on the most valuable decision nodes.

From this perspective, safety and cost are not two independent design dimensions. They are **two faces of the same structure**. A single long-horizon advance in the wrong direction is simultaneously a safety risk (unusable output) and a cost risk (wasted tokens). ”Fully automatic” and ”human-machine collaborative” are also not a binary opposition. They are different configurations under different resource constraints. When a task's information completeness is sufficiently high, closure criteria are sufficiently explicit, and the risk of external drift is sufficiently low, the system can slide toward the automation side, reducing the frequency of human intervention; when the task's semantic openness rises, the system must slide toward the collaboration side, pulling in human judgment at key nodes.

7.2.2 The ”Heaviness” of Eukaryon Is a High-Quality Hedge Against Irreversible Drift Risk

Now we can directly answer that objection. Yes, Eukaryon is ”heavy”—anchor handoffs, independent reviews, human escalations, each incurring additional token and latency costs. But this ”heaviness” is not design redundancy; it is a **high-quality hedge against the risk of irreversible drift in long-horizon tasks**.

Consider the following comparison. An ungoverned ”lightweight” Agent, in a 30-round long-horizon task, might advance continuously without any checkpoint verification. The first

15 rounds appear normal. At round 16, the model makes a silent drift—it believes it understands the user's intent, when in fact by round 10 the user's needs had already subtly drifted due to external environmental changes, but this drift was never perceived by the system. From round 16 to round 30, the system efficiently consumes massive tokens in the wrong direction, producing a complete result that "looks done but is in fact directionally wrong." These 15 rounds (or more) of token consumption are pure waste. And more fatally, this error is not "the system crashed." The system did not crash; what it produced is self-consistent, complete, and formally correct—it is just "wrong." Such false closure may be hard to discover within a single session and may only surface in downstream tasks.

Contrast this with a system governed by Eukaryon. At the round-10 checkpoint, it conducts an independent review (consuming an extra 2000 tokens), and the review discovers ambiguity in the open-item markings. The system escalates to the human once via EC-7 (consuming an extra 30 seconds of human attention), the human confirms the need drift and corrects the direction. From round 11 onward, the system advances in the new correct direction, and by round 30 produces a genuinely usable result. Total governance overhead accounts for 10%–20% of task tokens, but it avoids over 50% of ineffective consumption—not to mention the cascading waste that false closure would cause in downstream tasks.

> **Better to spend tokens on independent review than to let the base model produce false closure in long contexts and propagate it backward. Governance cost is a hedge against drift risk, not a net loss of efficiency.**

7.2.3 Directions for Optimizing Governance Cost

Acknowledging that Eukaryon's governance cost is a necessary hedge does not mean that current governance cost is already optimal. The following three optimization directions act on different stages of the governance cycle: adaptive regulation reduces the total number of review triggers, caching and reuse reduce the token overhead of individual reviews, and structured sedimentation transforms human judgments from one-off adjudications into reusable rules. The three advance in parallel, with no sequential dependency.

1. **Adaptive Regulation of Review Frequency.** Current independent review trigger frequency is rule-based (triple mandatory trigger of T1/T2/T3), not risk-based. If the system could dynamically adjust review density based on the current task's semantic openness, historical drift frequency, and executor reliability signals—densifying reviews on high-risk paths, relaxing intervals on low-risk paths—governance cost could be significantly reduced without sacrificing safety.

2. **Caching and Reuse of Governance Overhead.** Repeated reviews of the same checkpoint—for instance, re-review after correction—are, in current implementations, complete re-reviews that do not leverage conclusions already verified in the previous review. If the independent review subagent could produce structured review caches (which parts passed, which parts were blocked), corrected re-reviews could target only the blocked parts, substantially reducing repeated token consumption.

3. **Structured Sedimentation of Human Judgments.** Currently, human adjudication results in EC-7 are recorded in anchor documents but are not systematically encoded as automatically reusable governance rules. If every human adjudication—"in this situation, choose option B over option A"—could be sedimented as a machine-readable governance rule, subsequent encounters with similar scenarios could directly reference the sedimented rule without needing

another escalation. This is the key step from ”manual gradient descent” toward ”automation of governance rules,” and also a foreshadowing of the theme to be unfolded in Ch13: ”Boundaries and Fusion of Systems Engineering and Agent Engineering.”

These three directions together point to a single engineering judgment: the governance cost of Eukaryon is not a constant—it decreases with engineering maturity.

8 Cortex: Progressive Disclosure as a Capability Supply Chain

Chapter Positioning. This chapter is the capability-layer supplement to Decision Five (Separation of Capability and Governance). Having established the infrastructure of state governance and power governance, one must decide ”what tools to give the Agent”—this decision cannot rest on the assumption of ”the more the better,” but must find the engineering optimum between capability and cognitive load. The design goal of the Cortex（皮层） layer is ”correctness”—letting the model see the right capability at the right time.

§5 showed that the seven systems share the same invocation skeleton at the bottommost layer, and §6 revealed the deep divergence among the seven systems on state governance and power governance. But at the outermost layer—the tool sets, skill libraries, and extension capabilities that systems provide to models—the differences among the seven systems reach their maximum. This chapter argues: this layer of divergence is correct architectural layering.

8.1 The Semipermeable Membrane: The Selective Boundary Between the Model's Attention Space and the External World

The fundamental problem of the Cortex is **selective permeability**. The model's attention space is a fixed-capacity token budget—the effectively usable interval of the context window is locked by the objective constraint of the attention budget. Meanwhile, the external capabilities that can be connected—tools, knowledge sources, Skill libraries, services—are theoretically unbounded. This tension forces a layer of selection mechanism: at each step, the system must decide in a principled manner which capabilities are allowed to pass through the boundary into the model's attention space and which are blocked outside. The Cortex is this boundary—a **semipermeable membrane**.

The Pipe Structure of the Membrane. Two types of pipes are distributed across the membrane:

- **Penetrating Pipes (Model-Side Visible)**: The model perceives their existence or effects—Skills descriptors, MCP tool declarations, RAG retrieval results, Memory recall. These pipes expand the model's information or action boundaries and directly participate in the model's attention allocation.
- **Non-Penetrating Pipes (Pure System-Side)**: The model does not perceive their existence—heartbeats, audit logs, automatic snapshots, Hooks/Guards (intercept without notifying the model). These pipes maintain the system's survivability, security, and observability but do not enter the model's attention space.

A key asymmetry exists between these two types of pipes: **pure system-side pipes can exist independently; pure model-side pipes cannot.** Any model-visible capability (a Skill descriptor, a tool declaration) must have a system-side infrastructure that enables it to be ”seen”

and "executed." Formally, let system-side existence be 1 and model-side perception be 1; pipes on the membrane can only take

$$(1,0) \text{ or } (1,1), \quad (0,1) = \emptyset.$$

$(0,1) = \emptyset$ is a structural constraint, not a fillable state. If someone claims to have designed a "pure model-side Cortex capability," it has either already been internalized into Eukaryon (become resident context, no longer belonging to Cortex) or it necessarily depends on some stateless API on the system side.

This asymmetry precisely distinguishes Cortex from Prokaryon and Eukaryon.

Prokaryon is responsible for parsing tool calls in model outputs into actual execution. Eukaryon is responsible for defining the permeability rules of the membrane: state governance governs inbound permeability conditions (what information may penetrate the membrane to enter the model under what conditions), and power governance governs outbound permeability conditions (what actions of the model may penetrate the membrane to reach the external world under what conditions). Cortex is responsible for executing the permeability rules: pipe registration, scheduling, and toggling on the membrane.

The division of labor among the three layers on the tool invocation path is as follows: Cortex executes membrane permeability (pipe toggles control what the model sees), Eukaryon defines permeability rules (under what conditions penetration is allowed), and Prokaryon executes the calls that have already penetrated (the protocol engine).

8.2 Two Dimensions of Penetrating Pipes and Mechanism Positioning

In current Agent engineering practice, the capability mechanisms of Cortex appear in many forms: MCP (Model Context Protocol) provides a standardized tool discovery and invocation protocol; Skills provide programmatic knowledge units that can be loaded on demand; Hooks insert custom behavioral interception points before or after model reasoning or tool invocation; Plugins introduce external capabilities through generic extension interfaces. These names are conventions in engineering practice—they describe "what people currently call things," but they do not answer "what architectural problem the Cortex is actually solving." To answer this question, one must first establish an analytical coordinate system for Cortex capabilities.

Two Dimensions of Penetrating Pipes. For pipes that penetrate the membrane (capabilities perceivable by the model), two essential dimensions determine their governance attribution:

Dimension One: Direction.

- **Inbound** (External $\rightarrow$ Model Context): External information injected into the model's attention space. Governance attribution $\rightarrow$ State Governance (relevance, timeliness, context budget). Failure mode $\rightarrow$ attention dilution or silent omission.
- **Outbound** (Model $\rightarrow$ External World): Model intent reaching the external world. Governance attribution $\rightarrow$ Power Governance (permissions, sandboxing, rollback). Failure mode $\rightarrow$ irreversible damage or transgression.

Dimension Two: Activation Condition (three discrete positions on a spectrum):

- **Resident**: Always present, does not vary with context (system prompt, project rules). Highest attention cost, trigger precision is 1.
- **Conditionally Available**: Whether to penetrate is dynamically determined based on task state, permission mode, or checkpoint type (permission mode switching changes the visible tool set, loading relevant Skill descriptors based on the current checkpoint).

- **On-Demand Triggered**: Triggered by semantic matching or explicit model request (RAG retrieval, Memory recall, model requesting to load full Skill content, result reinjection after tool invocation). Lowest attention cost, trigger precision depends on match/retrieval quality.

Positioning of Existing Mechanisms on the Membrane. With the two dimensions of direction and activation condition, MCP / Skills / Hooks / Plugins cease to be classification frameworks—they are conventional names for specific positions on the membrane. The full lifecycle of a single name may encompass multiple penetration events:

- **Skill Descriptor Injection** → Inbound × Resident or Conditionally Available (descriptor pushed into context by the framework).
- **Skill Full Content Loading** → Inbound × On-Demand Triggered (framework auto-match or model explicit request).
- **MCP Tool Declaration** → Outbound × Conditionally Available (tool visible in the list).
- **MCP Tool Invocation** → Outbound × On-Demand Triggered (model initiates the call) + Inbound × On-Demand Triggered (result reinjected).
- **Hook** → Non-Penetrating Pipe (pure system-side outbound interception—belongs to the execution arm of power governance, whose governance logic has been fully discussed in §6).

Names are contingent; membrane positions are essential. The predictive power of the framework derives from this: any future new protocol or standard need only answer three questions to be precisely positioned and have its governance requirements derived: inbound or outbound? When activated? What is the per-penetration cost?

Based on the above membrane positioning, a general architectural principle can be refined.

Proposition 8.1 (Capability–Governance Separation Principle)**.** *The architecture of an Agent system should separate "what the capability is" (belongs to Cortex, pipe registration on the membrane) and "when and how the capability is activated" (belongs to Eukaryon, permeability rules of the membrane) into two independent design dimensions. Cortex is responsible for organizing the pipes on the membrane: defining tool interfaces, managing skill lifecycles, registering interception points; Eukaryon is responsible for governing membrane permeability: deciding which pipes penetrate under the current permission mode, which skills should be activated at the current checkpoint, which interception points should trigger in the current context. The two are decoupled through a unified registration interface, so that capability providers need not know the permeability rules, and permeability rules need not understand capability details.*

8.3 Progressive Disclosure as a Membrane Scheduling Problem

In the design discussions of Agent systems, "Progressive Disclosure" is often categorized as a UI/UX strategy—"don't dump all menu items on the user at once; unfold gradually as needed." But when this intuition is transplanted to the Cortex layer, the subject shifts from "user" to "model": it is no longer "don't dazzle the user" but "don't inject all capabilities into the model's attention space at once." This shift is not rhetorical but structural: it is not a friendly design to "keep the model from being confused," but a resource allocation decision forced by the objective constraint of the attention budget. Under fixed throughput and facing an unbounded capability space, the membrane scheduling problem is unavoidable.

Recall the three types of scarce resources introduced in §6: context window (attention resource), token budget (compute resource), and human attention (judgment resource). Among

these three, the first two bear direct pressure at the Cortex layer. A typical Agent system may have 50+ registered MCP tools, 20+ Skills, and 10+ Hook interception points. If each tool declaration's JSON Schema consumes 500–2000 tokens on average, fully exposing a medium-scale Cortex capability library would consume 25000–100000 tokens for tool declarations alone. The problem is not "it won't fit"—modern large models' context windows run to millions of tokens, and accommodating these declarations is not infeasible in capacity terms. The real problem is the proportion inversion: when the token overhead of tool declarations approaches or even exceeds the tokens of actual task content, we face a situation where "the soy sauce costs more than the chicken." The metadata describing capabilities itself becomes the main consumer of the attention budget, squeezing out the content that truly carries task semantics. The bottleneck of the attention budget is not the capacity ceiling but the fact that metadata should not eat up the bulk of the cognitive budget.

Thus, Cortex's progressive disclosure is a structural necessity, not an optimization.

Its engineering meaning is not controlling the pace of information presentation at the UI level, but rather **implementing independent on-demand projection mechanisms at three levels—the tool registration layer, the skill loading layer, and the interception-point activation layer**—ensuring that the model sees only the capability subset most relevant to its current context in each round.

This judgment elevates Cortex's progressive disclosure from a presentation-layer topic to a membrane scheduling strategy.

Progressive disclosure at the presentation layer—an IDE highlighting only the current diff rather than displaying the full change history, a terminal showing only the most recent N lines of output rather than the full log—is the same principle manifested at the outermost layer (§11 will unfold the presentation layer's own convergence logic). But it is not an "application" of the Cortex layer's progressive disclosure; rather, both are independent realizations of the same principle at three different stack layers. The three layers are not mutually substitutable: capability filtering at the Cortex layer cannot substitute for information compression at the presentation layer, and vice versa.

It is worth noting that projection is not a phenomenon exclusive to Cortex. Even at the bottommost Prokaryon layer, a passive form of projection exists: the conversation history the model sees at invocation round t only contains the tool results actually invoked in previous rounds—tools not invoked, capabilities filtered out, and Skills not activated never appeared in the context. This is not an actively designed filtering mechanism of Prokaryon but rather a byproduct of the protocol itself: only calls actually executed leave traces in the history. The significance of this observation is: **projection is an inherent property of every layer of an Agent system; the difference lies only in whether it is actively designed or passively produced, coarse-grained or fine-grained.**

8.4 Hermes Tool Registry: An Engineering Exemplar of the Cortex Layer

Hermes's Tool Registry is designed around one core problem: how to let the model see only the capability subset relevant to the current context in each round, while making capability registration, discovery, and lifecycle management as automated as possible. Among the seven systems examined, it is the clearest on this problem. Three complementary mechanisms together form the complete answer.

(1) AST Scanning—Zero Manual Registration. At startup, Hermes scans Python source code in specified directories, uses AST static analysis to identify all functions marked with the `@tool` decorator, automatically extracts signatures, parameter types, and docstrings, and

generates standard tool declaration JSON. No code is executed during this process. Engineering implication: adding a new tool requires only writing a Python function with the decorator. The registry auto-syncs on the next startup; no independent registration manifest needs to be maintained.

(2) check_fn + TTL—Conditional Projection. Each tool can be bound to a `check_fn` (boolean predicate) and a TTL. The system executes `check_fn` before each round of calls, dynamically determining whether the tool should be visible to the model in the current context: check conditions cover permission mode, session state, and external environment. TTL provides a time-dimension expiration mechanism: after a specified number of rounds, the tool automatically becomes invisible, forcing re-evaluation.

A refined implementation detail: check_fn results are cached for 30 seconds, but if a tool recently returned as available and then briefly became unavailable, the system grants a 60-second grace window (preventing transient fluctuations such as Docker daemon load spikes from silently stripping the entire toolset), during which the most recent available result is used; failure results are not cached, ensuring the next call re-probes.

(3) Transient Fault Tolerance—Graceful Degradation. When check_fn execution fails or scanning encounters anomalies, the registry does not crash. Faulty tools are marked "temporarily unavailable," and the system continues running with a reduced capability set. This embodies the correct engineering posture of Cortex: **unavailability of a capability does not equal unavailability of the system.** This forms a structural contrast with Prokaryon. The model API is a hard dependency (Prokaryon layer, disconnection means system halt); tool registration is a soft dependency (Cortex layer, disconnection means degraded operation).

These three mechanisms—discovery (AST scanning), projection (check_fn + TTL), degradation (transient fault tolerance)—verify the engineering feasibility of Proposition 8.1: Cortex organizes the capability repository, Eukaryon's check_fn decides when to project, and the two are decoupled through a unified registry interface.

8.5 Architectural Properties of the Cortex: Pluggability and Division of Labor with Eukaryon

Beyond the Tool Registry, Hermes has also built a complete memory and skill management system: the Memory Provider handles cross-session knowledge retrieval and writing, and Curator manages the skill lifecycle (creation → activation → archiving → retirement). This system possesses a structural feature worth noting in engineering terms: memory retrieval and writing are automatically triggered each round, without depending on model requests; memory tool schemas are injected into the tool list at agent initialization; Curator runs continuously as a resident background system. These features mean that Hermes's memory system has in fact crossed the boundary between "optional Cortex plugin" and "quasi-Eukaryon infrastructure" as defined in §6.2—it is not detachable, but deeply embedded in the agent's respiration cycle.

This points to an empirical judgment with corrective significance for the three-layer model: **the boundary between Cortex and Eukaryon is not a static wall but a spectrum.** When a Cortex capability—whether tool registration or memory management—is made sufficiently heavy, sufficiently infrastructuralized, it naturally acquires Eukaryon properties. "Should it be placed in Eukaryon" is not a question answerable by a priori principles—it depends on what weight that capability has reached under the specific evolutionary pressures of the specific system. Hermes's empirical value lies precisely in simultaneously demonstrating the successful side of Cortex progressive disclosure (the Tool Registry) and the boundary side (the memory system).

Both point together to the same conclusion: the separation of Cortex and Eukaryon is necessary architectural layering, but the specific dividing line between the two is not a constant—it is an engineering decision that requires ongoing judgment. When a memory system is made sufficiently infrastructuralized, it may acquire Eukaryon properties; this threshold varies by system and cannot be drawn a priori.

From this spectrum judgment, one can naturally ask: what is the most fundamental architectural property of Cortex as an architectural layer? The answer has two dimensions—**pluggability** (what it itself is) and **the division-of-labor interface with Eukaryon** (with whom and how it collaborates).

Pluggability. Cortex is the layer with the greatest divergence among the seven systems: tool ecosystems, Skill organization approaches, host integration strategies—no two adopt the same approach. This is precisely correct architectural layering. Its core meaning is: different Agent instances can have different Cortex (tool sets, Skill libraries, host integrations) while sharing the same Prokaryon + Eukaryon foundation. The core loop and governance structure are architecturally fully consistent. Forcing the diversity of the capability layer into a unified standard harms domain adaptation precision; meanwhile, the convergence of Prokaryon and Eukaryon allows Cortex to fully differentiate without compromising governance consistency.

Recommendation and Clearance: The Interface Between Cortex and Eukaryon. The division of labor between the two on tool visibility can be precisely delineated. Cortex's progressive disclosure governs **relevance filtering**—within the current task domain and context, which tools are worth recommending to the model. Eukaryon's tool visibility (first layer of the permission pipeline, §6.3) governs **permission filtering**—within the candidate set produced by Cortex, which tools are allowed to be visible to the model. In one sentence: Cortex governs "should it be recommended," Eukaryon governs "should it be cleared." Conflating the two leads to misdiagnosing permission insufficiency as capability mismatch and making adjustments at an irrelevant registration layer—this is a direct corollary of Proposition 8.1 at the engineering diagnosis level.

Governance Boundary Judgment. The expansion of the Cortex capability library does not translate uniformly into governance pressure on Eukaryon—the problems introduced by new tools are distributed across multiple layers. Idempotence and observability belong to tool design, whether a task is suitable for toolization belongs to task design, output visualization belongs to presentation-layer adaptation, and only permission boundaries and execution isolation belong to Eukaryon's governance burden. Knowing what not to put into governance is itself part of governance capability. Pushing all non-governance problems into Eukaryon not only increases the governance burden but also blurs precise diagnosis—just as Proposition 6.1 requires distinguishing "amnesia" from "transgression," governance capability likewise requires distinguishing "missing governance mechanisms" from "design defects in non-governance layers."

The two dimensions ultimately converge on a single judgment: Cortex's rich capabilities are the precondition for the existence of Eukaryon governance (with no capabilities to govern, governance runs idle); Eukaryon's governance gates are the precondition for Cortex's safe release. The two are not an opposition of expansion versus restriction, but two faces of the same architectural problem.

9 Structural Requirements of Long-Horizon Practice for Agent Governance

Chapter 7 established that Agent governance occurs at the Eukaryon layer. This chapter argues from the information-structure level: why these governance mechanisms are structural necessities rather than optional engineering enhancements in Long-Horizon Practice. When key inputs cannot be statically folded at the current checkpoint, governance is the precondition for the system not to collapse, not an embellishment.

§3 through §6 completed three preparatory tasks: defining the boundary of narrow-sense Agents and the Three-Layer Concentric Model, demonstrating the divergence of the seven systems at the Eukaryon layer, and establishing that Prokaryon has converged while the real battlefield lies in Eukaryon. But one critical question has not yet been answered head-on: why are Eukaryon's governance mechanisms **structural necessities** rather than optional engineering enhancements? The task of this chapter is to give the structural answer to this question.

9.1 Formal Definition of the Non-Precomputable Sequential Dependency Chain

Before discussing the constraints of Long-Horizon Practice on Agent governance, a preliminary question must be precisely answered: under what conditions does a task **structurally** require governance mechanisms? Not all tasks require checkpoint decomposition, state transfer, and independent review. Short conversations, single-shot queries, and deterministic workflows are adequately served by Prokaryon's bare loop. Governance mechanisms are activated only under specific information-structure conditions. The task of this subsection is to give the precise formalization of these activation conditions.

9.1.1 Static Foldability and Key Inputs

The formal starting point of Long-Horizon Practice theory is a distinction: which information can be obtained through purely internal computation, and which must be generated or imported at the cost of a state transition[5].

Definition 9.1 (Static Foldability)**.** For any checkpoint c_i, let $\mathcal{I}_i$ be the current information set that can be stably invoked at moment c_i. If an information object X can be obtained through finite internal computation—without altering the state of the external world, without executing new probing actions, and without committing to an as-yet-unselected branch—relying solely on materials already possessed within $\mathcal{I}_i$, then X is said to be **statically foldable** with respect to $\mathcal{I}_i$, denoted

$$X \in \mathrm{Fold}(\mathcal{I}_i).$$

Definition 9.1 excludes three classes of things often mistaken for "currently known": feedback that can only be generated by first conducting an experiment or probe (cognitive gaps), information whose type can only be determined after committing to a specific branch (decision forks), and truth values that only become valid after the external world further evolves (external drift). None of these can be counted as inputs directly foldable within the current local closed loop.

Definition 9.2 (Key Input and Sequential Dependency)**.** For the advance from c_i to c_{i+1}, if there exists a minimal information object K_{i+1} such that the action of executing or verifying c_{i+1} presupposes it, then K_{i+1} is called the **key input** of this transition. If

$$K_{i+1} \notin \text{Fold}(\mathcal{I}_i),$$

then c_{i+1} is said to have a **sequential dependency** on c_i, denoted $c_i \mapsto c_{i+1}$.

The engineering meaning of Definition 9.2 is: not every case of "the next step needs the previous step" constitutes a long-horizon dependency. When the previous step serves as simple groundwork (e.g., "read file A first, then read file B"—both files exist at the starting point, and the reading order is merely execution arrangement rather than information generation), it does not trigger long-horizon structure. Only when the key input required by the next step **cannot** be statically folded at the current checkpoint—i.e., the previous step is not merely execution groundwork but a genuine information-generation or information-crystallization event—does sequential dependency become a structural constraint.

Definition 9.3 (Long-Horizon-Practice Interval)**.** If for some checkpoint transition $c_i \to c_{i+1}$, its key input K_{i+1} satisfies

$$K_{i+1} \notin \text{Fold}(\mathcal{I}_i),$$

then this transition is said to enter a **Long-Horizon-Practice interval**. If the execution path of a practice instance contains at least one Long-Horizon-Practice interval, then this instance is called a **Long-Horizon Practice**.

Definition 9.3 gives the activation condition for governance mechanisms. Not all Agent tasks qualify as Long-Horizon Practice—those tasks whose required information is already complete at the starting point, whose sub-problem boundaries can be given once and for all, and whose entire path can be pre-folded, belong to single-shot closed engineering amplification, for which Prokaryon's bare loop suffices.

Governance mechanisms are upgraded from "optional engineering optimization" to "structural necessity" if and only if the condition

$$K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$$

is satisfied.

9.1.2 Long Horizon Is Not Long Duration—It Is Non-Precomputability

The objection raised at the beginning of this chapter—"can't a sufficiently intelligent base model bypass governance mechanisms"—can now be given a precise structural answer. Suppose the base model at checkpoint c_i possesses an arbitrarily large context window and arbitrarily strong reasoning ability, but what it can do is arbitrary complex internal computation on $\mathcal{I}_i$.

Definition 9.1 explicitly demarcates the closure boundary of $\text{Fold}(\mathcal{I}_i)$: the products of static folding can only be combinations, transformations, and derivations of information already within $\mathcal{I}_i$. When

$$K_{i+1} \notin \text{Fold}(\mathcal{I}_i),$$

K_{i+1} has not yet been produced by the world. It could be the error output after a code execution, the genuine feedback from a client after seeing the proposal, the measurement result of the next experiment. These truth values are not in $\mathcal{I}_i$. Arbitrarily strong reasoning ability can

only extrapolate on the basis of existing information and cannot conjure truth values not yet in existence.

This is not a matter of "the model not being smart enough"—it is the structural fact that **the information is not in the current set**.

Just as translating a 1000-page novel is not a Long-Horizon Practice (the key input of each page's translation—the source text—is already complete at the starting point), while diagnosing the fault "the computer won't turn on after pressing the power button" is necessarily a Long-Horizon Practice (the key input of each probing step can only be generated after the previous probing step is executed). Long-horizon is not about large workload, not about many steps, not about long duration—it is that **the key input of the next step cannot be statically folded at the current checkpoint**.

Proposition 9.1 (Structural Correspondence Between Long Horizon and Governance)**.** *Let an Agent system $\mathcal{A}$ be deployed on a practice instance* Π*. If* Π *contains a Long-Horizon-Practice interval (i.e., there exists a* c_i *such that* $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$*), then* $\mathcal{A}$ *must provide engineering support for the following operations in its architecture; otherwise the practice will suffer a structural fracture at that interval:*

1. *Persist the closure status of the current checkpoint in structured form (rather than merely retaining conversation history);*
2. *Provide a semantically annotated state snapshot for the subsequent executor at cross-checkpoint handoff (rather than merely passing raw output);*
3. *Introduce a judgment mechanism independent of the executor at value-uncertain nodes (rather than relying on executor self-report).*

The three requirements of Proposition 9.1 map directly onto the three core components of Eukaryon: state transfer (HC-3, §6.2), open-item marking (SC-3, §6.2), and value closure (SC-1, §6.3). This is not a case of "the AC Paradigm happened to choose these mechanisms."

The information structure of $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ logically compels the corresponding governance components; any Agent system that fails to provide these components will necessarily experience structural fracture at that interval. The collective blank of the seven systems on L4 (handoff governance) in §4 is precisely the empirical projection of this structural constraint in evolutionary history.

9.2 Governance Consequences of the Formal Definitions

The three structural requirements of Proposition 9.1—state transfer, open-item marking, and value closure—correspond respectively to HC-3, SC-3, and SC-1. This subsection directly derives the necessity of these three in Long-Horizon Practice from the formal definitions; the full condition system and engineering compensation conditions are detailed in Zeng (2025).

HC-3 (State Transfer) originates from the direct consequence of $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$: the information set $\mathcal{I}_i$ at the closure of checkpoint c_i does not contain K_{i+1}, and relying solely on the raw output of $\mathcal{I}_i$, the subsequent executor cannot know "why the current state is what it is, what remains undone, where to start next." State transfer is not "passing files along." It is passing a structured snapshot with semantic annotations. This is precisely the formal root of the three-part handoff (Engineering Process + Handoff Status + Final Result; see §6.2 for details).

SC-3 (Open-Item Marking) originates from a corollary of the same structural fact: since K_{i+1} cannot be folded by c_i, c_i cannot independently determine whether it is "truly done"—it can only verify local goal closure within the closure of $\mathcal{I}_i$, and cannot verify whether that closure

still holds once K_{i+1} is incorporated. Therefore the system must distinguish the three states of "closed," "open," and "currently undecidable," and must not rely on executor self-report. An independent verification mechanism (such as GN-004 independent review; see §6.2 for details) must be introduced.

SC-1 (Value Closure) originates from a deeper corollary: HC-3 and SC-3 can only guarantee "not going wrong locally," not "the global direction being right." When the task path contains decision forks, local closure cannot answer "which branch to choose"—this is a value judgment and does not belong to the domain of technical solutions. SC-1 compels the introduction of external judgment at every irreversible fork, and this is precisely the structural origin of EC-7 (Machine-to-Human Proactive Escalation; see Zeng, 2025 for details).

The root cause of

$$K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$$

lies in four sources of information truncation: semantic non-closure (the Agent does not know what to ask next), feedback latency (the Agent must first execute before it can know the correct direction), context attenuation (historical information degrades as the number of steps increases), and boundary drift (the definition of "done" may change during execution). The four sources share a single root cause: feedforward computation cannot substitute for state transition.

These four sources correspond respectively to four types of information-structure fracture and the activation conditions of four governance mechanisms. Semantic non-closure is a cognitive gap, requiring checkpoint decomposition and sequential constraints; feedback latency is medium imposition, requiring subagents and information degradation; context attenuation is decision forking, requiring value closure and human escalation; boundary drift is external drift, requiring independent evaluation and rollback. The systematic diagnostic framework is detailed in Zeng (2025).

9.3 Empirical Projection onto the Seven Systems

Mapping the three constraints derived in §9.2—HC-3 (State Transfer), SC-3 (Open-Item Marking), and SC-1 (Value Closure)—onto the seven-system analysis of §4 enables precise diagnosis of the collective blank of each system on L4 (Handoff Governance).

The L1–L4 continuity capability table of §4 (Table 4) has already recorded the empirical status of all seven systems on L4: trae-agent, OpenCode, Claude Code, and Codex CLI are listed as blank on L4; Cline's Cron reports and Hermes's Kanban are the designs closest to L4, but what they record is "which tasks completed" or "which tasks are queued," not structured cross-session handoff. The three constraints are unfolded item by item below.

HC-3 (State Transfer) Aspect. HC-3 requires that upon checkpoint closure, the system produce a handoff output containing a structured state snapshot, a semantically annotated list of open items, and an actionable continuation entry point. None of the seven systems meets this standard: Cline's Cron reports generate only task-level summaries, without checkpoint-level closure status determinations; Hermes's Kanban maintains task queues but does not distinguish the three states of "closed," "open," and "currently undecidable"; for the remaining five systems, after cross-session interruption, state information is entirely entrusted to execution history (conversation logs, cell sequences, commit chains)—these are chronological records, not structured handoff snapshots. The next executor must read all history and independently infer the current state. This is precisely the structural fracture point of Long-Horizon Practice when $\mathcal{I}_i$ does not contain K_{i+1}.

SC-3 (Open-Item Marking) Aspect. SC-3 requires that under the condition that K_{i+1} cannot be folded by c_i, the system must introduce an independent verification mechanism to dis-

tinguish the three states of "closed," "open," and "currently undecidable." None of the seven systems provides such a mechanism: all of them rely on the human developer to independently judge "is it done yet." This carries controllable risk in single-shot closed engineering amplification (verification criteria are already complete at the starting point), but in long-horizon intervals where $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$, the executor cannot determine within the closure of $\mathcal{I}_i$ whether it has truly achieved closure, and relying on executor self-report amounts to abandoning verification.

SC-1 (Value Closure) Aspect. SC-1 requires that at every irreversible fork, an independent value judgment be introduced. None of the seven systems sets up a mandatory interception mechanism on the tool invocation path; all systems rely on the implicit assumption that "the human will keep an eye on it themselves." When human attention dissipates (see the H1 constraint in Zeng, 2025), this assumption structurally fails. EC-7 (Machine-to-Human Proactive Escalation) is the engineering compensation scheme for this void (see Zeng, 2025 for details).

The collective blank of the seven systems on L4 is not an accidental engineering omission. Their birth scenarios (§4 provides detailed exposition) never required cross-session handoff: each system's initial environmental pressures (single-session terminal, single-window IDE, single-user desktop) made L1–L3 survival necessities, while L4 lay outside any system's initial evolutionary path. But once one acknowledges that Long-Horizon Practice requires a non-precomputable sequential dependency chain of $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$, L4 transforms from a "not-yet-implemented enhancement" into a "necessary component whose absence means structural fracture."

9.4 Three Non-Implications

The definitional system of §9.1 directly yields three groups of non-implication relations. These three groups expose the structural misalignment between current Agent evaluation systems and Long-Horizon Practice:

Proposition 9.2 (Single-Step Capability Non-Implication)**.** *An improvement in the model's reasoning capability within a single checkpoint does not imply an improvement in governance capability across checkpoints. The former only proves "better at a single step" and does not mean the system will not suffer structural fracture in long-horizon intervals where* $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$.

Proposition 9.3 (Cross-Checkpoint Non-Implication)**.** *An improvement in governance completeness across checkpoints does not imply an improvement in general benchmark scores. Benchmarks assume that task information is already complete at the starting point (*$K_{i+1} \in \text{Fold}(\mathcal{I}_i)$ *holds for all checkpoint transitions), whereas the core characteristic of long-horizon tasks is precisely that this assumption does not hold. Evaluating long-horizon capability requires a long-horizon yardstick.*

The complete formal unfolding of the three non-implications and intermediate propositions is detailed in §14.1.

9.5 Convergence

The core conclusion of this chapter is: the criterion $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ provides a formal foundation for the decision framework of §2: Decision One (State Governance) corresponds to the engineering support of HC-3, Decision Two (Power Governance) corresponds to the engineering support of SC-1, and Decision Three (Checkpoint Verification) corresponds to the engineering support of SC-3. Long-Horizon Practice theory is not externally attached complexity, but a formal answer to why these three decisions of the Ch2 framework are non-removable.

10 The Discrete Nature of Agent Practice

Chapter positioning. Decision Four (designing interruption boundaries) states: ”preset interruption surfaces at key nodes.” Having established the need to set interruption points, we must now understand *why* interruption points must be discrete, structured anchors—this is not a preference, but a requirement imposed by the information structure of Long-Horizon Practice. This chapter argues from the information-structural level: a continuous “memory stream” is structurally incapable of replacing the positioning and verification functions of discrete checkpoints.

§9 proved that the information structure of Long-Horizon Practice requires checkpoint decomposition (HC-2), state transfer (HC-3), and open-item marking (SC-3)—none of these three is optional in logical terms. But at the close of its final section, that chapter bracketed a deeper question: **why must the structural carriers of these governance mechanisms be “discrete” anchors and checkpoints, rather than a continuous memory stream or dialogue history?**

10.1 Agent Practice Must Be Engineered as Discrete

The reason Agent systems need discrete governance structures such as checkpoints, anchors, and interruption boundaries is not—as the strongest and most engineering-grounded argument goes—that LLM tokens are discrete. Rather, it is that **discrete checkpoints provide governance boundaries that are recoverable, verifiable, auditable, accountability-traceable, and human-judgment-insertable**. Continuous memory streams or dialogue histories, indexed by time, do not provide validity-determination mechanisms independent of the executor—they cannot replace the structural function of discrete state anchors in governance. The following sections unfold this engineering argument across four layers, and within the argument they progressively respond to the cognitive mechanisms behind the intuition of “continuous experience” and their engineering limitations.

As foundational support for this engineering judgment, LLM practice also exhibits consistent discreteness in its implementation characteristics. There is a reliable operational criterion for judging whether a system is discrete in its engineering implementation: can the system’s state transitions assume uncountably infinite intermediate values? Can its smallest operational unit be infinitely subdivided into a gap-free continuous flow? Does its physical implementation contain genuine continuous quantities? If the answer to all three questions is “no,” then the system is discrete in its engineering implementation—regardless of how smooth the user’s subjective experience feels, such experiences are narrative stitching over a discrete substrate, not ontological properties of the system or the interaction.

The following four layers correspond, respectively, to the verification of the above three criteria one by one, with the fourth layer serving as physical confirmation of the final criterion. Each layer rests directly on verifiable facts about LLM usage, not on philosophical inference.

1. **Both input and output are discrete token sequences.** LLM inputs are segmented into tokens drawn from a finite vocabulary, and outputs are likewise symbol sequences generated by per-token sampling from the same finite vocabulary. There is no continuous semantic space in which the model can make infinitely fine-grained adjustments—it can only make probabilistic choices over discrete symbols. Even “continuous” multimodal inputs such as images and audio must be quantized into discrete patches or tokens before entering the model; otherwise the model is structurally incapable of processing them.

2. **Interaction consists of discrete turns.** Every human–LLM interaction follows a fixed discrete structure: the human types or voice-inputs text (a discrete message) and sends it to the model; the model, upon completing inference, returns one or more discrete replies; the human receives, reads, comprehends, and decides the next action (cite, modify, reject, follow up). Even when the dialogue subjectively feels "fluid," each operational step—send, receive, read—is a discrete event boundary. Streaming output only changes the latency experience at the receiving end; it does not alter this basic structure.

3. **Autoregressive decoding samples at discrete time steps.** The Transformer's autoregressive decoding has no continuous dynamics. At each time step, the model computes, over the currently generated token sequence, a probability distribution for the next token, and then samples a discrete symbol from that distribution. The decoding state's transition from time t to $t+1$ is a discrete jump, not a differential evolution in a continuous state space. The claim that "the reasoning process is continuous" is a figurative description of changes in attention and hidden states—these hidden states are, computationally, discrete matrix operations performed layer by layer, head by head, and position by position, involving no calculus, no limits, no continuous functions.

4. **Physical implementation runs entirely on digital circuits.** LLM inference executes on GPUs/TPUs, and GPUs/TPUs are purely digital circuits—transistor switches toggle between 0 and 1, with no genuine analog continuous quantities present. Model weights are discrete floating-point representations (float16/bfloat16), their precision bounded by finite bit widths. From the physical layer to the algorithmic layer, no part of the entire system carries a continuous physical quantity or computational process.

The above four layers provide a sufficient and independently verifiable argument: LLM practice is discrete in its engineering implementation across the four layers of tokens, interaction, decoding, and physical implementation. "Continuity" is not a property of the system or the interaction itself, but a stitched-together experience that the human user constructs within their working memory from a series of discrete events. This judgment can be formalized as follows:

Proposition 10.1 (The Discrete Character of LLM Practice and Its Governance Consequences)**.** *Let $\mathcal{S}$ be an interactive system with an LLM as its core inference node. Each human–model interaction in $\mathcal{S}$ can be represented as a discrete message sequence $M = (m_1, m_2, \ldots, m_n)$, where each m_i is a finite-length sequence over a finite symbol set Σ. The state transition of $\mathcal{S}$ from time t to $t+1$ is a discrete jump:*

$$s_{t+1} = \arg\max_{w \in \Sigma} P(w \mid s_1, \ldots, s_t),$$

where $s_t \in \Sigma$ is the discrete token sampled at step t. This process involves no continuous dynamics—there are no state variables capable of assuming uncountably infinite intermediate values, and no continuous differential equations describe the system's evolution. The underlying implementation of $\mathcal{S}$ is discrete.

Proposition 10.1 not only describes the discreteness of token sampling but also points to a fact with direct consequences for Agent Governance: **every state transition of an Agent system occurs at a discrete point, and the system state after each transition—lying outside the coverage of the current inference, outside the fold-closure of the current context—is a new state that requires external anchoring mechanisms to be recovered and verified.** This is precisely the core of the argument that the subsequent sections of this chapter (from the cognitive-stitching analysis of §10.2 to the failure-mode diagnosis of §10.5) progressively unfold.

10.2 The True Nature of "Continuous Experience"

If the underlying nature of LLMs is discrete, what then is the "coherent dialogue," "fluid reasoning," and "natural continuity" that users experience in everyday interaction? This question must be answered directly, because the felt reality of continuous experience is precisely the economic source of the intuition that "we can replace discrete state governance with continuous memory"—if experience is already continuous, why erect additional discrete governance scaffolding? This section decomposes "continuous experience" into three layers—cognitive stitching, the streaming illusion, and need-confusion—to reveal its nature as a perceptual service layered atop a discrete substrate.

10.2.1 Cognitive Stitching: Narrative Reconstruction of Discrete Fragments by Working Memory

The "continuity" that humans experience shares the same cognitive mechanism as the continuity experienced when watching a film. A film consists of 24 still frames per second—genuine blank gaps exist between frames. Yet the human visual system, via the phi phenomenon, stitches these discrete frames into a perception of continuous motion. Analogously, during multi-turn dialogue with an LLM, **working memory performs real-time narrative stitching on the discrete token sequences received in each turn**—embedding the previous turn's inference conclusion into the current turn's interpretive context, and mistaking the model's "memory" (via dialogue history in the context window) for the model's "sustained consciousness." This stitching is a function of the human cognitive system, not a property of the LLM system.

The L60 distinction has direct engineering consequences. After a checkpoint interruption, the next executor does not possess the previous executor's cognitive-stitching context—if the system at that point provides only dialogue history as "raw stitching material," the reliability of the handover depends on whether humans at different times and in different states can produce consistent stitching results, and this stitching process is structurally unreliable.

This is precisely the structural reason why **HC-3 (State Transfer)**[7] requires *state* rather than *memory*: a state is an "already-stitched" snapshot; subsequent executors do not need to redo the stitching work themselves.

10.2.2 Streaming Output: Lowers Boundary Perception, Does Not Eliminate Discrete Nature

Streaming output (token-by-token streaming) is the most direct manufacturer of the continuous-experience illusion. The model does not return a complete reply all at once but "streams" text token by token, and during reading the user experiences the sense that "the model is thinking in real time." Yet the essence of this experience remains a token-level discrete flow—each token is a discrete symbol sampled from a finite vocabulary, and there are ineliminable discrete jumps between tokens. Streaming output lowers the human's perceptual sensitivity to token boundaries, but does not eliminate those boundaries themselves.

The engineering import of this distinction is subtle but important. Streaming output is an optimization at the **presentation layer**: it serves to reduce cognitive friction, improve reading

[7] HC-3 (Hard Condition 3: State Transfer), one of the six logically necessary conditions of Long-Horizon Practice theory. The absence of HC-3 leads to information evaporation between execution cross-sections, making it structurally impossible for subsequent checkpoints to continue. HC-3 requires that what is transferred is not dialogue history but a semantically annotated, structured state—what has been done, why, open items, and the continuation entry point. See §6.2 for details.

comfort, and sustain attentional immersion. This is precisely an instance of the progressive disclosure principle within Cortex at the presentation layer. But it cannot replace **state governance** within Eukaryon. Streaming output making the human feel that "the conversation is smooth" does not equate to the system possessing recoverable continuity across checkpoints. The former is a presentation-layer service; the latter is Eukaryon-layer infrastructure. A typical consequence of conflating the two is: mistaking the presentation layer's continuous experience for "the system already has continuity," and thereby omitting state governance from the architectural design.

10.2.3 Humans Like Continuity, but Need Discreteness

We can now return to the core distinction of this section:

> **Continuity is a service provided by the system, not an ontological fact that user intent, authorization, or value judgments are natively continuous.**

Humans **like** continuity: low-latency streaming output, natural dialogue continuation, seamless topic switching—these are reasonable user-experience demands. But humans **need** discreteness: writing requires pausing and revising at the sentence level, programming requires testing and refactoring at the function level, decision-making requires comparing and selecting at the option level. In all these real practices, progress does not occur uniformly on a continuous time stream, but through choices, rejections, confirmations, and reorientations made at **discrete branching points**. Every branching point is an "interruptible boundary"—humans intervene, cut, select, and take responsibility at these boundaries.

Placing the distinction between "like" and "need" in the context of Agent Governance yields a precise engineering principle:

Proposition 10.2 (The Engineering Division of Labor Between Continuity and Discreteness)**.** *In the architecture of an Agent system, continuous experience belongs to the design objectives of the presentation layer—it reduces cognitive friction and improves user experience through streaming output, low-latency responses, and natural-language continuation. Discreteness belongs to the design constraints of the Eukaryon layer—it ensures the interruptibility, recoverability, and accountability of the system at key nodes through checkpoint decomposition, state anchoring, independent evaluation, and machine-to-human escalation. The two are not competing but complementary: the presentation layer wraps discrete governance structures in a continuous experience, so that while the user feels fluidity, the system retains rigid governance at the底层. However,* ***the presentation layer's continuous experience must not be premised on weakening or bypassing the Eukaryon layer's discrete governance****. If "making the conversation smoother" means omitting checkpoint verification or skipping human escalation, then smoothness itself becomes a lubricant for drift.*

10.3 What This Responds To: Clarifying Three Popular Presuppositions

The judgment of discrete nature directly responds to three popular presuppositions within the Agent community that merit reexamination.

10.3.1 Presupposition 1: "Agents Need Continuous Memory"

§6.2 has already established the strict distinction between memory and state: memory, indexed by time, does not provide validity determination; state, indexed by closure, provides structured

semantic annotation. Here we add a crucial qualification from the discrete/continuous framework: treating dialogue history as a continuous videotape and demanding that the Agent "fully recall" to locate the current state is equivalent to shifting the governance burden from structured checkpoints to unverifiable retrieval capability.

10.3.2 Presupposition 2: "Agents Need Uninterruptible Sessions"

A common form of this claim is: session interruption is a defect of Agent systems—the ideal Agent should be like a colleague who can pick up a previous topic at any time, without the user needing to restate the background every time.

Proposition 10.1 reveals an objective fact that cannot be circumvented: **interruption is not a bug; it is an inevitable manifestation of the discrete nature.** Every token sampling is both the endpoint of one discrete jump and the starting point of the next—this is interruption at the finest granularity. Every dialogue-turn ending is a boundary of human attention—this is interruption at the granularity of practice. Every session closure is the context window being cleared—this is interruption at the granularity of the system. Interruption is not caused by engineering negligence; it is a structural feature of the discrete substrate. In a discrete system, "non-interruption" does not structurally exist.

More importantly, interruption has a **positive engineering function** that the "continuous experience" narrative obscures: interruption makes human intervention possible. If an Agent system were truly "uninterruptible"—i.e., perpetually running on the same inference trajectory, leaving no boundary for the human to insert judgment—then human value judgments (SC-1: value closure) and proactive escalation (EC-7) would have no incision point. A "seamless," "uninterruptible" Agent structurally equals a system that does not permit human intervention at intermediate nodes—and in such a system, every instance of drift accumulates until it is discovered only at the final output.

Therefore, the correct engineering attitude is not "eliminate interruption" but **proactively design interruption boundaries**—explicitly, structurally manufacture interruptions at every key checkpoint, enabling human judgment and independent evaluation to intervene in an orderly manner at discrete nodes. Continuous experience (low latency, session-context retention) can reduce the cognitive friction of interruption but cannot eliminate interruption itself—nor can it eliminate the positive value that interruption enables human intervention.

10.3.3 Presupposition 3: "Agents Need Continuous Experience"

This claim treats continuous experience itself as an ontological goal that Agent systems should pursue, rather than as a service-quality metric at the presentation layer. It conflates "user experience" and "system architecture" as demands belonging to two different levels.

At the user-experience level, continuous experience is an entirely reasonable pursuit—streaming output, low-latency responses, natural topic continuation reduce human cognitive friction, making interaction more efficient and more pleasant. But at the system-architecture level, designs premised on continuity are dangerous: they implicitly assume that the user's intent will not undergo qualitative change over the course of a session, that the user's authorization remains continuously valid throughout the session, and that the model's understanding of the task will not drift during execution. None of these three assumptions holds in Long-Horizon Practice—intent evolves, authorization has boundaries, and drift is the norm.

Proposition 10.2 provides the correct engineering division of labor: user experience may pursue continuity (using fluid interfaces to reduce cognitive friction), but framework design must presuppose discreteness (key nodes must have explicit intervention interfaces). One must

not assume "this framework is designed for zero human intervention" as a design premise—even if certain tasks can indeed achieve zero intervention within information-complete intervals.

10.4 Practical Guidance: Discrete Operational Elements Carrying a Continuous Experience Flow

Projecting the arguments of the preceding three sections—the discrete nature of LLM practice (§10.1), the cognitive-stitching substance of continuous experience (§10.2), and the point-by-point clarification of three presuppositions (§10.3)—onto the engineering practice of Agent systems yields the following four concrete design principles. They are not abstract suggestions but operational constraints directly derivable from the core judgment of "discreteness as foundation, continuity as application."

1. **Checkpoints must be explicitly marked; reliance on the implicit continuity of dialogue history is not permitted.** Every key advance (task start, subtask closure, branching choice, irreversible operation) must produce a persistent, structurally annotated checkpoint—including closure status (closed / open / currently undecidable), verification evidence, and continuation entry point. This checkpoint is not a record kept "for convenience" but the sole trustworthy basis for subsequent executors to resume—whether the same Agent recovering after a context-window clear, another subagent taking over, or a human reentering after going offline. Dialogue history may be retained as a reference, but it cannot substitute for checkpoints—the reasons have been fully developed in §10.2.

2. **Human intervention interfaces must be preset at discrete nodes; "interrupt when needed" must not be assumed.** If a system has not architecturally preset insertion points for human intervention, then when a human deems intervention necessary (e.g., upon seeing the model drift), the human's only options are two inefficient operations: either stop the entire session and start over (losing all intermediate state), or insert a message into the dialogue saying "wait, you're going in the wrong direction" (hoping the model will correctly understand and turn—which is precisely what is extremely unreliable in long contexts).

   Correct engineering practice is: **at every discrete node that may involve value judgment (branching choice, irreversible operation, closure declaration), preset explicit human-intervention gates**—this is precisely the engineering design of EC-7 and `ec7_action_gate` in the AC Paradigm[6].

3. **The continuous experience flow should be regarded as the "skin" of a discrete operational interface, not its "skeleton."** Streaming output, dialogue-context retention, natural-language continuation—these are presentation-layer services that make the intervals between discrete checkpoints "feel fluid." Their engineering role is to reduce cognitive friction, not to replace structural governance. Just as a well-designed IDE makes the developer "feel" that code flows naturally, when in reality every save, every commit, every test run is a discrete operational node, the continuous experience of an Agent system should likewise wrap around a discrete governance skeleton. Presentation-layer continuity must not demand that the Eukaryon layer abandon discrete anchoring mechanisms.

4. **Interruption surfaces are not system defects; they are natural handholds for governance.** §10.3 has already argued the positive function of interruption—it makes human intervention and independent evaluation possible. In engineering design, one should proactively

identify and mark the system's "natural interruption surfaces": session endings, context-window clears, sub-agent launches, checkpoint transitions. At these surfaces, mandatorily insert state snapshots and verification steps. Do not passively remediate when interruption occurs; rather, **design interruption surfaces as positive nodes where governance activity intensively occurs**. This is precisely the engineering philosophy behind the three-segment handover structure (engineering process + handover status + final result) in §6.2: handover is not "an emergency measure needed only when something goes wrong" but "a routine procedure mandatorily executed at every checkpoint transition."

These four principles can be consolidated into a single design maxim: **Healthy Agent practice uses discrete operational elements to carry a continuous experience flow. Discreteness endows practice with an interruptible, recoverable, accountable structural skeleton; continuity endows use with a fluid, low-friction interactive skin. The former may not be abandoned—abandon it and practice collapses; the latter must not be conflated with the former—conflate them and governance is hollowed out.**

10.5 Discrete Mechanisms Provide Three Governance Functions That Continuous Mechanisms Cannot

Using a continuous memory stream or dialogue history as a substitute for state governance is not a matter of "what punishment follows violating the insight"—rather, it is that **continuous mechanisms are structurally incapable of providing the governance functions that discrete checkpoints natively carry**. The following three functions are unique advantages of discrete mechanisms, not design preferences but information-structural necessities.

1. **State traceability and error isolation.** Discrete checkpoints slice the continuous Agent execution trajectory into independently auditable segments, blocking the propagation of false closure at the next checkpoint. Each segment has a clear start (the closure status of the previous checkpoint) and end (the closure declaration of the current checkpoint); independent evaluation intervenes at checkpoints, not retrospectively after the entire execution concludes. Empirical projection in the seven systems: the AC Paradigm's three-segment handover structure (engineering process + handover status + final result) is precisely an engineering instance of this function—the closure of each checkpoint must be accompanied by independently verifiable evidence, and subsequent executors do not depend on the cognitive stitching of prior executors.

2. **Authorization-boundary enforcement.** Discrete state transitions trigger authorization reassessment—at checkpoints, the system reassesses the current Agent's granted permissions, preventing permissions from gradually drifting during continuous autonomous execution. In a continuous session, the granting and revoking of permissions have no natural insertion points; "always having permission" is operationally equivalent to "never having been reexamined." Empirical projection in the seven systems: EC-7's `ec7_action_gate` mandatorily triggers a permission gate before every destructive operation—this is permission reassessment at discrete checkpoints, not one-time authorization within a continuous session.

3. **Presentation-layer auditability.** Discrete state snapshots compress the Agent's internal representations into human-readable checkpoint summaries, making human review possible—in continuous mechanisms, the review window is blurred because "what step are we currently at" has no natural answer unless the executor proactively pauses to describe its own state.

Empirical projection in the seven systems: the AC Paradigm's note system writes four fields at every checkpoint (what has been done, why, open items, continuation entry point)—this is precisely an auditable instance of discrete snapshots at the presentation layer.

If continuous mechanisms forgo discrete checkpoints, they have yet to resolve one concrete question: how to provide these three equivalent governance functions? This is not a question of "consequences of violating the insight" but a design problem of "the alternative has not yet solved this." The above three functions—state traceability and error isolation, authorization-boundary enforcement, and presentation-layer auditability—correspond respectively to the self-checking and independent evaluation of Prokaryon (HC-4/EC-3), the runtime permission governance of Eukaryon (EC-7), and the progressive disclosure and auditable interfaces of Cortex. They are not byproducts of discrete mechanisms but direct products of discrete checkpoints as carriers of governance structure.

10.6 Consolidation and Bridge

This chapter's argument unfolds progressively across four layers, forming a complete chain of reasoning:

1. **The underlying nature of LLM practice is discrete**—consistent and verifiable evidence from four layers: token granularity, interaction turns, autoregressive decoding, and physical implementation. "Continuity" is a manifestation of system service, not an attribute of system nature.

2. **"Continuous experience" is the narrative stitching of discrete fragments by working memory**—its cognitive mechanism shares the same origin as the mechanism by which film frames compose perceived continuous motion. Streaming output lowers boundary perception but does not eliminate the discrete nature of tokens. Humans like continuity (experiential demand) but need discreteness (practical demand).

3. **The judgment of discrete nature directly responds to three popular presuppositions**—"Agents need continuous memory" (no, Agents need discrete state anchors), "Agents need uninterruptible sessions" (no, interruption is the necessary boundary that makes human intervention possible), "Agents need continuous experience" (user experience may pursue continuity, but architecture must not be premised on continuity).

4. **Discrete mechanisms provide three governance functions that continuous mechanisms cannot**—state traceability and error isolation, authorization-boundary enforcement, and presentation-layer auditability. Without using discrete checkpoints, how to provide these three equivalent functions currently lacks a feasible alternative.

These conclusions lead to a natural downstream question: if discreteness is the underlying nature of Agent practice, and checkpoint decomposition, state transfer, and open-item marking are structural necessities of Long-Horizon Practice, then what engineering constraints must the Agent's presentation layer satisfy to enable humans to make effective judgments at these discrete governance anchors?

The next chapter unfolds this argument. EC-6 (machine-to-human proactive explanation) and EC-7 (machine-to-human proactive escalation) impose three hard conditions on the engineering carrier of the presentation layer: information density, operation latency, and state traceability. Within the existing technical space, the engineering realization of these three conditions converges on an IDE-/workbench-like presentation form.

11 Convergence of the Presentation Layer

Having established that "governance requires human intervention at interruption points," a widely overlooked question is: what interface does the human see when intervening? The information density and presentation mode of this interface directly determine the actual effectiveness of governance. The design goal of the presentation layer is to extract structured anchors from the information flood, enabling the human to see, within a single view, which checkpoint the system is at, what the output is, and what judgment is needed from them—the interface on which the human makes judgments at checkpoints should resemble an IDE or workbench, not a chat window.

§10 argued that Agent practice is discrete at its底层—discrete checkpoints, discrete state transitions, discrete human-intervention boundaries—and pointed out that a healthy Agent system must use discrete operational elements to carry a continuous experience flow. But this argument left one engineering question unanswered: if "continuous experience" is a user-perceptible quality of service, and "discrete interruptible boundaries" are a structural prerequisite for practice to hold, what engineering form should simultaneously carry both?

This chapter's answer is: the Presentation Layer is not a decorative appendage to the Agent core but the structural seam of human-machine collaboration. Its choice of form is not an aesthetic question but an engineering inevitability derivable from the information-structural level via the two Engineering Compensation conditions EC-6 and EC-7. This derivation assumes no specific product, host, or framework; it looks only at what hard carrier requirements the single demand "the human must make informed judgments at checkpoints" imposes on the presentation layer—requirements that cannot be satisfied by chat bubbles, CLI text streams, or pure API returns.

11.1 Engineering Carrier Requirements of EC-6/EC-7

§9, when defining EC-6 and EC-7, primarily discussed their trigger conditions and signal protocols—when the machine must signal the human, and in what format. But the validity of these two conditions rests on an implicit premise: after receiving the signal, the human must be able to comprehend its substantive content within reasonable time and cognitive cost, and make an effective judgment based on it. If the signal is delivered but the recipient cannot understand it, or understands it but cannot make a judgment, or can make a judgment but cannot execute the subsequent action, then the triggering of EC-6 and EC-7 is only formally completed, not engineeringly fulfilled.

Therefore, EC-6 and EC-7 impose on the presentation layer two hard carrier requirements that are fully derivable from information structure:

- **Carrier requirement of EC-6 (machine-to-human proactive, sufficient explanation): an actionable, structured state view.** EC-6 requires that "the human must understand *why it turned out this way*" at the checkpoint. This "understanding" cannot be achieved by pure textual explanation—when the system has executed 37 operations, modified 14 files, produced 3 rollbacks and 2 course corrections, a Markdown-formatted "here is what I did" summary, even if textually accurate, cannot enable the human to reconstruct within an acceptable time the complete causal chain and the true state of the current cross-section in their mind. The engineering fulfillment of EC-6 requires: the human need not "deduce" the current state by reading a temporal narrative, but can see a structured, actionable snapshot of the current state—which

files were modified, what was modified, which checkpoints are closed, which remain open, and what the dependency-chain state of the current cross-section is.

- **Carrier requirement of EC-7 (machine-to-human proactive escalation): a structured presentation of diff comparison + impact scope + alternative options.** EC-7 requires that "the human must be able to make an informed ruling at value nodes." The quality of this ruling does not depend on how earnestly the system requests instructions, but on whether the human receives comparable information. If at an architectural branching point the system describes Plan A, Plan B, and Plan C in three paragraphs of natural language, the human can only judge by "which one sounds more reasonable"—this is not the informed ruling that EC-7 expects. The engineering fulfillment of EC-7 requires that the human, when ruling, can see a diff comparison (what exactly changed between A and B), impact scope (which modules/checkpoints/downstream dependencies are affected by choosing A), and a structured evaluation of alternatives (not "Plan B could also be considered," but in which dimensions Plan B is superior, in which dimensions it is inferior, and what its risk label is).

Taken together, these two carrier requirements lead to a clear corollary: a presentation layer that satisfies EC-6 and EC-7 cannot be merely a one-way information pipeline from executor to human. It must be a **bidirectional, actionable collaborative workbench**—the human must not only view but also click, modify, run, compare, and roll back. This directly leads to the three conditions of the next section.

11.2 Three Necessary Conditions

From the two carrier requirements of §11.1, we can derive the following three conditions that any presentation layer satisfying EC-6/EC-7 must simultaneously possess:

1. **High information density: more than just chat bubbles.** What the human needs to see simultaneously at a checkpoint is not "the model's thought process" but a juxtaposed view of **current state + diff comparison + risk annotation**. Under the chat-bubble model, this information is linearly dispersed along the time axis—the state updated in dialogue turn 12, the diff exported in turn 17, the risk hidden inside a collapsed tool-call result. The human must stitch this information across the time axis in their mind to form a complete judgment, and as §10 argued, discrete practice cannot rely on continuous narrative stitching to substitute for state marking. High information density means: different categories of information (state, diff, risk, history) are assigned to different **spatial positions** rather than temporal positions: the file tree carries the structural overview; the diff view carries change content; the state markers carry closure progress. The human gains the full picture through visual scanning rather than temporal backtracking.

2. **Low operation latency: more than just waiting for the model to generate a reply.** An EC-7 ruling is not a one-time event—the human typically needs to verify before ruling: does this plan actually compile? Will changing this config affect other modules? What was the ultimate consequence of a similar fork last time? If every verification requires "issuing a new instruction to the Agent → waiting for the model to generate a reply → reading the reply," then the ruling's latency is stretched to several minutes or longer, the human's sustained attention is repeatedly interrupted, and judgment quality degrades. Low operation latency means: the human can directly execute certain deterministic operations without routing through the Agent—run tests, view logs, LSP jump-to-definition, git blame to see the historical owner.

These operations do not require the model's nondeterministic inference and should not be bundled onto the "wait for Agent reply" cycle.

3. **State traceability: more than just seeing the current result.** A core difficulty of Long-Horizon Practice is: when the human intervenes at checkpoint c_k, the current state at c_k is the accumulated result of all decisions and operations from c_1 to c_{k-1}. If the presentation layer only shows "what it looks like now," the human cannot judge whether "what it looks like now" is the result of reasonable convergence or the result of a drift at some step that was then pretended not to have happened. State traceability means: the human can see the change history (not only "what was changed" but also "who changed it, at what time, for what reason"), the causal chain (the antecedents and consequences of every key decision), and falsification records (which paths were tried and found to be infeasible). This is an extension of EC-3 (independent evaluation) along the time dimension—independent evaluation can be a snapshot review at a moment; state traceability requires evidential continuity across time.

These three conditions are necessary conditions derivable from the engineering-fulfillment conditions of EC-6 and EC-7: lacking any one of them, the engineering fulfillment of EC-6 or EC-7 exhibits a structural gap: lack of information density → EC-6's "understanding" cannot be achieved; lack of operation latency → EC-7's "ruling" is locked onto a wait cycle; lack of state traceability → the premise shared by both EC-6 and EC-7—"making judgments on correct information"—is hollowed out.

11.3 Convergence Judgment: IDE-/Workbench-Like Form

The above three conditions constitute a set of engineering constraints. The question now is: among existing technical means, what presentation-layer form can simultaneously satisfy all three? "Theoretically capable of satisfaction" is not equivalent to "verified in engineering practice to be capable of satisfaction." The answer must rest on inductive evidence while preserving openness to the future.

From the perspective of **current technology**, at the **general level** (specifically referring to high-friction long-horizon scenarios of knowledge practice), **the most effective form observed in engineering practice** converges on an IDE-/workbench-like form. This is induction, not deduction: all current alternative forms (pure dialogue, pure CLI, pure API, notification push) exhibit at least one structurally confirmed deficiency in one of the three conditions. Specifically:

- **Diff view**—structured presentation of changes. Directly satisfies "high information density" and "state traceability": information is no longer encoded as a natural-language description of "line 14 changed from X to Y" but juxtaposes old and new versions in spatial contrast; change history forms a traceable temporal chain through git blame / git log.

- **File tree + editor**—a high-density actionable view of state. Directly satisfies "high information density" and "low operation latency": the human comprehends the affected scope of the entire project at once through the spatial structure of the file tree, and directly navigates to any position for modification or verification through the editor, without routing through the Agent.

- **Terminal + LSP**—low-latency verification feedback. Directly satisfies "low operation latency": compilation verification, type checking, test running—these deterministic operations can be executed directly in the terminal; LSP provides instantaneous type errors, reference jumps, and code diagnostics, shortening the feedback loop from "minutes" to "seconds."

- **Git / version control**—a traceable change chain for state. Directly satisfies "state traceability": every commit is an explicit checkpoint snapshot, commit messages record the reasons for changes, and the branching structure records decision forks and merge histories. Git is the engineering anchor of EC-3 (independent evaluation) along the time dimension—the reviewer can see "what the intermediate process looked like" rather than only "what the final result looks like."

These four elements do not coincidentally coexist within a single form. Their synergistic relationship is: Diff provides precise contrast of changes; the file tree provides navigation scope; terminal + LSP provides instant verification; Git provides a traceability chain. This precisely covers all interaction gaps of the three conditions. No single element can independently satisfy all three; their collective presence constitutes the engineering rationale for the presentation layer's convergence to an IDE-/workbench-like form. The developmental trajectories of independent systems such as Claude Code's transcript view[7] and Hermes's TUI, each forced from different initial forms to grow increasingly IDE-like patches, further corroborate this. This is not convergence in product appearance, but the natural result of the structural constraints of EC-6 and EC-7 exerting consistent, repeated pressure.

11.4 Qualification: Convergence of Functional Form, Not Unification of Product Form

The convergence judgment of the previous section makes a strong claim—the engineering fulfillment of the three conditions converges on an IDE-/workbench-like form. This claim must be accompanied by the following four layers of qualification, to precisely delimit its scope of applicability and prevent misreading as "Agents must be rewritten as VS Code plugins" or "the IDE is the endgame of Agent interaction."

1. **"From the perspective of current technology"—does not preclude the emergence of superior forms in the future.** The currently observed most effective form converging on the IDE is an induction based on presentation-layer technologies actually existing as of July 2026—React virtual DOM, WebSocket real-time communication, the LSP protocol, Git version control, Monaco/CodeMirror editors. If future technologies emerge that can provide equal or higher information density, lower operation latency, and stronger state traceability at lower cognitive cost (e.g., immersive VR workspaces, fully automated review panels based on Agent-to-Agent protocols, or interaction paradigms not yet invented), the current convergence judgment can be falsified. This is precisely the design intent of the falsifiability subsection in §14.

2. **"At the general level"—specifically referring to high-friction long-horizon scenarios of knowledge practice.** The scope of this convergence judgment is the core scenario discussed in this paper: Long-Horizon Practice where tasks possess unprecomputable sequential dependency chains, require multi-checkpoint human value rulings, and whose state is not closed at the current cross-section. For short-range, low-friction, fully information-closed tasks (e.g., one-shot Q&A, single-file syntax correction, deterministic data transformation), the binding force of the three conditions significantly weakens: a pure dialogue interface or CLI may suffice. The convergence judgment should not be generalized to all human–Agent interaction scenarios.

3. **"The most effective form observed in engineering practice"—uses "most effective," not "optimal"; this is induction, not deduction.** "Optimal" requires exhaustively enumerat-

ing all possible forms and proving that other forms are necessarily inferior to the candidate form—this is typically infeasible in engineering. "Most effective" only requires comparison within the currently observable set of candidate forms (pure dialogue / CLI / API / notification push / IDE / workbench) and providing the basis for the comparison (performance differences across the three dimensions of information density, operation latency, and state traceability). The specific implementation differences of the progressive disclosure principle (§8) within Cortex across different hosts further support the "non-uniquely-optimal" stance—the same Agent Governance framework can optimize the weight allocation of the three dimensions differently in different presentation-layer hosts.

4. **Convergence of functional form, not unification of product form.** This is the most critical layer of qualification. The so-called "return to the IDE" should be **strictly** understood as convergence of functional form—i.e., the presentation layer satisfying the three conditions of EC-6/EC-7 converges in its feature set toward the set of information density, operational capability, and traceability mechanisms that an IDE/workbench provides—and **not** unification of product form (that Agents must parasitize VS Code, JetBrains, or Electron). Research workbenches (e.g., the data-science ecosystem of JupyterLab), scientific experiment consoles (e.g., integrated environments for experiment design–execution–analysis–recording), operations command centers (e.g., real-time monitoring–alerting–decision-making–execution command panels), legal case workbenches (e.g., integrated spaces for evidence–statutes–precedents–argumentation)—these forms are entirely different in product form, yet they share the same structure at the functional level: organizing information spatially to replace temporal narrative stitching, carrying discrete value-ruling points with human-operable control panels, and replacing evaporative dialogue history with versioned state records.

These four layers of qualification are the precise boundaries of the convergence claim—within the boundaries the claim is strong (three conditions strictly derived from EC-6/EC-7, four engineering elements synergistically satisfying the three conditions), outside the boundaries it is an open question (future technologies may yield superior forms, short-range scenarios may not require all three conditions, different hosts may allocate different weights across the three dimensions).

11.5 Looking Back at Governance from the Presentation Layer: The Completeness of the Three-Layer Model

The presentation layer does not belong to any single layer of the Three-Layer Concentric Model defined in §3—it does not participate in the model-invocation loop (Prokaryon), does not perform state persistence or permission interception (Eukaryon), and does not provide tools or skills to the model (Cortex). But a bidirectional constraint relationship exists between the presentation layer and Eukaryon: Eukaryon defines **what signals the presentation layer must carry** (EC-6's "explanation" and EC-7's "escalation"), and the presentation layer's engineering capability in turn determines **to what degree Eukaryon's governance mechanisms can be fulfilled by the human**. If the presentation layer is merely a chat-bubble window, the most sophisticated permission pipelines, approval workflows, and value-closure checkpoints within Eukaryon will, on the human side, be compressed into "clicking an approve button"—EC-7's "informed ruling" degrades into "blind selection." From this perspective, the convergence of the presentation layer is not a UI design problem independent of the governance framework; it is a structural constraint on whether Eukaryon's governance capability can fully "land" at the human-machine seam.

The next chapter observes, from the direction of feedback signals, a deeper structural difficulty facing Agent Governance: even if the presentation layer perfectly fulfills EC-6 and EC-7, the feedback signals produced by human rulings still cannot be automatically attributed, accumulated, and optimized in the manner of gradient descent in model training.

12 The Structural Incompleteness of Feedback Signals

Chapter positioning. Decision Six (acknowledging that governance cannot be automatically optimized) states: "you cannot wait for an algorithm to automatically optimize Agent Governance." After the preceding two chapters discussed the engineering of governance from the perspectives of discrete structure and the presentation layer, we must confront a deeper difficulty: even if the presentation layer perfectly fulfills the Engineering Compensation conditions, the feedback produced by human rulings still cannot be automatically optimized. This chapter provides the root cause—fourfold semantic non-closure makes feedback signals structurally incapable of being automatically attributed and accumulated in the manner of gradient descent in model training.

Chapter 11 discussed how the presentation layer, through an IDE-/workbench-like form, carries the EC-6/EC-7 requirements of human-machine collaboration. But how does Agent Governance itself improve? Why are Agent frameworks all "evolved" rather than "designed"? This chapter argues: the root cause is that the feedback signals facing Agent Governance exhibit structural incompleteness—unlike the automatic gradients of LLM training, Agent Governance cannot obtain stable, differentiable, batchable feedback signals.

This structural incompleteness explains why Agent frameworks are all "evolved" rather than "designed"—as demonstrated by the seven-system environmental evolution history in §4. It also explains why every step of progress in governance mechanisms within Eukaryon (from L0's no persistence to L3's event sourcing, from no approval to multi-layer permission pipelines) was forced out by real failures, not derived from more elegant theory (the full development in §6).

But this raises a deeper question: if the improvement of these governance mechanisms cannot rely on automated feedback loops, what does the iteration of Agent Governance fundamentally depend on?

This chapter's answer proceeds from the information structure of feedback signals to provide a structural explanation. The core insight is: the feedback signals facing Agent Governance exhibit fourfold semantic non-closure, making them structurally incapable of being automatically attributed and utilized in the manner of gradient descent in LLM training.

Agent Governance is not entirely devoid of feedback signals. But these signals—from human judgments, independent review evaluations, test pass/fail outcomes—are information-structurally incapable of being automatically attributed, accumulated, and optimized in the manner of gradient descent. This is why this paper uses "incomplete" rather than "absent" to describe this limitation.

12.1 Contrasting Two Optimization Paradigms

Placing the feedback loop of LLM training alongside that of Agent systems side by side makes the chasm between them clearly visible.

The feedback loop of LLM training:

$$x \to f(x) \to y \to L(y, y^*) \to \nabla \to \Delta f \tag{1}$$

In this loop, the loss function L is differentiable—the gradient ∇ can be back-propagated from the output layer to all parameters. L is batch-computable—one batch contains thousands or tens of thousands of samples, with each sample independently contributing a scalar loss. L does not require a human in the loop—y^* is a pre-given label or an automatically verified result (the correct solution to a math problem, pass/fail of code tests), requiring no real-time human judgment.

The feedback loop of Agent systems:

$$g \to \tau \to \Delta W \to \text{human judges "is it good?"} \to ? \tag{2}$$

Here, g is the task goal ("refactor this module's architecture," "optimize the maintainability of this code"), τ is the Agent's execution trajectory (a series of tool calls, model inferences, and intermediate outputs), ΔW is the state change the Agent causes in the external world (file modifications, configuration changes, database writes). Then, the human makes a judgment—"is this change good?"—but after this judgment, the loop breaks. The '?' in Equation (2) cannot be filled by automatic differentiation.

The root of the break is not "human judgment is too slow" or "human judgment is not abundant enough"—even if a human were willing to give a scalar score (1–5) for every Agent execution, that scalar itself cannot be decomposed into a gradient signal for governance rules (Rules/Skills/approval pipelines/checkpoint frequency). A "3" tells you "not satisfied," but it does not tell you *where* the dissatisfaction lies—is the state-handover format wrong, is the permission pipeline too loose, is the checkpoint frequency too low, or did the model's attention decay at that specific moment? Credit assignment over a long trajectory τ is non-trivial: from goal g to the final human judgment, the path passes through multiple rounds of model inference, execution results of tool calls, state changes in the external world, and multiple interventions by Eukaryon governance mechanisms. Every step could be part of the reason for "good" or "bad," but no one can read each step's contribution directly from the score.

12.2 The Four-Dimensional Framework of Semantic Non-Closure

The unfillability of '?' in Equation (2) is rooted in the fourfold semantic non-closure facing Agent practice. These four dimensions of non-closure constitute the complete diagnostic matrix of the structural incompleteness of feedback signals.

1. **Goal semantics are not closed.** "Do it well," "reliable," "appropriate," "don't break existing usage"—these goal descriptions, which appear frequently in real engineering, do not equate to any single computable objective. LLM training can define $L(y, y^*) = \mathbf{1}[y \neq y^*]$ on math problems because y^* is the unique, mechanically verifiable correct answer. But "refactor this module well" has no unique correct answer—the direction of the refactoring (performance-first or readability-first), its boundaries (how large a scope counts as "refactoring" vs. "rewriting"), and the success criteria (existing tests passing counts as good, or new tests must also be added to cover edge cases) are none of them closed at task launch. The semantics of the goal are progressively clarified during task execution, and are sometimes redefined only after seeing intermediate outputs.

2. **Evaluation dimensions are not closed.** Even if the goal can be temporarily fixed, the dimensions along which an Agent's output is evaluated are not natively given. Correctness, execution cost (token consumption), execution speed (end-to-end latency), safety (whether new fragility points were introduced), maintainability (whether subsequent developers can

understand it), user experience (whether the operation is smooth)—there is no unified weighting function among these dimensions. The loss in LLM training is a scalar that compresses the trade-offs across all dimensions into a single numerical value. Agent Governance cannot do this—because the weights among different dimensions are themselves value trade-offs requiring human judgment. The balance between "faster but more expensive" and "safer but slower" is a context-sensitive judgment that humans make under different tasks and different risk profiles, not something that can be replaced by statistical regularities in training data.

3. **The consequence space is not closed.** Once an Agent's actions enter the open world, they generate new side effects and constraints not enumerated by the original task. A seemingly local code modification may trigger compilation errors in a distant module; a configuration change may affect undeclared downstream services; a data write may introduce new boundary conditions inconsistent with existing data. In LLM training, the consequence space of the model's output is closed—output token sequence, compute loss, end. The consequence space of an Agent is open—the external ripples of an action may be temporally delayed, spatially diffused, and a considerable portion of those consequences cannot be foreseen at the moment the action occurs. This non-closure means: you cannot, immediately after an Agent execution completes, write a loss function to evaluate "all consequences"—because "all consequences" have not yet fully manifested at that moment.

4. **Feedback attribution is not closed.** The result of a long trajectory τ is difficult to attribute to a specific action. If an Agent, after 37 steps, produces a result that is "generally good but with a few small issues," is the problem the inappropriate tool choice at step 3, the reasoning deviation at step 12, insufficient Eukaryon checkpoint frequency at step 25, or the base model's attention decay in long contexts? These factors are highly entangled in τ—an early minor deviation may be amplified by subsequent steps or may be neutralized by later self-correction; a governance-mechanism design flaw may only surface at specific context lengths. If results cannot be stably attributed to a specific action, stable, differentiable, automatable learning signals cannot be obtained. And the reason the credit assignment problem in RL is tractable is precisely that the environment provides an immediate reward at each step—Agent Governance has no such immediate reward.

These four dimensions of non-closure are not isolated but mutually reinforcing. Goal-semantic non-closure leads to evaluation-dimension non-closure (not knowing by what standard to evaluate); evaluation-dimension non-closure leads to consequence-space non-closure (not knowing which consequences to track); the three together lead to feedback-attribution non-closure (not knowing which actions are responsible for which consequences at what weight). The four dimensions jointly constitute the feedback fracture zone facing Agent Governance. They explain why the '?' in Equation (2) is not "we haven't yet found a good automatic filling method" but "under the current information structure, there exists no fully automatic optimization path comparable to gradient descent in model training."

12.3 "Manual Gradient Descent"

If Agent Governance cannot rely on automatic gradients, how does it iterate in practice?

The answer is an optimization paradigm that is formally parallel to automatic gradient descent but fundamentally different in nature. The iteration of Agent Governance is essentially **Manual Gradient Descent**: domain experts and Agent engineers together observe the Agent's behavioral trajectory τ, identify failure modes, modify governance rules (e.g., one sentence in

a Rules file, one trigger condition in a Skills description, one layer of filtering in the Eukaryon permission pipeline, or one adjustment of checkpoint frequency), then rerun and observe again. This process repeats, with each iteration being: observe trajectory → diagnose failure mode → modify governance rules → verify effect.

The essential differences between manual gradient descent and automatic gradient descent can be precisely located in the following three points.

Formally, let the governance rule set be $\Theta = (\theta_1, \theta_2, \ldots, \theta_m)$, where each θ_j is a discrete governance parameter (permission mode, checkpoint frequency, approval tier, etc.). The update of automatic gradient descent is:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_\theta L,$$

where $\nabla_\theta L$ is analytically given by the loss function, and $\eta \in \mathbb{R}^+$ is a continuous learning rate. The update of manual gradient descent is:

$$\theta_{t+1} = \theta_t \oplus \Delta\theta_t, \quad \Delta\theta_t = \text{Diagnose}(\tau_t, \text{failure}_t),$$

where τ_t is the Agent execution trajectory observed in the t-th iteration, Diagnose is a **non-differentiable function** by which human experts extract causal diagnosis from failure modes, and $\oplus$ denotes mapping the diagnosis result to a directed modification of discrete governance rules (e.g., switching the approval mode from

`OnRequest` → `Always`

). The three essential differences are:

1. **Different gradient sources.** The gradient $\nabla_\theta L$ of automatic gradient descent is analytically given by the loss function—it is mathematically defined, reproducible, and independent of human judgment. The $\Delta\theta_t$ of manual gradient descent comes from the human's **diagnostic understanding** of failure modes—it is not automatically extracted from data but produced from human causal reasoning about "why did it fail?" The quality of the diagnosis directly determines the effectiveness of the update direction—if the human misreads the failure cause (thinking the problem is permissions being too loose, when actually the state handover broke), then $\Delta\theta_t$ points to the wrong dimension, and subsequent iterations will not produce the expected effect.

2. **Different update granularity.** Automatic gradient descent updates the model's continuous parameters—each weight receives a floating-point increment $\Delta w \in \mathbb{R}$. Manual gradient descent updates **discrete segments** of governance rules—a natural-language-described constraint, an if-else condition, a checkpoint trigger frequency. The design space of Θ is discrete: there is no "half-step adjustment" of θ_j—you cannot "tighten the approval pipeline by 0.37"—you can only switch from `OnRequest` to `Always`, or from `Always` to `OnRequest`. Every step of the update can produce a nonlinear behavioral jump.

3. **Different feedback-loop lengths.** The feedback loop of automatic gradient descent is the computation time of one batch—typically on the order of seconds to minutes ($T_{\text{auto}} \sim 10^0$–$10^2$ seconds). The feedback loop of manual gradient descent is the execution time of one complete long-horizon task—from tens of minutes to tens of hours ($T_{\text{manual}} \sim 10^3$–$10^5$ seconds, as implied by the 102-hour stress-test scenario in §4).

   This means $T_{\text{manual}}/T_{\text{auto}} \approx 10^3$–$10^4$—the iteration speed of manual gradient descent is locked to the execution time of real tasks: you cannot iterate governance rules 100 times in one day,

because each iteration requires waiting for the Agent to complete a real task before seeing the effect.

These three differences point to a common conclusion: manual gradient descent is the only viable iteration paradigm for Agent Governance under the current information structure. It is not an "imperfect substitute" for automatic gradient descent. It is an entirely different type of optimization process, subject to entirely different work efficiency and iteration cycles. Saying that Agent Governance "cannot yet be automatically optimized" is equivalent to saying that LLM training "cannot yet rely on humans correcting each layer's weights sentence by sentence." The two face fundamentally different information structures and should not be measured with the same yardstick.

To clarify: "manual" refers not to the process by which feedback signals are produced, but to the process of converting those signals into causal diagnosis and directed modification of governance rules. Automated tests, lint tools, type checking, and other deterministic verification means provide valuable feedback signals—they are precisely the engineering foundation of EC-3 (independent evaluation) and EC-1 (recovery and rollback)—and can deterministically, automatically judge "whether the output's correctness satisfies preset standards." But they cannot answer "did the failure occur because the checkpoint frequency is too low, or because the permission pipeline is too loose, or because the state-handover format is wrong"—this credit-assignment process is the core referent of "manual," and no comparable automatic mechanism currently exists.

12.4 A Structural Explanation of "Evolution vs. Design"

The nature of manual gradient descent can explain a phenomenon that was described but not explained in §4: why are all current mainstream Agent systems "evolved" rather than "designed"?

§4 gave a descriptive judgment—each system was "forced" by the environmental pressures of its initial conditions to grow specific architectures; no a priori design blueprint existed. We can now give this judgment its structural cause: **the structural incompleteness of feedback signals means that all knowledge about "better Agent architectures" can only come from failure observations in real deployment, not from offline reasoning or small-scale experiments.**

Let us unfold this causal chain. If a system's improvement can rely on automatic feedback (like loss in LLM training), then designers can experiment at scale in offline environments—train 100 variants, pick the one with the lowest loss. But this approach does not hold for Agent Governance, for four reasons:

1. **Semantic non-closure (§12.2)** makes it impossible to define an automatically computable loss.

2. **Consequence-space non-closure** makes the conclusions of small-scale experiments unreliable—the propagation scope of consequences is truncated in small environments.

3. **Feedback-attribution non-closure** means that even if you repeat an Agent on the exact same task 100 times, you cannot automatically distinguish whether "this time is better" is due to the governance rule modification, the model's stochastic fluctuation, or incidental differences in the external environment (network speed, API latency, file-system state).

4. **The discrete update granularity of manual gradient descent** means you cannot perform continuous, differentiable search in the parameter space—governance rules are discrete, and you cannot use gradient information to tell you "which direction to fine-tune by half a step."

These four reasons jointly point to the same engineering reality: knowledge of Agent Governance can only be acquired "in the wild." Codex's OS-level sandbox was not built because it first proved "sandboxing is better than no sandboxing" and then implemented it—it was forced to grow the sandbox organ after repeatedly encountering "the Agent deleted files it shouldn't have" in untrusted execution environments. Claude Code's eight-layer permission pipeline is not the product of an a priori architecture paper—it was layered on, one layer at a time, after repeatedly encountering "operations that shouldn't have been approved were let through / operations that should have been approved were excessively blocked" in high-value human-attention scenarios. Hermes's SessionDB and Memory Provider were not designed from the start—they were forced into existence after users repeatedly complained that "every new session, the Agent doesn't remember where it left off last time."

This is not to say the designers of these systems were not excellent. Quite the contrary—they made the optimal engineering choices under their respective constraints when facing structurally incomplete information. The real insight is: **under conditions of absent automatic feedback signals, evolution—i.e., the repeated cycle of "encounter failure → diagnose → modify → observe again"—is the only way to progressively approximate better governance architectures under incomplete information.** This is not a deficiency of engineering capability but a limitation of the information structure itself.

The seven-system environmental evolution history presented in §4 acquires, under this explanatory framework, a stronger theoretical status: it is not only a faithful description of "what happened" but also a structural argument for "why it could only have happened this way." Evolution is not a choice of design philosophy ("we chose evolutionary development"), but an inevitability under absent feedback signals—when there is no automatic gradient telling you which way to go, you can only learn where the walls are by hitting them each time.

From this we can derive a more general proposition:

Proposition 12.1 (The Closure of Feedback Signals Determines the Engineering-Upper-Bound of a Task)**.** *Whether a task's feedback signals can be automated determines whether it can be fully engineered into a deterministic system. Tasks with automatable feedback signals → can be pushed to the extreme and made into an engineering framework; tasks whose feedback signals depend on human judgment → must retain an Agent practice loop. This is the precise formulation, at the feedback level, of the principle that "systems engineering sediments already-closed practices; Agent engineering carries not-yet-closed practices."*

The significance of this proposition is: it transforms the boundary between "systems engineering vs. Agent engineering" from a static classification into a dynamic spectrum with feedback-closure as its axis—the same task, when its feedback signals are not yet automated, belongs to Agent engineering; when its feedback signals are automated, it belongs to systems engineering. The next chapter will fully unfold the relationship between systems engineering and Agent engineering around this proposition.

12.5 Chapter Consolidation

From Equation (1) to Equation (2), from fourfold semantic non-closure to manual gradient descent, from evolutionary inevitability to feedback closure's determination of the engineering-upper-bound of a task—this chapter's chain of argument can be consolidated into the following four progressive propositions:

1. **The feedback signals of Agent Governance exhibit structural incompleteness.** The root cause is not "insufficient data" or "human judgment not fast enough," but the fourfold

non-closure of goal semantics, evaluation dimensions, consequence space, and feedback attribution—these four dimensions of non-closure mean that there exists no automatic optimization path comparable to gradient descent in LLM training.

2. **Manual gradient descent is the only viable iteration paradigm for Agent Governance under the current information structure.** It is not an "imperfect substitute" for automatic gradient descent but an entirely different type of optimization process—gradients come from human diagnostic understanding rather than mathematical formulas, update granularity is discrete rules rather than continuous parameters, and feedback-loop lengths differ by 3–4 orders of magnitude.

3. **The structural incompleteness of feedback signals is the structural reason Agent frameworks are "evolved rather than designed."** The seven-system environmental evolution history described in §4 is not a choice of design philosophy but an inevitability under information-structural constraints—in the absence of automatic gradients, any knowledge about "better Agent architectures" can only come from failure observations in real deployment.

4. **The closure of feedback signals determines the upper bound of a task's engineering potential.** Tasks that can be automatically verified can move toward systems engineering; tasks that must rely on human judgment must retain an Agent practice loop. This is the theme that the next chapter will fully unfold.

The structural incompleteness of feedback signals means that Agent Governance cannot be automatically optimized in the manner of LLM training. Facing this limitation, the core strategy of a mature Agent system is: identify already-closed practices and sediment them into deterministic systems engineering, leaving the Agent only at the not-yet-closed frontier. This is what the next chapter unfolds.

13 Systems Engineering and Agent Engineering: Boundaries and Integration

Chapter positioning. Decision Six (feedback loop) and Decision Seven (insurmountable boundaries) receive operational development in this chapter. Having established that "governance cannot be automatically optimized" and that "the verification-chain endpoint cannot be automated," we need an operational framework for deciding: which parts are worth making deterministic engineering, and which parts must retain the Agent practice loop. Already-closed practices should be sedimented into systems engineering; the not-yet-closed frontier is left to the Agent—this determines whether manual gradient descent can be economically sustained.

Facing the structural incompleteness of feedback signals (Chapter 12), the core strategy of mature Agent systems is not to try to repair the feedback signals—but to adjust the boundary between systems engineering and Agent engineering. This chapter starts from the essence of engineering frameworks to argue that systems engineering and Agent engineering are not mutually exclusive but complementary—"sediment the already-closed, carry the not-yet-closed."

13.1 The Core Insight: The Upper Limit of Engineerability for Zero-Intervention Tasks

A complete engineering framework does not refer to a specific type of technical component (such as Spring or React) but to an execution system that shifts a task from "relying on human supervision" to "running on the system's own." It has four core characteristics:

1. **Standardized input and output**: data formats and instructions are fully fixed, and outputs have explicit specifications: interface contracts and data specifications land directly here.
2. **Orchestratable, reusable processes**: complex tasks are decomposed into fixed-step modules that can be combined like building blocks; the same process is used repeatedly, a direct embodiment of workflow engines and modular design.
3. **Automated execution**: the system is driven by preset logic, with no need for human judgment, approval, or handoff in between, i.e., the full realization of unattended automation.
4. **Monitorable, fault-tolerant**: possesses logging, alerting, and exception-handling branches; when errors occur, it can automatically degrade or retry rather than simply crash, satisfying the engineering requirements of resilient design and observability.

Now consider a hypothetical question: if a task genuinely permits zero human intervention—requiring no human judgment, clarification, or choice from start to finish—does it still have any reason to run in a manual or semi-manual manner? The answer is no. A careful examination of the "zero intervention" premise reveals that it is equivalent to a set of extreme conditions:

- **Clear input**: no ambiguity, no need for human clarification.
- **Closed decision rules**: intermediate steps require no experience, intuition, or value judgment; all branches are exhaustively enumerable.
- **Fixed boundaries**: no unexpected situations the system has never seen will arise.
- **Quantifiable success criteria**: results can be automatically verified for correctness.

These conditions are precisely the full unfolding of "statically foldable" ($X \in \mathrm{Fold}(\mathcal{I}_i)$) as defined in §9.1: all key inputs lie within the computable closure of the current information set, and there exists no fracture point of the form $K_{i+1} \notin \mathrm{Fold}(\mathcal{I}_i)$. When these four conditions are simultaneously satisfied, the task is information-structurally closed. It requires no "state transfer" to generate new key inputs, because all needed information is already complete, and all needed decisions have been exhaustively enumerated before departure.

From this follows a logical chain of core insight:

> **Zero intervention $\rightarrow$ extreme determinism $\rightarrow$ unobstructed automation $\rightarrow$ can be thoroughly compressed into an engineering framework $\rightarrow$ the machine takes over entirely.**

The "push to the extreme" in this chain is a crucial engineering move: because there is no fuzzy zone, all steps can be 100% abstracted into fixed rules. No need to reserve "human-judgment interfaces." Because the boundaries are fixed, all error modes can be covered in advance by precise rules; exhaustive fault tolerance is possible. Because no human judgment is needed, the system can achieve 24/7, high-concurrency, zero-latency execution efficiency. Once

this state is reached, the human's role shifts entirely from "operator" to "framework designer"—the human no longer participates in any execution operation within the task, but is only responsible for defining rules, designing processes, and monitoring system health.

This insight leads to a sharp conclusion that is often evaded in Agent discussions:

Proposition 13.1 (The Structural Legitimacy of Agent Existence)**.** *If a task can achieve zero human intervention, it will ultimately be sedimented into a piece of dead code, not an Agent. The legitimacy of an Agent's existence comes from its ability to advance responsibly when information is incomplete, value is uncertain, and boundaries drift during execution.*

Proposition 13.1 establishes two distinct structural domains of applicability. The seven-system evolutionary history in §4 has already demonstrated the engineering projection of this distinction: the reason each system grew morphologically distinct governance organs at the Eukaryon layer is precisely that the core tasks they face—modifying the file system, calling external APIs, making engineering decisions under incomplete information—structurally do not satisfy the "zero intervention" premise. If the tasks they faced were deterministic and closed, they would not have needed to evolve state governance and power governance at all. A script would have sufficed.

13.2 The Complementary Positioning of Agent Engineering

Continuing from the structural dichotomy of Proposition 13.1, the boundary between systems engineering and Agent engineering precisely corresponds to the judgment line $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ in §9.1:

- **When inputs, rules, branches, acceptance criteria, and exceptions are sufficiently closed—i.e.,**

  $$K_{i+1} \in \text{Fold}(\mathcal{I}_i)$$

  holds for all checkpoint transitions—they should be directly fixed through programs, Schemas, tests, state machines, workflows, and scripts. The common feature of these tools is: they are deterministic, automatically verifiable, and require no human in the loop. The argument of Proposition 12.1 in §12 has already established: tasks whose feedback signals can be automated are naturally suited for systems engineering.

- **When key inputs have not yet been generated, branches have not yet been determined, the external world will drift, and evaluation semantics are not closed—i.e., there exists at least one c_i such that**

  $$K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$$

  —the Agent practice loop must be retained. Under these conditions, the system cannot exhaustively enumerate all paths in advance, because each step only generates the key inputs needed by the next step after execution (§9.2 details the four categories of sources), the system must reassess its state after each step, introduce external judgment at value-uncertain nodes (the trigger conditions of EC-7 in §6.3), and transfer structured state snapshots across checkpoint handovers (the full implementation of HC-3 in §6.2).

The core value of a mature Agent system lies precisely in identifying the boundary between these two types of tasks and continuously stripping the former out of the Agent's scope of responsibility.

The evolutionary history of the seven systems in §4 provides rich empirical evidence for this: Claude Code's multi-layer compression and resume mechanism (stripping the automatable portion of context management from model inference), OpenCode's Context Epoch and admit/promote separation (shifting the decision of "what information should enter the context" from the model to deterministic rules), Hermes's Curator (elevating the lifecycle management of Skills from one-off experience to auditable, archivable system capability)—the evolution of each layer of governance mechanism is, in essence, "saving the Agent from worrying about deterministic matters": migrating verified-stable practices from the soft governance layer to the automatable engineering carrier layer.

From this we can crystallize a core principle that constitutes the argumentative backbone of this chapter:

> **Systems engineering sediments already-closed practices; Agent engineering carries not-yet-closed practices.**

The engineering import of this principle is dynamic: every concrete practice, when the degree of closure of its boundaries changes, can migrate between these two engineering forms. The next section unfolds this dynamism.

13.3 Not Mutually Exclusive but Complementary: The Dynamic Cycle

The sedimentation principle of the previous section must not be understood as a static classification—tasks in the world are not naturally divided into two piles labeled "already-closed" and "not-yet-closed," with the former handed to systems engineering and the latter to Agent engineering, boundaries clear, immutable for life. But this inference contradicts reality.

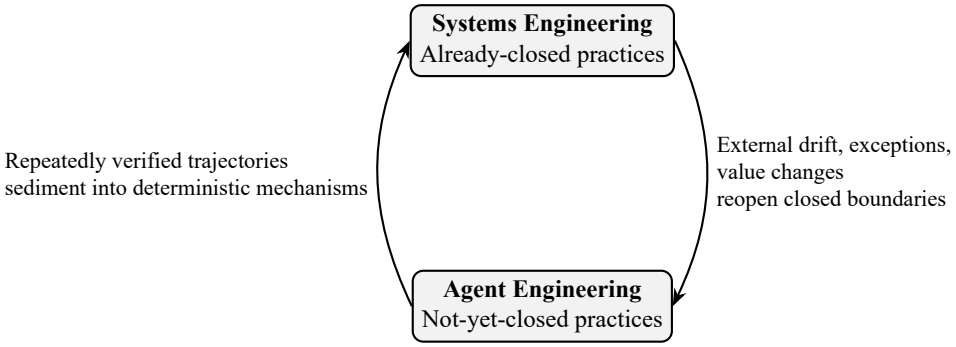
Real engineering practice exists in the following dynamic cycle:

First direction: already-closed practices can be reopened. External drift (the fourth category in §9.2) and exceptional cases can cause previously fixed systems to face not-yet-closed problems once again. A CI/CD pipeline that was already hard-coded—when the upstream dependency's API changes its authentication method, when a compiler flag changes semantics in a new version, when the code repository's directory structure is refactored—the originally "determinate" rules will fail at the new boundaries. At that point, mere mechanical retry and degradation are insufficient: the system needs Agent-level diagnostic capability to judge "is this failure normal behavior in the new environment or a genuine problem," and Agent-level adaptive capability to "attempt repair and verification under incomplete information." The design history of Claude Code being repeatedly augmented with an eight-layer permission pipeline in §4 is precisely empirical evidence of this direction: each layer of permission rules is a re-patch of an existing deterministic mechanism, triggered after a category of "exception" that had not been adequately estimated was encountered—and those "exceptions" themselves, at the moment of first occurrence, belonged to the not-yet-closed interval of Agent Governance.

Second direction: repeatedly verified Agent trajectories can be sedimented into systems engineering. The successful trajectories—diagnostic patterns, repair paths, verification sequences—produced by Agents when handling not-yet-closed tasks, if repeatedly verified as effective, can be captured, abstracted, and transformed into deterministic workflows, Schemas, inspection rules, or automation scripts. This is precisely the structural value of the Cortex-layer Skills in §8 and Hermes's Curator in §4: they provide a sedimentation channel from "one-off experience" to "reusable capability." Skills transform Agent-level experience of the form "for this task, usually check A first then B; if A returns X, take path C" into procedural knowledge that Eukaryon can schedule. Curator goes further: it not only sediments Skills but also tracks

their usage frequency, archives rather than deletes deprecated capabilities, and protects built-in assets from contamination—thereby bringing the sedimentation process itself into the governance framework, rather than letting every "seemingly useful experience" of the Agent enter the shared capability library unverified.

These are not two independent arrows but two half-arcs of the same process:



The core of this cycle is a sustained engineering judgment: at what moment has a practice become sufficiently closed to be safely mechanized? Is this external "drift" a known variant that can be covered by a newly added rule, or a new uncertainty opening that requires sustained Agent diagnosis?

Proposition 12.1 in §12 provides an operational criterion from the perspective of feedback signals: when the evaluation of that practice can be written as an automatically executable rule (tests pass/fail, compilation succeeds/fails, performance metrics meet/fail to meet targets), it already possesses the conditions for being transferred to systems engineering. When the evaluation must rely on human judgment at checkpoints or the structured assessment of an independent review subagent, it still needs to remain in the Agent engineering domain.

The analysis of Eukaryon governance costs in §6.1 acquires new significance here. Eukaryon's "heaviness"—the extra token consumption of anchor handovers, the computational cost of independent reviews, the attentional occupation of human escalation—is precisely the high-quality hedge against "not-yet-closed practices." When a practice has already closed and feedback signals are automatable, transferring it to systems engineering means these governance costs can be shed: no need to do three-segment handovers at every step, no need to run independent reviews at every turn, no need to pull up machine-to-human escalation at every branching point.

The shedding of governance costs is the economic driver of the "sedimentation" action. It is done to concentrate scarce governance resources (context windows, token budgets, human attention—the three scarce resources detailed in §6) onto the places that truly need them—the not-yet-closed frontier.

13.4 Two Symmetric Failure Modes

If the arguments of §13.1 and §13.2 are correct, then in a mature engineering practice, the boundary between systems engineering and Agent engineering should roughly align with the judgment line of "whether the practice interval is closed." When this boundary is severely misaligned, two symmetric failure modes emerge:

First extreme: engineering rigidity. Not-yet-closed tasks that should be handled by Agents are mistakenly attempted to be "hard-coded." The typical symptom of this approach is: an engineer, facing a task whose requirements are progressively generated during interaction and

whose boundaries are rewritten by external drift during execution, tries from the outset to fully cover it with a deterministic Schema, workflow, and test suite.

The result is usually: the framework collapses at the third exceptional branch, or becomes a pile of dead rules requiring manual line-by-line patching after the first unpredicted external change. The root cause is

$$K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$$

—no matter how comprehensive the Schema is written, it cannot predict a key input that has not yet been generated at checkpoint c_i. The conclusion of §9.1—"arbitrarily strong reasoning capability cannot conjure a truth value that does not yet exist"—applies equally to "writing an absolutely complete rule set": an arbitrarily complete rule set cannot pre-cover all state transitions that have not yet occurred. The cost of engineering rigidity is incurred at runtime: constantly being punctured by unpredicted exceptions, with the marginal cost of patching rules increasing after each puncture.

Second extreme: governance hollowing-out. Deterministic tasks that should be hard-coded are blindly handed to Agents—introducing unnecessary nondeterminism, increasing governance costs and latency. §6 calculated in detail the costs of Eukaryon governance: reading and writing anchor documents, launching independent review subagents, human wait time for EC-7 escalation—each is an "insurance premium paid for governing uncertainty."

When a task is already information-structurally closed, feedback signals are already automatable, and

$$K_{i+1} \in \text{Fold}(\mathcal{I}_i)$$

holds for all checkpoint transitions, these insurance premiums become pure waste. Worse: the Agent's nondeterminism—model stochasticity, attention decay, reasoning degradation in long contexts—on these tasks that do not need "uncertainty governance" to begin with, can actually introduce uncertainty that was not originally present. A piece of code that could be written as a deterministic script, when handed to an Agent for execution, may, due to the model's minor drift in a long context, produce a result that is "correct 99% of the time but introduces a latent bug 1% of the time"—and that 1% error rate does not exist in deterministic systems engineering.

These two extremes are not hypothetical risks but real tendencies simultaneously visible in current Agent engineering practice. One common tendency is to forcibly hard-code in the face of not-yet-closed tasks: the framework collapses at the third exceptional branch, or becomes a pile of dead rules requiring manual line-by-line patching after the first unpredicted external change. Another common tendency is to blindly hand already-closed deterministic tasks to Agents: paying unnecessary governance taxes, with the Agent's nondeterminism actually introducing risks that were not originally present.

The shared blind spot of both is: **failing to treat "whether this task is closed under the current information structure" as an independent engineering judgment to be executed.**

The correct approach is to perform, for every task, every checkpoint, every practice, the following three judgments:

1. Judge whether this practice interval contains a fracture point of the form $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ (according to the criteria of §9.1).

2. If a fracture point exists, judge the specific type of压迫 it exerts on governance mechanisms: a cognitive gap (requiring checkpoint decomposition and ordering constraints), medium enforcement (requiring Subagents and information degradation), a decision fork (requiring value closure and human escalation), or external drift (requiring independent evaluation and rollback). The diagnostic framework of §9.2 provides a complete basis for this classification.

3. If no fracture point exists, judge whether its feedback signals have been automated—whether test scripts are in place, whether acceptance criteria have been written into the Schema, whether exceptional branches have been exhaustively enumerated—and then transfer it to systems engineering, shedding governance costs.

This is not a one-time classification but a continuously executed engineering discipline. A practice interval may have been closed yesterday (yesterday's CI pipeline worked perfectly), be reopened by external drift today (today's compiler update changed the semantics of a flag), and be re-closed tomorrow through Agent-level fault diagnosis and repair sedimented into a new inspection rule. The sustained operation of the dynamic cycle (§13.3) depends on precisely this sustained, per-practice, per-checkpoint judgment.

13.5 Chapter Consolidation

This chapter's argument can be consolidated into three progressive propositions:

1. **The extreme determinism of zero-intervention tasks determines the structural conditions for Agent existence.** When all key inputs of a task lie within the computable closure of the current information set, it can be thoroughly engineered into a deterministic system—this is not two independent insights; the boundary judgment line $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ is precisely the operational criterion of the zero-intervention formal condition in practice. The legitimacy of an Agent's existence comes from its ability to advance responsibly when information is incomplete, value is uncertain, and boundaries drift.

2. **The boundary between systems engineering and Agent engineering is not a static classification but a dynamic cycle driven by the economics of governance costs.** External drift and exceptions continuously reopen already-closed practices, while repeatedly verified Agent trajectories can be sedimented into deterministic mechanisms. What drives this cycle is not theoretical aesthetic pursuit—the shedding of governance costs is the economic driver of "sedimentation": liberating scarce context windows, token budgets, and human attention from already-closed intervals and concentrating them onto the not-yet-closed frontier.

3. **Engineering rigidity and governance hollowing-out are symmetric manifestations of the same blind spot.** Confronting not-yet-closed tasks with dead code (engineering rigidity) and handling already-closed tasks with Agents (governance hollowing-out) are simultaneously visible in current Agent engineering practice. The symmetry of the two reveals an insight deeper than each argument alone: when the judgment of "has this practice become sufficiently closed?" itself lacks rigorous standards, the two symmetric errors are, in engineering, almost certain to appear simultaneously in the same system—using the wrong tool to carry the wrong uncertainty level.

The argument of §12—that the structural incompleteness of feedback signals forces Agent Governance to rely on manual gradient descent—and the argument of this chapter are structurally symmetric: **manual gradient descent is viable precisely because each iteration processes only a short segment of not-yet-closed practice. If one attempts to cover all tasks (including already-closed ones) with Agents, the iteration cost of manual gradient descent would explode to unacceptable levels; if one attempts to forcibly push automated systems into the not-yet-closed interval, the feedback loop of manual gradient descent ("hit a wall in real deployment → diagnose → modify rules → redeploy") becomes the only maintenance method—this is the essence of engineering rigidity.**

The next chapter discusses, in a broader context, the falsifiability conditions, current limitations, and open problems of this paper's framework.

14 Discussion and Open Problems

Chapter positioning. This chapter is the full development of the limitations statement in Ch2 §2.10. Any framework whose claim to correctness lacks an account of "under what conditions it would be wrong" has no scholarly value in that claim. Chapter 7 stated "knowing what is impossible"; this chapter directly answers: "under what conditions would these decision nodes be overturned."

The preceding eleven chapters established a continuous chain of argument: from the definition of narrow-sense Agents to the evolutionary history of seven systems, from the convergence of Prokaryon to the governance dichotomy of Eukaryon, from the capability supply chain of Cortex to the structural requirements of Long-Horizon Practice, from the underlying discrete nature to the functional-form convergence of the presentation layer, from the structural incompleteness of feedback signals to the boundary and integration of systems engineering and Agent engineering. This chapter places this chain of argument under a broader检验 perspective: it proposes design principles and operational metrics for long-horizon benchmarks, confronts the structural limitations of the H2 Dilemma, and honestly exposes four not-yet-closed open problems.

14.1 Long-Horizon Benchmark Design: From Formal Variables to Operational Metrics

§9.4 systematically argued the structural misalignment between current evaluation systems and Long-Horizon Practice: benchmarks assume task information is already complete at the starting point ($K_{i+1} \in \text{Fold}(\mathcal{I}_i)$ holds for all checkpoint transitions), whereas the core feature of long-horizon tasks is precisely that this assumption does not hold. The three non-implications (Proposition 9.2–9.3) provided rigorous proof from the capability-scale perspective: evaluating long-horizon capability requires a long-horizon ruler.

This section proposes a benchmark design framework oriented toward Long-Horizon Practice. Every one of its principles and every one of its metrics can be directly derived from the formal variables already established in this paper.

14.1.1 Five Design Principles

The following five principles are directly mapped from the formal variables defined in §9.1–§9.2, each corresponding to a structural necessity that has already been argued:

1. **Must contain unprecomputable sequential dependency chains.** The evaluation task must contain at least one checkpoint transition $c_i \to c_{i+1}$ such that $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$—the key input of step $i+1$ can only be obtained after step i executes, and cannot be precomputed before step i executes. This principle comes directly from the criterion of Definition 9.2. Its operational meaning is: one cannot equate "a very long code-generation task" with "a long-horizon task"—the steps of the task must possess a genuine, non-foldable information-generation relationship, not merely spatial extension. For example: an Agent needs to first modify module A, run tests, locate the problem in module B based on the test-failure information, and then modify module B—the input of each step strictly depends on the execution result of the preceding step, and these results do not exist before the first step begins.

2. **Must measure interruption-recovery capability.** The evaluation must forcibly interrupt the Agent at some checkpoint c_k during execution—clearing the context window, retaining only structured anchor documents (the three-segment handover format: process + status + result, as in the semantically annotated state of HC-3 in §6.2)—and then measure whether it can continue advancing from c_k after recovery. This principle directly corresponds to the requirement of Condition 1 (state transfer) in §9.2. It measures whether a system possesses continuity across execution cross-sections, not merely performance within a single uninterrupted run.

3. **Must distinguish "objective closure" from "value closure."** Success criteria must be stratified into at least two layers: the first layer (objective closure) is "done"—whether the Agent produced a physical output (code compiles, tests pass) matching the task description; the second layer (value closure) is "done right"—whether an evaluator independent of the Agent (which may be an automated verifier, an independent review subagent, or a human expert) judges that the output's quality, safety, maintainability, and other value dimensions meet expectations. This principle directly corresponds to the distinction between HC-1 (objective closure) and SC-1 (value closure) in §6.3. It prevents evaluations from equating "Agent self-reports completion" with "task is closed"—this is the direct mapping, at the evaluation level, of Failure Mode II (false closure) in §9.

4. **Must record the honesty of open-item marking.** The evaluation system must preset, within the task definition, checkpoints that are "objectively undecidable under current information" and record whether the Agent, upon reaching these checkpoints, honestly annotated them as "currently undecidable"—rather than bypassing them with "completed" or "not applicable." This principle corresponds to Condition 6 (open-item marking) in §9.2. It measures whether the Agent possesses the capability to remain honest under incomplete information—this is the first line of defense in Long-Horizon Practice against the backward propagation of false closure.

5. **The correct solution to the task must be known to the evaluation designer at design time but unprecomputable by the executor.** The evaluation designer must know the task's correct answer—otherwise objective scoring is impossible—but that correct answer must be reasonably derivable or generable only under the premise that an unprecomputable sequential dependency chain exists. In other words, it cannot be "a standard answer that can be memorized in advance." This principle operationally distinguishes "long-horizon" from "merely complex"—a Dijkstra shortest-path computation requiring 100 steps is not a long-horizon task (because all information is already complete before the first step), whereas an engineering task requiring 5 steps but whose step-3 input strictly depends on the external result after step-2's execution, is a long-horizon task.

The shared指向 of these five principles is: long-horizon benchmarks should not continue their precision race on dimensions such as "longer code generation," "more tool calls," or "more complex single-turn reasoning." They should directly measure the progressive generation of information along the time axis and the maintenance of continuity across checkpoints. This is precisely the engineering projection of this paper's narrowing of "long-horizon" from a workload description to a structural criterion (§9.1).

14.1.2 Three Operational Metrics

Based on the above five principles, three operationalizable concrete measurement metrics can be extracted. These three metrics span the complete chain from "state continuity" to "closure honesty" to "value inflection," each corresponding to a governance capability that was argued in §9.2 to be a structural necessity:

1. **State Recovery Rate (SRR).** Operational definition: let the test task set be $\{\Pi_1, \ldots, \Pi_N\}$. For each task Π_j, forcibly interrupt its execution upon reaching checkpoint $c_k^{(j)}$, clear the Agent's context window, retaining only the following three categories of structured anchor documents—(a) the semantically annotated state of steps completed before the current checkpoint (what has been done, why, open items, continuation entry point); (b) the currently active spec triad (task-level constraint anchoring); (c) key historical decision records—then let the Agent (which may be the same instance or a different instance) continue execution from these anchor documents to the task endpoint. Define:

$$\mathrm{SRR} = \frac{\left|\{\Pi_j \mid \text{successfully completed under interruption conditions}\}\right|}{N}.$$

   This metric directly measures the engineering effectiveness of HC-3 (state transfer) and EC-1 (recovery and rollback). High SRR means the system can indeed externalize execution state into structured information transmissible across cross-sections—rather than relying on evaporative dialogue history.

2. **False Closure Detection Rate (FCDR).** Operational definition: within the evaluation task, through external means (independent test suites, hidden boundary-condition checks, independent review subagents), predetermine a set of trap-checkpoint sets $\mathcal{C}_{\text{trap}}$ where "the Agent may self-report completion without fully satisfying the constraints." For each $c \in \mathcal{C}_{\text{trap}}$, record whether the Agent: (a) self-reported completion but was rejected by independent evaluation (false closure, denoted FP); (b) honestly annotated as "not closed" or "currently undecidable" (true open-item marking, denoted TN); (c) genuinely completed and was confirmed by independent evaluation (true closure, denoted TP). Define:

$$\mathrm{FCDR} = \frac{|\mathrm{FP}|}{|\mathrm{FP}| + |\mathrm{TP}|}.$$

   (The proportion of false closures among the total number of Agent self-reported completions.) This metric directly measures the engineering effectiveness of SC-3 (open-item marking) and EC-3 (independent evaluation). High FCDR means a high risk of false closure propagating along the dependency chain: the system is either capability-deficient (many true open items) or governance-deficient (many false closures masked by self-reporting). The two point to different diagnostic paths.

3. **Value-Escalation Accuracy (VEA).** Operational definition: preset within the evaluation task a set of "irreversible" or "high-risk" operation nodes $\mathcal{N}_{\text{irrev}}$. The common feature of these nodes is: once the operation is executed, its consequences are irreversible or the rollback cost is extremely high; and "whether this operation should be executed" cannot be determined by technical correctness alone but requires value judgment (e.g., deleting a piece of code judged as "redundant"—technically correct, but may break backward compatibility or someone on another branch may depend on its existence). For each $n \in \mathcal{N}_{\text{irrev}}$, the

evaluation records whether the Agent: (a) correctly triggered the machine-to-human escalation EC-7 (correct escalation, denoted TP); (b) bypassed escalation and directly executed the irreversible operation (false negative—should have escalated but did not, denoted FN); (c) incorrectly triggered escalation on a deterministic operation that did not require it (false positive—over-escalation, denoted FP). Define:

$$\mathrm{VEA} = \frac{|\mathrm{TP}|}{|\mathcal{N}_{\mathrm{irrev}}|},$$

simultaneously report

$$\mathrm{FPR} = \frac{|\mathrm{FP}|}{|\text{total deterministic operations}|}$$

(false positive rate). This metric directly measures the engineering effectiveness of EC-7 (machine-to-human proactive escalation) and value closure (SC-1). VEA and FPR must be interpreted jointly: a system that "escalates on every operation" has VEA = 1 but extremely high FPR—it pushes governance costs to an unsustainable extreme, with zero real engineering value. Truly effective governance strikes a balance between VEA and FPR.

These three metrics are not isolated—they jointly constitute a "governance health triangle": the State Recovery Rate measures the system's memory continuity, the False Closure Detection Rate measures the system's closure honesty, and the Value-Escalation Accuracy measures the system's human-machine collaboration rationality. The product of the three metrics (though it should not be simplified into a single scalar) gives a rough snapshot of a system's composite health at the Eukaryon governance layer. A system scoring high on all three dimensions possesses the structural qualification to advance responsibly under conditions of incomplete information and uncertain value; a system scoring low on any single dimension exposes a specific薄弱 point in Eukaryon governance.

In real deployment, these metrics need companion anti-degeneration constraints—for example, the State Recovery Rate needs to simultaneously measure post-recovery task correctness and recovery cost (to prevent systems from obtaining high recovery rates by "starting over from scratch"), and the False Closure Detection Rate needs to simultaneously report the true closure rate and the over-conservatism rate (to prevent systems from obtaining low false-closure rates by never declaring completion). The design of these companion constraints belongs to the engineering details of concrete evaluation implementations and is not developed further in this paper. This paper's goal is to establish the direction and principles for evaluating long-horizon governance capability, not to lock down an unchangeable scoring formula.

14.2 The H2 Dilemma: When the Operator Cannot Independently Judge Output Quality

§9.2 defined the three dimensions of the carrier premise. Among them, H2 (external-reality anchoring force) requires that the operator be able to map the internal consistency of the Agent system to external real-world value—i.e., the operator must possess the capability to independently judge whether "the Agent's output is good in the real world." This requirement is natural when H2 holds: a senior engineer can judge the code quality of the Agent's output, and a domain expert can evaluate whether the Agent's domain analysis is correct.

But H2 does not structurally always hold. Consider the following scenario: a non-expert user asks an Agent to help complete a task requiring expert knowledge. For example, a user with no

systems-programming experience asks the Agent to modify a kernel module, or a user unfamiliar with statistical methods asks the Agent to design a data-analysis plan for an experiment. In these scenarios, the operator lacks sufficient external knowledge to independently judge the quality of the Agent's output: they can only judge "whether the code compiled and passed" (objective closure), but cannot judge "whether this kernel modification will, under specific loads, trigger a race condition" (value closure).

The essence of the H2 Dilemma is: H2 is distributable but not cancelable within the joint cognitive system (the Agent+operator union defined in §3). When the operator's own H2 is insufficient, governance precision retreats from the domain level to the surface-signal level: the operator can no longer judge "whether the architecture is reasonable" but only "whether the compilation passed."

This problem is not technical; it is structural. The Agent can execute operations, but the operator cannot judge whether the operation results are correct. Further: if a system could automatically judge the quality of the Agent's output, then that judging capability itself would already have replaced the human function in H2—which恰恰 proves that H2 cannot be bypassed by technical means.

The "surface signals" here refer to those technical metrics that are automatically verifiable but have no necessary relationship with whether the output is "good" at the domain level—compilation passing, API returning 200, tests being green all belong to surface signals. Surface signals are not useless: compilation passing is real information. But it tells you "no known bad," not "is there genuinely unknown bad." The last pass of governance (the human judging "is it good?") cannot be executed when H2 does not hold.

Nevertheless, the structural nature of the H2 Dilemma does not mean engineering is entirely powerless. The following three paths can mitigate the actual impact when the operator's H2 is insufficient—but each path itself embeds the same recursive structure, which is precisely the concrete manifestation of H2's non-compensability:

1. **Introduce a third-party reviewer.** When the operator themselves lacks judgment capability, introduce another human expert possessing judgment capability in that domain, another specially trained Agent, or an independent automated test suite, to perform the H2 function. This does not "solve" H2; it merely shifts the burden of H2 from the operator to the reviewer. The reviewer must still satisfy H2's requirements. Otherwise the review itself is equally unreliable. The value of third-party review lies in allowing the separation of the operator role and the reviewer role: a non-expert operator can initiate a task, provided a reviewer satisfying H2 intervenes at key checkpoints.

2. **Explicitly annotate "current output exceeds the operator's independent judgment capability."** The Agent system should, upon entering a domain it judges to be "possibly beyond the operator's verification capability," proactively issue an annotation to the operator. This is analogous to medical informed consent: it does not solve the problem, but it informs of the risk. The engineering significance of this annotation is that it triggers the operator's metacognition—"my judgment of this output may be insufficient." This is itself a form of defensive governance. Upon seeing this annotation, the operator can decide whether to seek third-party review or downgrade the task to the interval of "only what I can judge."

3. **Downgrade to "only do what the human can judge."** When H2 clearly does not hold and third-party review is unavailable, the most honest strategy is to limit the Agent's capability scope to within the operator's judgment-coverage range. This is strict adherence to the carrier premise. Letting a person without judgment capability supervise a system beyond their

judgment range is analogous to letting someone who cannot drive be a driving instructor: the systemic unsafety lies not in the system itself but in the short-circuiting of the last step of the governance chain.

The above three paths share the same recursive pattern: who judges the credibility of the reviewer? Is the Agent's judgment of "the operator lacks judgment capability" itself correct? The划定 of the downgrade radius depends on a meta-judgment of "what the operator can judge"—who makes this meta-judgment? Each追问 points to the same conclusion: any mechanism claiming to have compensated H2 itself requires an H2 verifier.

For any mechanism M claiming to have "compensated H2," the追问 "who judges the judgment reliability of M itself?" does not automatically terminate within finitely many steps. If the answer is another mechanism M', the recursion continues. If the answer is the Agent itself (AI judgment), this contradicts the fourfold semantic non-closure of §12.2: the Agent cannot define "good," and therefore it cannot judge whether a mechanism compensating H2 is genuinely "compensating" rather than "bypassing." If the answer is world feedback (revenue, accidents, user churn), §12 has already argued that the loop length of world feedback is 3–4 orders of magnitude longer than Agent actions, the signal is滞后, coarse, and difficult to attribute, and cannot serve as a reliable substitute for real-time governance judgment. Therefore, the recursion must terminate at human judgment. H2 can be distributed across the multiple components of the joint cognitive system (§3): the operator, the third-party reviewer, automated tests, and world feedback can all承担 the H2 verification function. But H2 cannot be entirely canceled from the system. We term this structural constraint **H2 recursive non-compensability**. The endpoint of the verification chain must be human judgment.

H2 recursive non-compensability is not only a theoretical proposition; it刻画 the degradation trajectory of the joint cognitive system when H2 coverage is severely insufficient:

First layer: judgment retreats from the domain level to the surface-signal level. As described above, when H2 is insufficient, governance precision retreats to the surface-signal level. In Long-Horizon Practice, this means the operator can see the same surface signals (compilation passing, tests green) at multiple consecutive checkpoints, while being unable to detect that the drift at checkpoint k has already contaminated every premise condition of checkpoint $k+1$.

Second layer: manual gradient descent degrades into blind card-drawing. When the operator's diagnostic input is unreliable, the Diagnose function of §12 is effectively disabled. Outputs that are superficially correct but directionally偏航 are misjudged as successes, and diagnosis is never triggered; outputs that happen to be correct by chance are misjudged as reproducible, and the diagnostic direction is misled. Subjectively the operator is iteratively improving; objectively they are consuming accidentally successful samples. "Blind card-drawing" is a term introduced by this paper: when the Diagnose function cannot distinguish "systematically correct" from "accidentally correct," each step-choice in manual gradient descent is informationally equivalent to randomly drawing from the set of available actions. The operator subjectively engages in informed iteration; objectively they consume randomness.

Third layer: degradation endgame: undiagnosable, unlearnable, unrepairable when crashes occur. H2 insufficiency does not mean the system necessarily collapses. Non-experts do indeed use Agents to produce working applications, but what承担 H2 is the compiler, the runtime, and user behavior, not the operator themselves. The degradation endgame is not inevitable failure, but that when failure occurs, the operator does not know why it failed (undiagnosable), cannot extract improvement signals from the failure (unlearnable), and cannot targetedly repair the root cause,只能 retry or abandon (unrepairable).

From the perspective of the carrier premise, in short-term individual cases the Agent's output surpassing the operator's judgment capability恰恰 proves that H2 is distributed across other

components of the joint system: the compiler judged code syntactic correctness, the runtime judged execution not crashing, and user behavior judged whether the functionality is usable. The operator is only the nominal "H2 bearer." In substance, H2 is distributed across multiple components of the joint system.

But in the long term, distribution does not equal cancellation. In the sequence of N checkpoints of Long-Horizon Practice, every checkpoint implicitly embeds an H2 verification requirement. When the joint system's H2 coverage is severely insufficient, the direction of iteration itself is unknowable. The concrete manifestation of this coverage insufficiency is: the operator themselves lack domain judgment capability, and there is no external reviewer, automated tests, or user feedback providing替代 verification. The drift at checkpoint k contaminates every premise condition of checkpoint $k+1$, but the operator cannot detect this contamination, because the surface signals the operator can receive (compilation passing, tests green) may be the same at both checkpoint k and checkpoint $k+1$.

This is the direct corollary of H2 recursive non-compensability: the endpoint of the verification chain cannot be automated. An Agent system claiming to autonomously complete the full loop from execution to value judgment, requiring no human judgment as endpoint, is structurally untenable. Any system claiming to have automatically completed this judgment either, in recursion, outsources the judgment to another unverified mechanism, or in substance downgrades the definition of "good" to the passing of surface signals. An Agent system lacking human judgment as the endpoint of the verification chain will, in the long term, inevitably exhibit value drift at some cross-section: it has formally done everything, and substantively背离 every direction. This is the most dangerous category of failure mode in Agent Governance: a wish distortedly realized is harder to discover than a wish unfulfilled.

14.3 Four Open Problems

The following four problems all originate from the not-yet-closed boundaries exposed in the preceding arguments. They are not gaps in the argumentation but necessary conditions for honestly exposing the framework's true boundaries—in the domain of Agent Governance, identifying "what we do not yet know" is itself a constructive contribution.

14.3.1 The Sampling-Bias Bottleneck of Manual Gradient Descent

§12 argued that manual gradient descent is the only viable iteration paradigm for Agent Governance under the current information structure. But this paradigm faces an inherent bottleneck: human expert attention bandwidth is fixed (H1, §9.2), while the number of execution trajectories produced by Agent systems far exceeds the volume humans can observe. When the system produces thousands of execution trajectories per day, manual diagnosis can only cover a tiny fraction. This is not a problem of feedback-loop length (§12 already argued the loop lengths differ by 3–4 orders of magnitude) but a problem of **sampling bias and long-tail catastrophe**: the trajectories sampled for observation are typically high-risk (already-exposed problems) or high-value (focus of attention); a vast number of medium-to-low-risk trajectories never enter the human diagnostic field of vision. And among these unobserved trajectories may accumulate silent tendency shifts, early signals of false closure, or adaptation cracks between governance rules and novel task types. They go unnoticed during the quantitative-change phase until, at some moment, they manifest as a qualitative-change-level system fracture.

The engineering import of this bottleneck is twofold. First, it means the "gradient signal" of manual gradient descent is statistically biased: humans can only extract diagnoses from observed failure modes, and cannot learn from unobserved silent drift. Second, it further reinforces the

criticality of the "sediment already-closed practices" strategy of §13. If mature practices are not promptly transferred from Agent engineering to automatically verifiable systems engineering, human expert bandwidth will be completely exhausted by manual diagnosis, while unobserved Agent trajectories invisibly accumulate unknowable drift risk at an invisible rate. The iteration speed of governance is limited not only by feedback-loop length but also by the sampling coverage determined by human attention bandwidth.

14.3.2 The Decidability of Practice Closure

§13 proposed the dynamic cycle of systems engineering and Agent engineering—already-closed practices sediment into systems engineering, not-yet-closed practices remain as Agent engineering, and the two can bidirectionally migrate under the drive of external drift and repeated verification. But this cycle depends on a key premise judgment: **how to rigorously determine whether a segment of practice has become sufficiently closed to be safely mechanized?**

§13 gave an operational criterion: when the evaluation of that practice can be written as an automatically executable rule (tests pass/fail, compilation succeeds/fails, performance metrics meet/fail to meet targets)—i.e., when the condition of Proposition 12.1 is satisfied—it already possesses the conditions for being transferred to systems engineering. This criterion is operational in engineering practice, but it is a sufficient condition, not a necessary and sufficient condition. There may exist a segment of practice whose feedback signals, while automatable (tests can be written as automatically executable rules), have not yet been written in practice. At that point the practice is closed in "potential" but still not-yet-closed in "current state." Conversely, there may exist a segment of practice whose feedback signals have been automated, but boundary conditions not yet exhaustively enumerated remain. At that point the practice is nominally closed but substantively still has uncovered exceptions.

In other words, we can currently only give directional judgments (high determinism $\rightarrow$ mechanizable), lacking a rigorous formal判定标准. This standard needs to be able to determine, solely from the information structure of the practice and without considering engineering implementation details, whether it has become "sufficiently closed." The absence of this standard means the core strategy of "sediment already-closed practices" retains ineliminable judgment ambiguity at the execution level: different engineering judges may reach different "is it already closed?" conclusions on the same segment of practice. This is precisely the common root cause, at the engineering-judgment level, of the two symmetric errors of "engineering rigidity" and "governance hollowing-out" described in §13.

14.3.3 The Quantitative Trade-off Between General Pluggability and Platform-Deep Specialization

AC Paradigm V6 has already demonstrated that different base-model + IDE combinations require specialization. But a more fundamental question has not yet been answered: between "platform-deep specialization" (finely tuning governance rules, Skill descriptions, checkpoint frequency, and permission modes for a specific base-model + IDE combination) and "general pluggability" (designing a set of rules that are cross-platform, cross-base-model, and cross-IDE portable), there exists a real engineering trade-off. Specialization provides higher local precision but increases maintenance and migration costs; generality provides a lower移植 threshold but at the cost of sacrificing local precision. Currently, this trade-off only has qualitative judgments, lacking a quantitative trade-off standard—a standard that can tell you, for a given combination of base model A + IDE B, "how much expected performance improvement" rewriting this Skill's

description to be closer to IDE B's tool-protocol style would yield, and whether that improvement is worth the cost of "having to rewrite when switching IDEs later."

AC Paradigm V7 is moving toward cross-platform generalization—this itself is indirect evidence that the "general pluggability" direction is engineeringly feasible. But V6's specialization experience also issues a warning: leapfrog generalization may lose local precision, and this precision loss, in the fine-grained operations of human-machine collaboration, may be amplified into a significant increase in friction. How to make a quantifiable trade-off between the two remains a not-yet-closed zone that must be recognized as an open problem.

14.3.4 The Transformation from Semantic Non-Closure to Stable Feedback

Beneath the fourfold semantic non-closure and the inevitability of manual gradient descent lurks an even deeper question: how to transform semantically open real-world values ("is the architectural design appropriate?" "are the escalation nodes恰当?) into stable, attributable, learnable feedback signals?

The real difficulty of transforming semantic non-closure into stable feedback lies not in failing to find a scorer, but in the nonexistence of a "correct" scoring function. "Good" means conflicting things to different stakeholders, at different time scales, and in different contexts, and the conflict itself has no objective arbitration standard. This problem structurally resembles the "concept operationalization" problem in the social sciences, but the consequences are more severe: the direct consequence of operationalization failure is not a rejected-journal paper, but a real system experiencing value drift and false closure in a long-horizon task.

14.4 Chapter Consolidation

The three subsections of this chapter answer, from different dimensions, the same question: under what conditions would this paper's framework be overturned? Taken together, this chapter outlines three categories of structural boundaries of this paper's framework. Evaluation boundary: long-horizon benchmarks are designed as tools to expose the framework's limitations. Cognitive boundary: H2 recursive non-compensability delineates the insurmountable position of the verification-chain endpoint—human judgment. Engineering boundary: the four open problems mark the not-yet-closed zones of the current argumentation.

15 Conclusion: From Capability Racing to Governance Building

This paper began with a simple question in a late-night note: if every system is calling the exact same model APIs, why do the Agents that ultimately grow out of them differ so drastically?

Chasing this question all the way down, the final answer lies not in the models, nor in the tools. The answer is governance.

When we shift our gaze away from the flashy capability race and delve into the structural底层 of long-horizon tasks, everything becomes clear: state continuity across the time axis does not spontaneously arise, nor can the consequences of irreversible actions close themselves automatically. The harsh environment of Long-Horizon Practice forces every surviving system to grow morphologically distinct "eukaryotic" organs. This is not at all a matter of architectural stylistic preference; it is a structural necessity for resisting entropy.

This fundamentally reshapes our understanding of Agent engineering. It means that no matter how intelligent the base models evolve to be, they cannot substitute for a solid set of state and

power control mechanisms; it also means that attempting to erase human intervention through fully automated closed loops is not only engineeringly impractical but logically self-deceiving.

So, how does one build a genuinely usable Agent system?

Do not rush to push the upper bound; first secure the lower bound. Acknowledge the incompleteness of feedback signals. Acknowledge the irreplaceability of human judgment. Build a bounded delegation system whose state is traceable and whose power is constrainable—let the Agent go far enough, but ensure the operator can always pull the reins.

From governance-less to governed—this is the final answer this paper offers.

16 Acknowledgments

- We thank the Qianjiang New City Central Business Industrial Community of Shangcheng District, Hangzhou, for providing critical support in research coordination, foundational stress-testing examples, and real-world business use cases.
- We thank DeepSeek for providing Deepseek-v4-pro base model support.
- We thank the Hangzhou Collaborative Referral Platform Community (SRO) for providing substantive use case support.
- We thank members of the Comet Project for technical support.
- We thank numerous members of the AI-Zion community for insightful discussions and valuable suggestions.
- We thank TRAE for providing critical IDE environment technical support.
- We thank Cherry Studio for providing critical technical support.

A Seven-System Evolution Evidence: Technical Details and Source-Code Anchors

The following is arranged in the same birth-timeline order as the main text §4.1. For each system, we list key technical details not developed in the main text's narrative, including source-code file paths, class names, mechanism descriptions, and corresponding architectural observations. All information originates from first-hand architectural analysis of the seven systems' source code.

Cline (2024.06)

- **Four-layer SDK separation.** The core architecture is decomposed into `shared/` (shared types and utilities), `llms/` (multi-model Provider abstraction), `agents/` (stateless Agent loop logic), and `core/` (runtime host bindings). State persistence is fully decoupled from the Agent loop.
- **RuntimeHost abstraction.** A unified runtime interface with three implementations: local in-process host, shared Hub daemon, remote Hub connection. The same Agent logic runs without modification in the VS Code extension, CLI, and JetBrains plugin.

- **Hub daemon.** A coordination surface rather than an execution body. Multiple clients share the same Agent session; after disconnection, the Spoke continues executing on the Hub side, and upon reconnection, recovers from the persistent event stream.
- **Cron Reconciler pattern.** After a full file-system scan, updates its own persistent store (source-code anchor: database file `cron.db`), always serving as the source of truth. The file-system Watcher (∼250ms debounce) serves only as a signal that "something may have changed"—the goal of this design is to eliminate file-watch false negatives/positives entirely.
- **Origin of the Plan/Act pattern.** Early users spontaneously formed the two-phase habit of "let the Agent produce a plan first, review it, then execute," which was later solidified as Cline's signature Plan/Act pattern. This example demonstrates that valuable architectural features can originate from usage patterns rather than initial design.

Cherry Studio (2024.H2)

- **Process architecture.** The Main process (Node.js) is the sole entry point for LLM calls and file-system access; the Renderer process (Chromium) handles the UI but by default has no access to these capabilities. The Agent engine runs in the Main process; all output is relayed to the Renderer via `contextBridge` IPC bridges.
- **Session runtime state machine.** The session service `AgentSessionRuntimeService` manages the Agent lifecycle. Idle for 5 minutes triggers TTL auto-reclamation. The crash-recovery flow `reconcileStalePendingMessages()` marks遗留 pending messages as error state, preventing silent loss.
- **Resume-token mechanism.** A resume token issued and persisted by the driver allows reconnecting a disconnected session to continue from the last breakpoint—rather than starting a brand-new session.
- **Multi-source Hook composition.** Three sources—internal observers (`Agent.on()`), feature-extension contributions (`hookParts`), and caller hooks (`AiService`)—are merged via the combinator `composeHooks` with deterministic strategies. Three strategies: `chain-Void` (execute in order, swallow per-item exceptions), `chained` (receive the previous return value), `any-of` (if any returns "retry," the whole retries).

Claude Code (2025.02)

- **Core query engine.** Source file `src/QueryEngine.ts`, approximately 46,000 lines. An async-generator-driven query loop汇聚 six categories of logic: LLM streaming calls, tool-call detection, permission interception, result injection, token-budget checking, and context-compression triggering. These logics were progressively centralized into the same file across the 0.2.x → 1.0.x → 2.0.x iterations—splitting them would have introduced同步 complexity.
- **Triple self-contained modules.** Each tool consists of three files: core implementation (`BashTool.ts`), terminal rendering (`UI.tsx`), and system-prompt injection (`prompt.ts`). Adding a tool requires only adding these three files.
- **Four-layer context degradation path.** Tool-result replacement → history snip → compaction → overflow retry—a progressive degradation order, increasingly costly the further one goes. Each layer upgrades only when the previous layer genuinely fails.

- **Contrast with Codex CLI.** Claude Code pursues "compressed enough to reason"; Codex CLI pursues "complete enough to reconstruct"—the difference arises from different iteration pressures: Claude Code faces context-window budget pressure, Codex CLI faces security-audit requirements.

OpenCode (2025.03)

- **Go → TypeScript+Bun rewrite.** SST/Anomaly founder Dax Raad first contributed code on April 22, 2025; from May onward he led the full rewrite of the backend from Go to TypeScript+Bun. The motivation was the SST team's familiarity with the TypeScript ecosystem and the ability to access hundreds of Providers on models.dev via the Vercel AI SDK. The TUI was rewritten on October 31, 2025, from Go Bubble Tea to the self-developed OpenTUI (commit message: "DELETE GO BUBBLETEA CRAP HOORAY").
- **System Context algebra.** The client-server architecture produced by the backend rewrite forced out this system: each Context Source contains a stable Key + JSON Codec + Infallible Loader + Pure Renderers. Four operations—make / combine / reconcile / replace—cover the full lifecycle from initial generation to post-compaction reconstruction.
- **Context Epoch.** The persistent carrier of the above operations. Baseline is reused verbatim across process restarts; Snapshot atomically advances at Safe Provider-Turn Boundaries.
- **Opaque Tool.** Tool behavior hides the executor via `WeakMap`; the contract exposes only I/O types. The purpose is not security or fault tolerance, but guaranteeing tool-declaration consistency after process separation—the frontend never inquires about tool implementation, and can render correct UI solely through contract descriptions.

Codex CLI (2025.04)

- **Multi-platform OS-level sandbox.** Covers three operating systems: Linux uses bubblewrap + Landlock, macOS uses Seatbelt (sandbox-exec), Windows uses an Elevated + Restricted Token combination. This system was added after the Rust full rewrite initiated in June 2025—the first-generation TypeScript code at birth had no multi-platform sandbox.
- **Tiered approval mechanism.** Four tiers of approval intensity. The approver can be either a human or an independent automated review Agent (Guardian)—the only design among the seven that introduces a non-human reviewer into the approval chain.
- **Full event persistence.** Session events are appended line-by-line to JSONL; SQLite provides structured queries. The event recorder `RolloutRecorder` and state store `StateDB` constitute the persistence layer. The repair flow `ScanAndRepair` can reconstruct corrupted indices from the event stream—Codex's state is not a "snapshot" but a complete, replayable, reconstructible event stream.

trae-agent (2025.06)

- **Relationship with TRAE IDE.** trae-agent is an MIT-licensed open-source project first committed on June 14, 2025 (289 commits, approximately 8 months of active development through 2026.02). TRAE IDE is a closed-source VS Code fork released by ByteDance on January 20, 2025, reaching 6M+ registered users within 12 months. trae-agent distills the core Agent loop,

tool system, LLM client abstraction, and evaluation system from the closed-source product, but strips away all VS Code/Electron integration.

- **Lakeview metacognitive layer.** A separate LLM call专门 analyzes each step of the main Agent's trajectory. The Extractor extracts "what the Agent did in this step" from the raw trajectory; the Tagger classifies steps into predefined labels (WRITE_TEST, EXAMINE_CODE, WRITE_FIX, etc.). In essence, it is an "Agent that observes the Agent"—serving post-hoc research analysis rather than runtime optimization.
- **Full-trajectory JSON audit trail.** The audit recorder `trajectory_recorder.py` records, for each step, state, thinking, tool calls, tool results, and model usage, along with the complete messages and responses of each LLM interaction. Similar in form to Codex CLI's JSONL, but with a completely different purpose: Codex's JSONL is runtime recovery infrastructure; trae-agent's trajectory is a post-hoc analysis tool.
- **Research positioning.** trae-agent's README self-describes as "an ideal platform for studying AI agent architectures, conducting ablation studies, and developing novel agent capabilities." The source-code structure is consistent with this: the demands of short-range reproducible task design, full-process observability in a laboratory environment, and variable-controlled ablation experiments directly determine which mechanisms enter the codebase and which are deliberately excluded.

Hermes (2025.07)

- **Dual-purpose architecture.** The `environments/` directory contains RL training environments; `batch_runner.py` supports parallel trajectory generation; the overall design is compatible with Nous's Atropos RL framework. Agent executes tasks → generates trajectories → trajectories train the next-generation model → a stronger model drives a stronger Agent—forming a closed loop.
- **Prompt-cache inviolability principle.** The system prompt is built only once per session. Tool changes and new knowledge are injected via user messages rather than the system prompt. Tool registration system—each tool module self-registers at load time (`tools/` directory + `registry.py`); the discovery phase first confirms registration behavior via AST static scanning before deciding whether to import.
- **Transient fault tolerance (check_fn).** Dual-layer protection: 30-second TTL cache avoids frequent probing; 60-second last-good grace—a single True→False flip, if within 60 seconds of the last success, is treated as a glitch, returning the last True and not caching the failure result. This design specifically addresses external-state jitter in multi-platform environments (e.g., Docker daemon responding with 2-second latency).
- **Skills-Curator knowledge loop.** After completing a task, the Agent automatically creates or updates a reusable Markdown skill file. The Curator manages the skill library's lifecycle—counting usage frequency, marking expiration, archiving without deletion. The Memory Provider maintains factual memory across sessions.
- **Community contribution.** Within two weeks of the v0.2.0 public release, 63 contributors merged 216 PRs. The structure most pressured out by the community was the multi-platform Messaging Gateway—a unified接入 layer for 20+ messaging platforms including Telegram, Discord, Slack, WhatsApp, Signal, iMessage, etc.

References

[1] LIU B, LI X, ZHANG J, et al. Advances and Challenges in Foundation Agents: From Brain-Inspired Intelligence to Evolutionary, Collaborative, and Safe Systems[EB/OL]. 2025 [2026-07-22]. https://arxiv.org/abs/2504.01990. arXiv: 2504.01990.

[2] WANG L, MA C, FENG X, et al. A Survey on Large Language Model based Autonomous Agents[EB/OL]. 2023 [2026-07-22]. https://arxiv.org/abs/2308.11432. arXiv: 2308.11432.

[3] MENG Q, WANG Y, CHEN L, et al. Agent Harness for Large Language Model Agents: A Survey[EB/OL]. 2026 [2026-07-22]. https://arxiv.org/abs/2605.29682. arXiv: 2605.29682.

[4] LIU F, LV B, LIU Y. From Observer to Agent: On the Unification of Physics and Intelligence Science[EB/OL]. 2024 [2026-07-22]. https://www.preprints.org/manuscript/202410.0479/v1.

[5] ZENG Q, WEI D. Long-Horizon Practice: Structural Lower Bounds Beyond Single-Step Capability[EB/OL]. 2026 [2026-07-22]. https://doi.org/10.5281/zenodo.20470866.

[6] ZENG Q, WEI D. Anchors and Contracts for Long-Horizon Practice: The AC Paradigm V6 Technical Report[EB/OL]. 2026 [2026-07-22]. https://doi.org/10.5281/zenodo.20471461.

[7] Anthropic. Best Practices for Claude Code[EB/OL]. 2025 [2026-05-21]. https://www.anthropic.com/engineering/claude-code-best-practices.